



UNIVERSIDAD
DE GRANADA

IMPUTACIÓN DE DATOS FALTANTES
DE VARIABLES METEOROLÓGICAS Y DE CONTAMINACIÓN
EN SERIES TEMPORALES MULTIVARIANTES IOT
CON MODELOS TRANSFORMER

RICARDO RUIZ FERNÁNDEZ DE ALBA

Trabajo Fin de Grado

Doble Grado en Ingeniería Informática y Matemáticas

Tutores

María del Campo Bermúdez Edo

Daniel Bolaños Martínez

FACULTAD DE CIENCIAS

E.T.S. INGENIERÍAS INFORMÁTICA Y DE TELECOMUNICACIÓN

Granada, a 16 de junio de 2025

Imputación de Datos Faltantes de Variables Meteorológicas y de Contaminación en Series Temporales Multivariantes IoT con Modelos Transformer

Ricardo Ruiz Fernández de Alba

Ricardo Ruiz Fernández de Alba. *Imputación de Datos Faltantes de Variables Meteorológicas y de Contaminación en Series Temporales Multivariantes IoT con Modelos Transformer*.

Trabajo de fin de Grado. Curso académico 2024-2025.

Responsables de tutorización	María del Campo Bermúdez Edo <i>Departamento de Lenguajes y Sistemas Informáticos</i>	Escuela Técnica Superior de Ingeniería Informática y Telecomunicación
	Daniel Bolaños Martínez <i>Departamento de Lenguajes y Sistemas Informáticos</i>	Facultad de Ciencias Doble Grado en Ingeniería Informática y Matemáticas Universidad de Granada

This work is licensed under a [Creative Commons](#) “Attribution-NonCommercial-ShareAlike 4.0 International” license.



The source code of this text and developed programs are available in the Github repository <https://github.com/eigenric/TFG>

DECLARACIÓN DE ORIGINALIDAD

D. Ricardo Ruiz Fernández de Alba

Declaro explícitamente que el trabajo presentado como Trabajo de Fin de Grado (TFG), correspondiente al curso académico 2024-2025, es original, entendida esta, en el sentido de que no ha utilizado para la elaboración del trabajo fuentes sin citarlas debidamente.

En Granada a 16 de junio de 2025

Fdo: Ricardo Ruiz Fernández de Alba

A mi familia, por su apoyo constante durante todo este trayecto académico.
A mis compañeros Blanca y Jose, por su amistad y por acompañarme en los momentos importantes.
A todos los que han estado cerca, compartiendo esfuerzos, risas y aprendizajes.
Y a mis tutores, por su dedicación, claridad y paciencia a lo largo del desarrollo de este trabajo.

ÍNDICE GENERAL

Glosario de Acrónimos	10
I. BLOQUE I: INTRODUCCIÓN, OBJETIVOS Y PLANTEAMIENTO DEL PROBLEMA	14
1. INTRODUCCIÓN	15
1.1. Motivación	15
1.2. Descripción del Problema	16
1.3. Objetivos del Trabajo	18
1.4. Estructura del Trabajo	19
II. BLOQUE II: FUNDAMENTOS TEÓRICOS	21
2. PRINCIPIOS MATEMÁTICOS	22
2.1. Álgebra Lineal y Geometría	22
2.1.1. Vectores y Espacios Vectoriales	22
2.1.2. Matrices y Operaciones Matriciales	23
2.1.3. Tensores y Normas	26
2.2. Análisis Matemático	28
2.2.1. Funciones y Continuidad	28
2.2.2. Cálculo Diferencial Multivariable	28
2.3. Análisis Funcional	31
2.4. Probabilidad y Estadística	33
2.4.1. Estadística Descriptiva	33
2.4.2. Fundamentos de Probabilidad	34
2.4.3. Distribuciones de Probabilidad Discretas Comunes	35
2.5. La Función Softmax	37
2.5.1. Apunte sobre estabilidad numérica	38
2.6. Métodos Numéricos	39
2.6.1. Interpolación lineal	39
2.6.2. Limitaciones de la Interpolación Lineal	41
2.7. Teorema de Aproximación Universal	41
3. DEL APRENDIZAJE AUTOMÁTICO AL APRENDIZAJE PROFUNDO	45
3.1. ¿Qué es el Aprendizaje Automático?	45
3.2. Paradigmas del Aprendizaje Automático	46
3.2.1. Aprendizaje Supervisado	46
3.2.2. Aprendizaje No Supervisado	47
3.3. Evaluación de Modelos y Tratamiento de Datos Faltantes	48
3.3.1. Métricas de Evaluación	49
3.4. De Modelos Lineales a Arquitecturas de Atención Profunda	52

3.4.1.	Modelos Lineales Fundamentales	52
3.4.2.	De Perceptrones Simples a Redes Neuronales Profundas	53
3.4.3.	Redes Neuronales Profundas con atención: Transformer	53
3.4.4.	Self-Attention-based Imputation for Time Series (SAITS)	61
III. BLOQUE III: METODOLOGÍA Y CASO DE ESTUDIO: IMPUTACIÓN DE DATOS		66
4.	IMPUTACIÓN DE DATOS EN SERIES TEMPORALES	67
4.1.	Definición del Problema de Imputación Experimental y Evaluación . .	67
4.2.	Métodos de Imputación en Series Temporales	68
4.3.	Métricas de Evaluación para los Métodos de Imputación	72
4.4.	Aprendizaje Profundo para la Imputación en Series Temporales	73
5.	IMPUTACIÓN DE DATOS DE TRÁFICO Y CONTAMINACIÓN	75
5.1.	Infraestructura del Sistema IoT	75
5.2.	Conjuntos de Datos (Datasets)	76
5.3.	Análisis Exploratorio de Datos (EDA)	80
5.3.1.	Análisis del Porcentaje de Valores Faltantes	81
5.4.	Entorno Experimental y Flujo de Trabajo	85
5.4.1.	Estructura del Paquete pampaneira_imputation	85
5.4.2.	Flujo de Trabajo y Experimentación	86
5.4.3.	Control de Versiones y Pruebas Unitarias	87
5.5.	Preparación de Datos y Configuración Experimental	87
5.5.1.	Carga, Estructura y Preprocesamiento Inicial	87
5.5.2.	División, Escalado y Creación del Escenario de Evaluación . .	88
5.6.	Implementación de las Funciones de Imputación	89
5.6.1.	Métodos Estadísticos Simples	89
5.6.2.	Métodos Basados en Propagación e Interpolación (Locales) . .	90
5.6.3.	Métodos basados en Matrices	90
5.6.4.	Métodos Basados en Modelos Predictivos	91
5.7.	Visualización de Series Temporales Imputadas	92
5.8.	Evaluación del Rendimiento	94
5.9.	Validación en otro dataset de contaminación de la literatura	106
6.	CRONOGRAMAS Y PRESUPUESTO	109
6.1.	Planificación del Trabajo y Análisis del Presupuesto	109
6.1.1.	Planificación del Trabajo: Estimación vs. Realidad	109
6.2.	Cronogramas	110
6.3.	Metodología Utilizada	111
6.4.	Análisis del Presupuesto del Proyecto	111
6.5.	Presupuesto Estimado	112
IV. BLOQUE IV: CONCLUSIONES Y TRABAJO FUTURO		113
7.	CONCLUSIONES Y TRABAJO FUTURO	114
7.1.	Conclusiones Principales	114

7.2.	Cumplimiento de Objetivos	115
7.3.	Limitaciones del Estudio	116
7.4.	Trabajo Futuro	117
7.5.	Conclusión Final	117

GLOSARIO DE ACRÓNIMOS

Bfill	Backward Fill 68 , 94 , 104 , 106
DL	Deep Learning 12 , 13 , 15
DMSA	Diagonally-Masked Self-Attention 62
Ffill	Forward Fill 68 , 94 , 104 , 106
FFNN	Feed-Forward Neural Network 53
GAN	Redes Generativas Antagónicas 17
GRU	Gated Recurrent Unit 71
IA	Inteligencia Artificial 15
IoT	Internet of Things 12 , 13 , 15
LOCF	Last Observation Carried Forward 68
LPR	License Plate Recognition 75
LSTM	Long Short-Term Memory 18
MAE	Error Absoluto Medio 94–96 , 105
ML	Machine Learning 15
MRE	Error Relativo Medio 94–96 , 105
MSE	Error Cuadrático Medio 94–96 , 105
NOCB	Next Observation Carried Backward 68
ODS	Objetivos de Desarrollo Sostenible 16
RMSE	Raíz del Error Cuadrático Medio 94–96 , 105
RNN	Red Neuronal Recurrente 18

SAITS Self-Attention-based Imputation for Time Series [12](#),
[13](#), [92](#), [94](#), [104–106](#)

TFG Trabajo de Fin de Grado [12](#)

RESUMEN

Este Trabajo de Fin de Grado (TFG) se centra en la generación de datos sintéticos para sustituir valores faltantes, en series temporales multivariantes, en el contexto de sistemas IoT. El propósito es implementar y evaluar la eficacia de arquitecturas de aprendizaje profundo, especialmente modelos transformer como SAITS, para la imputación de datos. La investigación se aplica a un caso de estudio de monitorización de tráfico y contaminación del proyecto Smart Poqueira, buscando restaurar la completitud de los datos.

La metodología se fundamenta en las siguiente ramas de las **matemáticas**: álgebra lineal, métodos numéricos, análisis, probabilidad y estadística; y **Deep Learning (DL)**, complementada con un estudio del estado del arte en imputación. En la parte práctica, se ha elaborado un Jupyter Notebook y un paquete de Python, **pampaneira_imputation**, que integra diversos métodos de imputación, desde elementales hasta transformer y SAITS. El flujo experimental incluye preprocesamiento, generación controlada de datos perdidos y evaluación. El código, gestionado con Git, está disponible con licencia de código abierto en GitHub para fomentar la transparencia y reproducibilidad.

Los resultados experimentales, mediante métricas estándar de error, demuestran la superioridad de los modelos Transformer sobre los convencionales, logrando una reducción notable y consistente del error de imputación.

En conclusión, este estudio confirma que los modelos basados en autoatención, y SAITS en particular, son herramientas precisas y potentes para la imputación en series temporales, generando datos más completos. El trabajo aporta una solución contrastada y sienta bases para futuras investigaciones en IoT, orientadas a la gestión ambiental y la sostenibilidad.

Términos Clave: Imputación de Datos, Series Temporales, Modelos Transformer, SAITS, Aprendizaje Profundo, IoT, Datos Faltantes, Calidad del Aire, Monitorización de Tráfico, Smart Poqueira, Python, PyPOTS.

ABSTRACT

This Final Degree Project focuses on the generation of synthetic data to replace missing values in multivariate time series within the context of IoT systems. Its purpose is to implement and evaluate the efficacy of deep learning architectures - especially Transformer models like SAITS - for data imputation. The research is applied to a case study of traffic and pollution monitoring from the Smart Poqueira project, aiming to restore data completeness.

The methodology is based on the following branches of **mathematics**: linear algebra, numerical methods, analysis, probability and statistics; and **Deep Learning (DL)**, complemented by a state-of-the-art review in imputation. Practically, a Jupyter Notebook and a Python package, **pampaneira_imputation**, were developed, integrating various imputation methods, ranging from elementary to Transformer and SAITS models. The experimental workflow includes preprocessing and controlled generation of missingness for rigorous evaluation.

Experimental results, using standard error metrics, demonstrate the superiority of Transformer models over conventional ones, achieving a notable and consistent reduction in imputation error across the two study periods analyzed.

In conclusion, this study confirms that self-attention-based models, particularly SAITS, are accurate and powerful tools for imputing complex time series, yielding more complete data. The objectives related to theoretical foundation, analysis, application, development, and evaluation were all successfully met. This work not only provides a validated solution but also lays the groundwork for future research in IoT, oriented towards environmental management and sustainability.

Keywords: Data Imputation, Time Series, Transformer Models, SAITS, Deep Learning, Internet of Things (IoT), Missing Data, Air Quality, Traffic Monitoring, Smart Poqueira, Python, PyPOTS.

Parte I

BLOQUE I: INTRODUCCIÓN, OBJETIVOS Y PLANTEAMIENTO DEL PROBLEMA

INTRODUCCIÓN

1.1 MOTIVACIÓN

Nos encontramos inmersos en una revolución tecnológica sin precedentes, impulsada por la sinergia entre la Inteligencia Artificial (IA) y la masiva generación de datos. La IA, y más concretamente sus ramas del Machine Learning (ML) y Deep Learning (DL), han dejado de ser meros conceptos teóricos para convertirse en herramientas transformadoras con un impacto tangible en la ciencia, la industria y la vida cotidiana (Goodfellow et al., 2016a). Esta proliferación se debe, en gran medida, a la democratización del acceso a una capacidad de cómputo formidable, principalmente a través de las Unidades de Procesamiento Gráfico (GPUs), al refinamiento continuo de algoritmos cada vez más sofisticados y, de manera fundamental, a la disponibilidad de volúmenes de datos digitales que crecen a un ritmo exponencial.

Los cimientos sobre los que se erige el aprendizaje automático son profundamente matemáticos. El Álgebra Lineal ofrece el lenguaje para representar y operar con datos estructurados y los parámetros de los modelos; el Análisis Matemático, con el cálculo diferencial como piedra angular, posibilita los algoritmos de optimización (como el descenso de gradiente) que permiten a los modelos aprender ajustando dichos parámetros; la Teoría de la Probabilidad y la Estadística proporcionan el marco conceptual para modelar la incertidumbre, cuantificar la evidencia y evaluar la generalización de los modelos. Esta amalgama de disciplinas constituye la Teoría del Aprendizaje Estadístico, cuyo fin último es construir sistemas capaces de inferir patrones, relaciones y conocimiento a partir de datos empíricos para, subsecuentemente, realizar predicciones precisas o tomar decisiones óptimas ante nuevas situaciones (Hastie et al., 2009).

Una de las fuentes de datos más dinámicas y ricas de la era actual es el Internet of Things (IoT). El paradigma IoT describe una vasta red de objetos físicos cotidianos –desde simples electrodomésticos hasta complejos sistemas industriales o infraestructuras urbanas– que están equipados con sensores, actuadores, software y conectividad, permitiéndoles recopilar, transmitir e intercambiar datos en tiempo real con otros dispositivos y sistemas a través de Internet (Gubbi et al., 2013). Estos datos, a menudo, se manifiestan como *series temporales multivariantes*: secuencias de ob-

servaciones de múltiples variables registradas a lo largo del tiempo. Dichas series encapsulan la dinámica de los sistemas monitorizados, ofreciendo una visión de su comportamiento y evolución. La capacidad inherente de los algoritmos de ML y DL para discernir patrones intrincados y dependencias temporales complejas en estos flujos de datos masivos del IoT está catalizando una nueva ola de innovación, permitiendo la optimización de procesos, la personalización de servicios y la creación de soluciones inteligentes en un gran número de dominios como las ciudades inteligentes (smart cities), la telemedicina, la agricultura de precisión, la monitorización ambiental y la gestión energética eficiente. (Ahmed et al., 2017).

Es crucial destacar que la aplicación de estas tecnologías avanzadas no se limita a la búsqueda de la eficiencia económica o la mejora de la comodidad individual. Existe un potencial transformador en la contribución de la IA y el IoT para abordar desafíos globales críticos, como los articulados en los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030 de las Naciones Unidas. Por ejemplo, el análisis predictivo de datos IoT provenientes de sensores urbanos puede optimizar la gestión del tráfico, reducir las emisiones contaminantes y mejorar la planificación urbana, contribuyendo directamente al ODS 11 (Ciudades y Comunidades Sostenibles). Este trabajo se alinea con esta visión, buscando aplicar la IA a datos IoT con un fin último orientado a la sostenibilidad.

1.2 DESCRIPCIÓN DEL PROBLEMA

El presente TFG se enmarca en el contexto del proyecto de investigación "Smart Poqueira", una iniciativa que explora el uso de tecnologías IoT para la monitorización y gestión inteligente de la comarca de Poqueira, situada en la Alpujarra granadina (Universidad de Granada, 2021). Dicho sistema IoT, a través de una red de sensores que incluye cámaras, recopila y procesa información ambiental y de actividad humana. Los datos resultantes se estructuran como *series temporales multivariantes*, registrando la evolución de múltiples características (e.g., flujo de vehículos, parámetros ambientales) a lo largo del tiempo. El análisis avanzado de estas series mediante técnicas de Aprendizaje Automático alberga la promesa de desarrollar modelos predictivos capaces de anticipar, por ejemplo, flujos turísticos, picos de consumo de recursos, o la aparición de problemas ambientales. La información derivada de tales modelos sería de gran valor para los gestores locales, permitiendo la implementación de políticas y estrategias proactivas que fomenten una gestión más eficiente, resiliente y, fundamentalmente, sostenible de la región, en consonancia con los objetivos anteriormente citados.

No obstante, la transición desde los datos brutos generados por los sistemas IoT hasta la obtención de conocimiento accionable y modelos predictivos fiables está plagada de desafíos. Uno de los obstáculos más persistentes y problemáticos en el análisis de datos del mundo real, y particularmente en el contexto de series temporales IoT,

es la **presencia de datos faltantes (valores ausentes o "missing data")** en las series temporales. Estos vacíos en la información pueden originarse por diversos factores inherentes a la operativa de los sistemas de sensores en entornos no controlados: fallos técnicos en los propios sensores, errores o interrupciones en la transmisión de datos, cortes en el suministro energético de los dispositivos, periodos de mantenimiento programado o imprevisto, o incluso la incapacidad de un sensor para realizar una medición válida bajo determinadas condiciones ambientales extremas (Little and Rubin, 2002).

Frente a este desafío, la **recuperación de valores faltantes**, técnica más conocida en la literatura como **imputación de datos**, se erige como una tarea fundamental y a menudo ineludible en el preprocesamiento de datos para el análisis de series temporales. La imputación no consiste simplemente en rellenar huecos, sino en estimar de la manera más fidedigna posible los valores que se habrían observado si no se hubiera producido la ausencia, basándose para ello en la información contenida en los datos presentes y en la estructura subyacente del conjunto de datos. Una imputación exitosa debe aspirar a generar valores que sean plausibles y coherentes con la distribución conjunta de las variables y las dinámicas temporales observadas, preservando en la medida de lo posible las propiedades estadísticas del conjunto de datos completo. Al completar las series temporales de manera rigurosa, la imputación no solo permite la aplicación de un espectro más amplio de técnicas de análisis, sino que también contribuye a la construcción de modelos de Aprendizaje Automático más robustos, precisos y generalizables, maximizando así el valor extraíble de los datos recopilados.

El campo de la imputación de datos ha sido objeto de intensa investigación, resultando en una multitud de métodos que varían en complejidad y supuestos. Las técnicas estadísticas tradicionales, como la imputación basada en regresión o la imputación múltiple, han sido ampliamente utilizadas. Sin embargo, con el advenimiento del Aprendizaje Profundo, han surgido nuevos enfoques basados en Modelos Generativos Profundos (Deep Generative Models, DGMs) que han demostrado una capacidad superior para modelar distribuciones de datos complejas y generar imputaciones de alta calidad. Entre estos, arquitecturas como las Redes Generativas Antagónicas (GAN) y los Autoencoders Variacionales (VAEs) han sido adaptadas con éxito para tareas de imputación en diversos contextos (De et al., 2022).

La imputación en series temporales multivariantes, no obstante, añade capas adicionales de complejidad. No solo es necesario modelar las relaciones instantáneas entre las diferentes variables (correlaciones cruzadas), sino también capturar las intrincadas dependencias temporales inherentes a cada serie individual (autocorrelaciones) y las dinámicas de interacción temporal entre las distintas series (correlaciones cruzadas retardadas). En este escenario, las arquitecturas basadas en **Modelos Transformer**, originalmente concebidas para el Procesamiento del Lenguaje Natural (Vaswani et al., 2017a), han emergido como una herramienta de un potencial extraordinario. Su mecanismo de **autoatención (self-attention)** les permite modelar dependencias a largo plazo y capturar interacciones complejas entre puntos distantes de una secuencia de

manera eficiente, superando algunas de las limitaciones inherentes a las arquitecturas recurrentes (como las Red Neuronal Recurrente (RNN) o Long Short-Term Memory (LSTM)) en el manejo de secuencias largas. Recientemente, esta capacidad ha sido explotada para el análisis de series temporales, incluyendo la tarea de imputación. Un ejemplo paradigmático y prometedor de esta tendencia es el modelo **SAITS (Self-Attention based Imputation for Time Series)** (Du et al., 2023b). SAITS adapta la arquitectura Transformer para abordar específicamente el problema de la imputación de valores faltantes en series temporales multivariantes.

Dado el contexto del sistema IoT de Poqueira, la naturaleza multivariante y temporal de sus datos, y el desafío crítico que suponen los valores faltantes para la consecución de sus objetivos de sostenibilidad, este Trabajo de Fin de Grado se plantea como una investigación focalizada en la aplicabilidad y eficacia de los modelos Transformer, y en particular la arquitectura SAITS, para la tarea de imputación. La **pregunta de investigación central** que orienta este estudio es la siguiente: *¿En qué medida los modelos basados en Transformers pueden recuperar eficazmente los valores faltantes en las series temporales multivariantes generadas por el sistema IoT de Poqueira?*

Para facilitar la reproducibilidad y la transparencia de esta investigación, el código fuente desarrollado para la implementación de los modelos, los experimentos realizados y el análisis de los resultados se encuentra disponible de forma pública en el siguiente repositorio de GitHub: <https://github.com/eigenric/TFG>.

1.3 OBJETIVOS DEL TRABAJO

El presente TFG se ha establecido como **objetivo principal** determinar en qué medida los modelos de aprendizaje profundo basados en la arquitectura Transformer pueden recuperar eficazmente los valores faltantes en las series temporales multivariantes generadas por el sistema IoT de Poqueira, y, consecuentemente, proponer una solución validada para la imputación en este contexto.

Para la consecución de este objetivo global, se persiguen los siguientes **objetivos específicos**:

1. **Establecer la Fundamentación Teórica:** Revisar y consolidar los fundamentos matemáticos y los principios conceptuales del aprendizaje automático y el aprendizaje profundo, con especial énfasis en las arquitecturas Transformer y sus mecanismos de atención.
2. **Analizar el Estado del Arte en Imputación de Series Temporales:** Realizar una revisión de la literatura científica sobre modelos y técnicas estadísticas de imputación más utilizadas y aportar una formalización matemática homogénea para el problema de series temporales.
3. **Desarrollar el Pipeline de imputación:** Implementar y aplicar el modelo de imputación SAITS al conjunto de datos proveniente del sistema IoT de Poqueira.

4. **Desarrollar un Paquete de Python de integración:** Diseñar e implementar un paquete de software en Python completo y funcional, que encapsule toda la lógica necesaria para la implementación, aplicación y evaluación sistemática de los modelos de imputación en series temporales, incluyendo desde métodos elementales e intermedios hasta arquitecturas avanzadas como Transformer y SAITS, lo cual constituye un **desafío** al no haber tratado este modelo en la asignatura Aprendizaje Automático. Este prototipo no solo servirá como herramienta para el estudio presente, sino que será documentado y puesto a disposición de la comunidad científica a través de un repositorio público, fomentando así activamente la reproducibilidad de los resultados.
5. **Evaluar el Desempeño del Modelo Implementado:** Validar el prototipo desarrollado y evaluar de manera cuantitativa la eficacia de los modelos en la tarea de recuperación de valores faltantes, utilizando métricas de imputación apropiadas.

1.4 ESTRUCTURA DEL TRABAJO

Este Trabajo de Fin de Grado se organiza en los siguientes bloques temáticos y capítulos:

Bloque I: Introducción y Planteamiento

- **Capítulo 1: Introducción y Objetivos.** Se introduce el contexto de la IA, el IoT y la sostenibilidad, describiendo el desafío de los datos faltantes en series temporales IoT y su impacto. Se presentan la contribución y los objetivos del trabajo.

Bloque II: Fundamentos Teóricos

- **Capítulo 2: Fundamentos Matemáticos.** Se revisan los conceptos esenciales para desarrollar este trabajo de Álgebra Lineal, Análisis Matemático (incluyendo Análisis Funcional), Probabilidad y Estadística, y Métodos Numéricos, que son la base para comprender los modelos de imputación.
- **Capítulo 3: Del Aprendizaje Automático al Aprendizaje Profundo: Fundamentos.** Se exploran los paradigmas del ML, la evaluación de modelos y se profundiza en la transición hacia el Aprendizaje Profundo, con especial énfasis en las arquitecturas Transformer y el modelo SAITS para la imputación.

Bloque III: Metodología y Caso de Estudio: Imputación de Datos

- **Capítulo 4: Metodología para la Imputación de Datos en Series Temporales.** Se define formalmente el problema de los datos faltantes, se presenta una taxonomía de métodos de imputación y se detalla la implementación de diversas funciones (desde estadísticas simples hasta SAITS). Se especifican las métricas de evaluación y el flujo de trabajo del paquete de software desarrollado, `pampaneira_imputation`.
- **Capítulo 5: Imputación de Datos de Tráfico y Contaminación.** Se aplica la metodología a un caso de estudio con datos reales de tráfico y contaminación. Se realiza un Análisis Exploratorio de Datos (EDA), se detalla la preparación de datos, la configuración experimental, y se evalúa cuantitativamente el rendimiento de los métodos.
- **Capítulo 6: Cronogramas y Presupuesto.** Se presenta la planificación temporal del proyecto, comparando la estimación inicial con la ejecución real, y se realiza un análisis detallado del presupuesto estimado.

Bloque IV: Conclusiones y Trabajo Futuro

- **Capítulo 7: Conclusiones y Trabajo Futuro.** Se exponen las principales conclusiones del estudio, se discuten las limitaciones encontradas y el grado de cumplimiento de los objetivos. Finalmente, se proponen líneas de investigación futuras, incluyendo mejoras a los modelos y extensiones a otros dominios.

Parte II

BLOQUE II: FUNDAMENTOS TEÓRICOS

PRINCIPIOS MATEMÁTICOS

Este capítulo establece el marco conceptual y las herramientas matemáticas esenciales que servirán de base para comprender los modelos y métodos que presentaremos en los capítulos posteriores. Abordaremos las ramas fundamentales del Álgebra Lineal, el Análisis Matemático, la Probabilidad y Estadística, y los Métodos Numéricos, proporcionando las definiciones y propiedades necesarias.

2.1 ÁLGEBRA LINEAL Y GEOMETRÍA

El Álgebra Lineal constituye un campo ampliamente desarrollado en obras como [Strang \(2016\)](#) y [Noble and Daniel \(1988\)](#). Esta disciplina provee las herramientas necesarias para representar y manipular datos, así como para describir transformaciones y relaciones en espacios multidimensionales. Nos enfocamos en vectores, matrices y, en un sentido más general, tensores, junto con las operaciones que sobre ellos definimos.

2.1.1 Vectores y Espacios Vectoriales

Comenzamos esta sección formalizando el concepto de vector, y posteriormente el de matriz, que generaliza la noción de vector y escalar.

Definición 1 (Vector). Definimos un *vector* $\mathbf{v}$ en $\mathbb{K}^n$ (típicamente $\mathbb{R}^n$), donde $\mathbb{K}$ es un cuerpo, como una lista ordenada de n elementos de $\mathbb{K}$ ([Strang, 2016](#)). Lo representamos como una columna de tamaño n :

$$\mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}$$

o como una fila de tamaño n (vector traspuesto):

$$\mathbf{v}^T = (v_1 \ v_2 \ \dots \ v_n)$$

Llamamos **componentes** del vector a los elementos $v_i \in \mathbb{K}$. El conjunto de todos estos vectores conforma el espacio vectorial $\mathbb{K}^n$.

Sobre los vectores en $\mathbb{K}^n$ definimos la suma y el producto por un escalar componente a componente, operaciones que le confieren la estructura de espacio vectorial (Noble and Daniel, 1988).

Ejemplo 1 (Vector tridimensional). Consideremos un vector $\mathbf{u}$ en el espacio tridimensional $\mathbb{R}^3$ (es decir, $n = 3$ y $\mathbb{K} = \mathbb{R}$). Si sus componentes son $u_1 = 2$, $u_2 = -1$, y $u_3 = 5$, entonces escribimos el vector columna como:

$$\mathbf{u} = \begin{pmatrix} 2 \\ -1 \\ 5 \end{pmatrix}$$

Y su vector traspuesto (vector fila) es:

$$\mathbf{u}^T = (2 \quad -1 \quad 5)$$

2.1.2 Matrices y Operaciones Matriciales

Una matriz consiste en una disposición rectangular de elementos que organiza datos de forma estructurada y describe transformaciones lineales entre espacios vectoriales (Strang, 2016).

Definición 2 (Matriz). Definimos una **matriz** $\mathbf{A}$ de tamaño $m \times n$, con entradas en un cuerpo $\mathbb{K}$ (típicamente $\mathbb{R}$ o $\mathbb{C}$), como una disposición rectangular de m filas y n columnas de elementos de $\mathbb{K}$. Denotamos una matriz por $\mathbf{A} = (a_{ij})$ o $[\mathbf{A}]_{ij} = a_{ij}$, donde $a_{ij} \in \mathbb{K}$ es la entrada en la fila i y la columna j , para $1 \leq i \leq m$ y $1 \leq j \leq n$. Denotamos el conjunto de todas estas matrices como $M_{m \times n}(\mathbb{K})$ o $\mathbb{K}^{m \times n}$ (Bishop, 2006).

Definimos la igualdad entre matrices a partir de la equivalencia uno a uno de sus elementos.

Definición 3 (Igualdad de Matrices). Decimos que dos matrices $\mathbf{A} = (a_{ij})$ y $\mathbf{B} = (b_{ij})$ son **iguales** si pertenecen al mismo espacio $\mathbb{K}^{m \times n}$ (tienen el mismo tamaño) y $a_{ij} = b_{ij}$ para todos $1 \leq i \leq m$, $1 \leq j \leq n$.

Las operaciones básicas que definimos para matrices de igual tamaño son la suma y el producto por un escalar, que realizamos elemento a elemento (Strang, 2016).

Definición 4 (Suma de Matrices). Dadas $\mathbf{A}, \mathbf{B} \in \mathbb{K}^{m \times n}$, definimos su **suma** como la matriz $\mathbf{A} + \mathbf{B} \in \mathbb{K}^{m \times n}$ mediante $[\mathbf{A} + \mathbf{B}]_{ij} = a_{ij} + b_{ij}$.

Definición 5 (Producto por un Escalar). Dado $\mathbf{A} \in \mathbb{K}^{m \times n}$ y un escalar $c \in \mathbb{K}$, definimos el **producto por un escalar** como la matriz $c\mathbf{A} \in \mathbb{K}^{m \times n}$ mediante $[c\mathbf{A}]_{ij} = c \cdot a_{ij}$.

Con estas dos operaciones, el conjunto de matrices de tamaño fijo $\mathbb{K}^{m \times n}$ adquiere la estructura de un espacio vectorial (Noble and Daniel, 1988).

Definición 6 (Espacio Vectorial de Matrices). *El conjunto $\mathbb{K}^{m \times n}$, con las operaciones de suma (Definición 4) y producto por un escalar (Definición 5), forma un espacio vectorial sobre $\mathbb{K}$. La matriz cero $\mathbf{0}_{m \times n}$ (con todas las entradas 0) actúa como elemento neutro aditivo.*

Proposición 1 (Axiomas del Espacio Vectorial $\mathbb{K}^{m \times n}$). *Las operaciones definidas en $\mathbb{K}^{m \times n}$ satisfacen los axiomas de espacio vectorial (Strang, 2016).*

Demostración. Verificamos que $(\mathbb{K}^{m \times n}, +, \cdot)$ cumple los axiomas de un espacio vectorial sobre $\mathbb{K}$. A modo de ejemplo, demostramos la conmutatividad de la suma. Sean $\mathbf{A}, \mathbf{B} \in \mathbb{K}^{m \times n}$. Queremos demostrar que $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$. Por la definición de suma de matrices (Definición 4), la entrada (i, j) de $\mathbf{A} + \mathbf{B}$ es:

$$[\mathbf{A} + \mathbf{B}]_{ij} = a_{ij} + b_{ij}$$

La entrada (i, j) de $\mathbf{B} + \mathbf{A}$ es:

$$[\mathbf{B} + \mathbf{A}]_{ij} = b_{ij} + a_{ij}$$

Dado que a_{ij} y b_{ij} son elementos del cuerpo $\mathbb{K}$, y la suma en $\mathbb{K}$ es conmutativa, tenemos que $a_{ij} + b_{ij} = b_{ij} + a_{ij}$ para todo i, j . Por lo tanto, $[\mathbf{A} + \mathbf{B}]_{ij} = [\mathbf{B} + \mathbf{A}]_{ij}$ para todo i, j . Como las matrices tienen el mismo tamaño ($m \times n$) y todas sus entradas correspondientes son iguales, por la definición de igualdad de matrices (Definición 3), concluimos que $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$.

Verificamos los demás axiomas (asociatividad de la suma, existencia de elemento neutro aditivo - la matriz cero $\mathbf{0}_{m \times n}$, existencia de inverso aditivo $-\mathbf{A}$, asociatividad del producto por escalar, distributividad escalar y vectorial, y elemento neutro multiplicativo $1 \in \mathbb{K}$) de manera similar, apoyándonos en las propiedades correspondientes de la suma y el producto en el cuerpo $\mathbb{K}$ y las definiciones de las operaciones matriciales. La verificación detallada de estos axiomas se puede encontrar en textos de álgebra lineal como el de Strang (2016). $\square$

Además de las operaciones elemento a elemento, el álgebra matricial define un producto que nos permite combinar matrices bajo ciertas condiciones de compatibilidad de tamaño.

Definición 7 (Producto Matricial). *Definimos el producto matricial de $\mathbf{A} \in \mathbb{K}^{m \times p}$ y $\mathbf{B} \in \mathbb{K}^{p \times n}$ como la matriz $\mathbf{C} = \mathbf{AB} \in \mathbb{K}^{m \times n}$, mediante:*

$$[\mathbf{AB}]_{ij} = c_{ij} = \sum_{k=1}^p a_{ik}b_{kj}$$

para $1 \leq i \leq m$ y $1 \leq j \leq n$ (Bishop, 2006). Nota: Definimos el producto $\mathbf{AB}$ solo si el número de columnas de $\mathbf{A}$ es igual al número de filas de $\mathbf{B}$.

El producto matricial posee propiedades importantes, aunque no es conmutativo en general (Strang, 2016).

Proposición 2 (Propiedades del Producto Matricial). Sean $\mathbf{A}, \mathbf{B}, \mathbf{C}$ matrices de tamaños compatibles para que las operaciones estén definidas, y c un escalar. Nosotros establecemos las siguientes propiedades (Noble and Daniel, 1988):

1. Asociatividad: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$
2. Distributividad: $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$ y $(\mathbf{A} + \mathbf{B})\mathbf{C} = \mathbf{AC} + \mathbf{BC}$
3. Compatibilidad escalar: $c(\mathbf{AB}) = (c\mathbf{A})\mathbf{B} = \mathbf{A}(c\mathbf{B})$
4. No Conmutatividad general: $\mathbf{AB} \neq \mathbf{BA}$ en general.

Además del producto matricial estándar, a menudo resulta útil realizar operaciones elemento a elemento.

Definición 8 (Producto Hadamard). Dadas $\mathbf{A}, \mathbf{B} \in \mathbb{K}^{m \times n}$ (mismo tamaño), definimos su *producto elemento a elemento* o *producto de Hadamard* como la matriz $\mathbf{A} \odot \mathbf{B} \in \mathbb{K}^{m \times n}$ mediante:

$$[\mathbf{A} \odot \mathbf{B}]_{ij} = a_{ij} \cdot b_{ij}$$

Otra operación fundamental es la transposición, que intercambia filas por columnas.

Definición 9 (Matriz Traspuesta). Dada $\mathbf{A} \in \mathbb{K}^{m \times n}$, definimos su *matriz traspuesta* (Strang, 2016), $\mathbf{A}^T \in \mathbb{K}^{n \times m}$, mediante:

$$[\mathbf{A}^T]_{ij} = a_{ji}$$

La transposición tiene propiedades que interactúan con las operaciones matriciales.

Proposición 3 (Propiedades de la Traspuesta). Sean $\mathbf{A}, \mathbf{B}$ matrices de tamaños compatibles para que las operaciones estén definidas, y c un escalar. Nosotros enunciamos las siguientes propiedades (Bishop, 2006):

1. $(\mathbf{A}^T)^T = \mathbf{A}$
2. $(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$
3. $(c\mathbf{A})^T = c\mathbf{A}^T$
4. $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$

Para este trabajo es importante definir las matrices de Toeplitz (Gray et al., 2006) y Hankel (Widom, 1966), las cuales son matrices con patrones de elementos no arbitrarios.

Definición 10 (Matriz de Toeplitz). Sea $(t_k)_{k=-(m-1)}^{n-1}$ una secuencia de elementos en $\mathbb{K}$. Definimos la *matriz de Toeplitz* asociada como la matriz $\mathbf{T} \in \mathbb{K}^{m \times n}$ mediante

$$[\mathbf{T}]_{ij} = t_{j-i}, \quad 1 \leq i \leq m, \quad 1 \leq j \leq n,$$

de modo que cada diagonal constante paralela a la diagonal principal tiene el mismo valor.

Definición 11 (Matriz de Hankel). Sea $(h_k)_{k=1}^{m+n-1}$ una secuencia de elementos en $\mathbb{K}$. Definimos la **matriz de Hankel** asociada como la matriz $\mathbf{H} \in \mathbb{K}^{m \times n}$ mediante

$$[\mathbf{H}]_{ij} = h_{i+j-1}, \quad 1 \leq i \leq m, 1 \leq j \leq n.$$

2.1.3 Tensores y Normas

Así como las matrices organizan números en filas y columnas (dos dimensiones), los tensores son una generalización que nos permite organizar datos en un mayor número de dimensiones. Pueden visualizarse como ‘cubos de datos’ o estructuras aún más complejas si tienen más de tres dimensiones. Para una exploración matemática rigurosa de los tensores y cómo se pueden ‘descomponer’ en partes más simples, una referencia útil es el trabajo de [Kolda and Bader \(2009\)](#).

Definición 12 (Tensor y Orden). Un **tensor** $\mathbf{A}$ de **orden** k (o grado k) sobre el campo de los números reales $\mathbb{R}$ es una generalización de escalares ($k = 0$), vectores ($k = 1$) y matrices ($k = 2$). Lo podemos representar como un array multidimensional cuyos elementos direccionamos usando k índices. Si las dimensiones a lo largo de cada índice son $d_1, d_2, \dots, d_k$, entonces el tensor pertenece al espacio $\mathbb{R}^{d_1 \times d_2 \times \dots \times d_k}$. Denotamos un elemento del tensor $\mathbf{A}$ usando la notación de índices como $[\mathbf{A}]_{i_1 i_2 \dots i_k}$, donde cada índice i_j varía sobre la extensión de la j -ésima dimensión (i.e., $1 \leq i_j \leq d_j$).

Extendemos de manera natural a los tensores las operaciones elemento a elemento definidas para matrices.

Definición 13 (Operaciones Elemento a Elemento para Tensores). Sean $\mathbf{A}, \mathbf{B}$ dos tensores del mismo orden k y con idénticas dimensiones $d_1 \times \dots \times d_k$. Sea $c \in \mathbb{R}$ un escalar y $f : \mathbb{R} \rightarrow \mathbb{R}$ una función escalar. Definimos:

- **Suma/Resta:** $\mathbf{C} = \mathbf{A} \pm \mathbf{B}$ con $[\mathbf{C}]_{i_1 \dots i_k} = [\mathbf{A}]_{i_1 \dots i_k} \pm [\mathbf{B}]_{i_1 \dots i_k}$.
- **Multipliación por Escalar:** $\mathbf{C} = c\mathbf{A}$ con $[\mathbf{C}]_{i_1 \dots i_k} = c \cdot [\mathbf{A}]_{i_1 \dots i_k}$.
- **Producto de Hadamard:** $\mathbf{C} = \mathbf{A} \odot \mathbf{B}$ con $[\mathbf{C}]_{i_1 \dots i_k} = [\mathbf{A}]_{i_1 \dots i_k} \cdot [\mathbf{B}]_{i_1 \dots i_k}$ ([Definición 8](#) generalizada).
- **Aplicación de Función Escalar:** $f(\mathbf{A})$ con $[f(\mathbf{A})]_{i_1 \dots i_k} = f([\mathbf{A}]_{i_1 \dots i_k})$.

Cuando necesitamos comparar tensores o medir cuán diferentes son (por ejemplo, para evaluar un error de aproximación), recurrimos al concepto de norma. La norma nos proporciona una medida cuantitativa de la ‘distancia’ o discrepancia entre ellos, o del ‘tamaño’ general de un tensor individual.

Definición 14 (Norma L_p de un Tensor). Definimos la **norma** L_p de un tensor $\mathbf{A} \in \mathbb{R}^{d_1 \times \dots \times d_k}$, para $1 \leq p < \infty$, como:

$$\|\mathbf{A}\|_p = \left(\sum_{i_1=1}^{d_1} \dots \sum_{i_k=1}^{d_k} |[\mathbf{A}]_{i_1 \dots i_k}|^p \right)^{1/p}$$

donde $[\mathbf{A}]_{i_1 \dots i_k}$ denota el elemento del tensor $\mathbf{A}$ correspondiente a los índices $i_1, \dots, i_k$. Para $p = \infty$, la norma L_∞ (también conocida como norma del máximo) es el máximo valor absoluto de sus elementos:

$$\|\mathbf{A}\|_\infty = \max_{i_1, \dots, i_k} |[\mathbf{A}]_{i_1 \dots i_k}|$$

Casos particulares importantes son la **norma** L_1 ($p = 1$), que es la suma de los valores absolutos de todos los elementos, y la **norma** L_2 ($p = 2$). Para matrices (tensores de orden $k = 2$), la norma L_2 es comúnmente conocida como la **norma de Frobenius** (Golub and Van Loan, 2013; Bishop, 2006).

La norma L_1 suma los valores absolutos de los elementos. La norma L_2 (o norma de Frobenius para matrices) está intrínsecamente relacionada con el producto escalar. Si consideramos el tensor $\mathbf{A}$ como un vector aplanado $\text{vec}(\mathbf{A})$ en un espacio de dimensión $d_1 \times \dots \times d_k$, entonces la norma L_2 al cuadrado es el producto escalar de este vector consigo mismo:

$$\|\mathbf{A}\|_2^2 = \sum_{i_1=1}^{d_1} \dots \sum_{i_k=1}^{d_k} ([\mathbf{A}]_{i_1 \dots i_k})^2 = \langle \text{vec}(\mathbf{A}), \text{vec}(\mathbf{A}) \rangle$$

Para una matriz $\mathbf{M} \in \mathbb{R}^{m \times n}$, la norma de Frobenius al cuadrado también puede expresarse como la traza del producto de la matriz por su traspuesta (Golub and Van Loan, 2013):

$$\|\mathbf{M}\|_F^2 = \|\mathbf{M}\|_2^2 = \sum_{i=1}^m \sum_{j=1}^n ([\mathbf{M}]_{ij})^2 = \text{Tr}(\mathbf{M}^T \mathbf{M}) = \text{Tr}(\mathbf{M} \mathbf{M}^T)$$

Ambas normas, L_1 y L_2 , son fundamentales cuando definimos métricas de error y funciones de regularización.

Ejemplo 2 (Norma de Frobenius). Consideremos la siguiente matriz $\mathbf{B} \in \mathbb{R}^{2 \times 3}$:

$$\mathbf{B} = \begin{pmatrix} 1 & -2 & 0 \\ 3 & 4 & -1 \end{pmatrix}$$

Calculamos la norma L_2 (o norma de Frobenius) de $\mathbf{B}$ como:

$$\begin{aligned} \|\mathbf{B}\|_2 &= \left(\sum_{i=1}^2 \sum_{j=1}^3 |[\mathbf{B}]_{ij}|^2 \right)^{1/2} \\ &= \sqrt{1^2 + (-2)^2 + 0^2 + 3^2 + 4^2 + (-1)^2} \\ &= \sqrt{1 + 4 + 0 + 9 + 16 + 1} \\ &= \sqrt{31} \end{aligned}$$

2.2 ANÁLISIS MATEMÁTICO

El Análisis Matemático, y en particular el cálculo diferencial, desarrolla un conjunto de técnicas formales para investigar cómo las funciones se transforman y evolucionan. A través de conceptos como el límite, la continuidad y la derivada, podemos determinar con exactitud la ‘velocidad’ con la que una función cambia en cada uno de sus puntos. Esta ‘velocidad de cambio’ no solo describe el comportamiento local de la función (si aumenta o disminuye), sino que también es la base para identificar sus puntos extremos (máximos o mínimos), lo cual es el objetivo central de los problemas de optimización. Los trabajos de Rudin (1976) y Apostol (1974) ofrecen una exposición detallada y formal de estas herramientas y sus aplicaciones.

2.2.1 Funciones y Continuidad

Comenzamos con el concepto fundamental de función, una regla que asigna a cada elemento de un conjunto de partida un único elemento de un conjunto de llegada.

Definición 15 (Función). *Una **función** f de un conjunto A a un conjunto B , que denotamos $f : A \rightarrow B$, es una regla que asigna a cada elemento $x \in A$ un único elemento $f(x) \in B$. Llamamos a A el **dominio** de f , y a B el **codominio**. La **imagen** de f , denotada $\text{Im}(f)$, es el conjunto $\{f(x) | x \in A\} \subseteq B$ (Apostol, 1974).*

En el contexto que nos ocupa, trabajaremos principalmente con funciones cuyo dominio y codominio son subconjuntos de espacios euclídeos $\mathbb{R}^n$ o $\mathbb{R}^m$. La continuidad, un concepto clave, describe funciones cuyas salidas varían de manera suave con pequeñas variaciones en sus entradas (Rudin, 1976).

Definición 16 (Continuidad). *Decimos que una función $f : A \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^m$ es **continua** en un punto $\mathbf{a} \in A$ si para todo $\epsilon > 0$, existe un $\delta > 0$ tal que si $\mathbf{x} \in A$ y $\|\mathbf{x} - \mathbf{a}\|_2 < \delta$, entonces $\|f(\mathbf{x}) - f(\mathbf{a})\|_2 < \epsilon$. Si f es continua en todos los puntos de A , decimos que f es continua en A . Esta es la definición $\epsilon - \delta$ estándar de continuidad (Apostol, 1974; Rudin, 1976).*

2.2.2 Cálculo Diferencial Multivariable

Extendemos la noción de derivada a funciones de varias variables o funciones con valores vectoriales, siguiendo el tratamiento de autores como Apostol (1974).

Definición 17 (Derivada Parcial). Sea $f : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ definida en un conjunto abierto U . Definimos la **derivada parcial** de f respecto a la i -ésima variable x_i en un punto $\mathbf{a} \in U$ como el límite:

$$\frac{\partial f}{\partial x_i}(\mathbf{a}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{a} + h\mathbf{e}_i) - f(\mathbf{a})}{h}$$

si el límite existe, donde $\mathbf{e}_i$ es el i -ésimo vector canónico de la base estándar de $\mathbb{R}^n$ (Rudin, 1976).

Es importante destacar que la mera existencia de todas las derivadas parciales de una función f en un punto $\mathbf{a}$ no garantiza que f sea diferenciable en $\mathbf{a}$. La diferenciabilidad es una condición más fuerte que implica que la función puede ser bien aproximada localmente por una transformación lineal (su diferencial) (Apostol, 1974; Rudin, 1976).

Una condición suficiente común para la diferenciabilidad es que la función sea de **clase C^1** . Decimos que una función f es de clase C^1 en un conjunto abierto U si todas sus derivadas parciales $\frac{\partial f}{\partial x_i}$ existen en U y son, además, funciones continuas en U . Si las derivadas parciales de f existen en un entorno de $\mathbf{a}$ y son continuas en $\mathbf{a}$ mismo, entonces f es diferenciable en $\mathbf{a}$ (Apostol, 1974).

Contraejemplo 1 (Función parcialmente derivable no diferenciable). Consideremos la función $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ definida por:

$$f(x, y) = \begin{cases} \frac{xy}{x^2 + y^2} & \text{si } (x, y) \neq (0, 0) \\ 0 & \text{si } (x, y) = (0, 0) \end{cases}$$

Calculamos las derivadas parciales en el origen $(0, 0)$:

$$\begin{aligned} \frac{\partial f}{\partial x}(0, 0) &= \lim_{h \rightarrow 0} \frac{f(h, 0) - f(0, 0)}{h} = \lim_{h \rightarrow 0} \frac{(h \cdot 0) / (h^2 + 0^2) - 0}{h} = \lim_{h \rightarrow 0} \frac{0}{h} = 0 \\ \frac{\partial f}{\partial y}(0, 0) &= \lim_{h \rightarrow 0} \frac{f(0, h) - f(0, 0)}{h} = \lim_{h \rightarrow 0} \frac{(0 \cdot h) / (0^2 + h^2) - 0}{h} = \lim_{h \rightarrow 0} \frac{0}{h} = 0 \end{aligned}$$

Ambas derivadas parciales existen en $(0, 0)$ y son iguales a cero. Sin embargo, la función f no es continua en $(0, 0)$, como se discute en Apostol (1974) o Rudin (1976) para ejemplos similares. Si nos acercamos al origen a lo largo de la recta $y = mx$, obtenemos:

$$\lim_{x \rightarrow 0} f(x, mx) = \lim_{x \rightarrow 0} \frac{x(mx)}{x^2 + (mx)^2} = \lim_{x \rightarrow 0} \frac{mx^2}{x^2(1 + m^2)} = \frac{m}{1 + m^2}$$

Dado que este límite depende de la pendiente m de la trayectoria de aproximación, el límite general $\lim_{(x, y) \rightarrow (0, 0)} f(x, y)$ no existe. Como la función no es continua en $(0, 0)$, no puede ser diferenciable en $(0, 0)$, a pesar de que todas sus derivadas parciales existan en ese punto.

Para funciones de $\mathbb{R}^n$ a $\mathbb{R}$, podemos agrupar todas las derivadas parciales en un vector llamado gradiente.

Definición 18 (Gradiente). Sea $f : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ una función real-valorada tal que todas sus derivadas parciales existen en un punto $\mathbf{a} \in U$. Definimos el **vector gradiente** de f en $\mathbf{a}$, denotado $\nabla f(\mathbf{a})$, como el vector en $\mathbb{R}^n$ cuyas componentes son las derivadas parciales de f en $\mathbf{a}$:

$$\nabla f(\mathbf{a}) = \left(\frac{\partial f}{\partial x_1}(\mathbf{a}), \frac{\partial f}{\partial x_2}(\mathbf{a}), \dots, \frac{\partial f}{\partial x_n}(\mathbf{a}) \right)^T$$

Si f es diferenciable en $\mathbf{a}$, el gradiente $\nabla f(\mathbf{a})$ indica la dirección y magnitud del mayor aumento local de f en el punto $\mathbf{a}$ (Apostol, 1974; Bishop, 2006).

Cuando el codominio de la función es vectorial ($\mathbb{R}^m$), organizamos las derivadas parciales en una matriz.

Proposición 4 (Diferenciabilidad y Gradiente para Campos Escalares). Para un campo escalar $f : \Omega \rightarrow \mathbb{R}$ donde Ω es un abierto de $\mathbb{R}^N$, y un punto $a \in \Omega$, las siguientes afirmaciones son equivalentes (Apostol, 1974):

1. f es diferenciable en a .
2. f es parcialmente derivable en a (es decir, existe $\nabla f(a)$) y verificamos que:

$$\lim_{\mathbf{x} \rightarrow \mathbf{a}} \frac{f(\mathbf{x}) - f(\mathbf{a}) - \langle \nabla f(\mathbf{a}), \mathbf{x} - \mathbf{a} \rangle}{\|\mathbf{x} - \mathbf{a}\|_2} = 0 \quad (1)$$

donde $\langle \cdot, \cdot \rangle$ denota el producto escalar usual en $\mathbb{R}^N$.

En caso de que se cumplan (i) y (ii), tenemos que la diferencial de f en a aplicada a un vector $v \in \mathbb{R}^N$, denotada $Df(a)v$, es:

$$Df(a)v = \langle \nabla f(a), v \rangle \quad (\text{también escrito como } \nabla f(a) \cdot v \text{ o } (\nabla f(a))^T v) \quad (2)$$

Definición 19 (Matriz Jacobiana). Sea $\mathbf{f} : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^m$ una función vectorial $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x}))^T$, donde cada $f_i : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$. Si todas las derivadas parciales $\frac{\partial f_i}{\partial x_j}(\mathbf{x})$ existen en un punto $\mathbf{x} \in U$, definimos la **matriz Jacobiana** de $\mathbf{f}$ en $\mathbf{x}$, denotada $\mathbf{J}_{\mathbf{f}}(\mathbf{x})$, como la matriz de tamaño $m \times n$ cuyas entradas son:

$$[\mathbf{J}_{\mathbf{f}}(\mathbf{x})]_{ij} = \frac{\partial f_i}{\partial x_j}(\mathbf{x})$$

Notar que si $m = 1$, entonces $\mathbf{f}$ es una función real-valorada f , y la matriz Jacobiana es el vector gradiente traspuesto: $\mathbf{J}_f(\mathbf{x}) = (\nabla f(\mathbf{x}))^T$ (Apostol, 1974; Bishop, 2006).

Una regla fundamental para derivar funciones compuestas es la regla de la cadena (Apostol, 1974; Rudin, 1976).

Teorema 1 (Regla de la Cadena Multivariable). (Apostol, 1974). Sean $U \subseteq \mathbb{R}^n$, $V \subseteq \mathbb{R}^m$ conjuntos abiertos. Sean $\mathbf{g} : U \rightarrow V$ diferenciable en $\mathbf{x}_0 \in U$, y $\mathbf{f} : V \rightarrow \mathbb{R}^p$ diferenciable en $\mathbf{y}_0 = \mathbf{g}(\mathbf{x}_0) \in V$. La composición $\mathbf{h} = \mathbf{f} \circ \mathbf{g} : U \rightarrow \mathbb{R}^p$ es diferenciable en $\mathbf{x}_0$, y su matriz Jacobiana en $\mathbf{x}_0$ es el producto de las matrices Jacobianas de $\mathbf{f}$ y $\mathbf{g}$:

$$\mathbf{J}_{\mathbf{h}}(\mathbf{x}_0) = \mathbf{J}_{\mathbf{f}}(\mathbf{y}_0)\mathbf{J}_{\mathbf{g}}(\mathbf{x}_0)$$

Demostración. Sea $\mathbf{h}(\mathbf{x}) = \mathbf{f}(\mathbf{g}(\mathbf{x}))$. Escribimos las funciones componentes: $\mathbf{x} = (x_1, \dots, x_n)^T$, $\mathbf{y} = \mathbf{g}(\mathbf{x}) = (g_1(\mathbf{x}), \dots, g_m(\mathbf{x}))^T$, y $\mathbf{z} = \mathbf{h}(\mathbf{x}) = \mathbf{f}(\mathbf{y}) = (f_1(\mathbf{y}), \dots, f_p(\mathbf{y}))^T$. Queremos encontrar la entrada (k, j) de la matriz Jacobiana $\mathbf{J}_{\mathbf{h}}(\mathbf{x}_0)$, que por definición es $\frac{\partial h_k}{\partial x_j}(\mathbf{x}_0)$. La función h_k es $f_k(g_1(\mathbf{x}), \dots, g_m(\mathbf{x}))$. Aplicando la regla de la cadena del cálculo multivariable para funciones escalares, como se detalla en [Apostol \(1974\)](#), calculamos la derivada parcial de h_k con respecto a x_j como:

$$\frac{\partial h_k}{\partial x_j}(\mathbf{x}_0) = \sum_{l=1}^m \frac{\partial f_k}{\partial y_l}(\mathbf{y}_0) \frac{\partial g_l}{\partial x_j}(\mathbf{x}_0)$$

donde $\mathbf{y}_0 = \mathbf{g}(\mathbf{x}_0)$. Examinamos el lado derecho de la ecuación. El término $\frac{\partial f_k}{\partial y_l}(\mathbf{y}_0)$ es la entrada (k, l) de la matriz Jacobiana $\mathbf{J}_{\mathbf{f}}(\mathbf{y}_0)$. El término $\frac{\partial g_l}{\partial x_j}(\mathbf{x}_0)$ es la entrada (l, j) de la matriz Jacobiana $\mathbf{J}_{\mathbf{g}}(\mathbf{x}_0)$. La suma $\sum_{l=1}^m [\mathbf{J}_{\mathbf{f}}(\mathbf{y}_0)]_{kl} [\mathbf{J}_{\mathbf{g}}(\mathbf{x}_0)]_{lj}$ es precisamente la definición de la entrada (k, j) del producto matricial $\mathbf{J}_{\mathbf{f}}(\mathbf{y}_0) \mathbf{J}_{\mathbf{g}}(\mathbf{x}_0)$. Por lo tanto, hemos demostrado que:

$$[\mathbf{J}_{\mathbf{h}}(\mathbf{x}_0)]_{kj} = \frac{\partial h_k}{\partial x_j}(\mathbf{x}_0) = [\mathbf{J}_{\mathbf{f}}(\mathbf{y}_0) \mathbf{J}_{\mathbf{g}}(\mathbf{x}_0)]_{kj}$$

Como esto es válido para todo $1 \leq k \leq p$ y $1 \leq j \leq n$, las matrices son iguales: $\mathbf{J}_{\mathbf{h}}(\mathbf{x}_0) = \mathbf{J}_{\mathbf{f}}(\mathbf{y}_0) \mathbf{J}_{\mathbf{g}}(\mathbf{x}_0)$. (Nota: Esta prueba asume la validez de la regla de la cadena para derivar una función escalar f_k compuesta con una función vectorial $\mathbf{g}$. Una prueba más rigurosa se basa en la definición de diferenciabilidad mediante aproximaciones lineales y el manejo cuidadoso de los términos de error, como se encuentra en [Apostol \(1974\)](#) o [Rudin \(1976\)](#). $\square$)

Esta regla es fundamental para el cálculo de gradientes en funciones compuestas, una operación recurrente en algoritmos de optimización, como el de retropropagación ([Rumelhart et al., 1986a](#); [Goodfellow et al., 2016b](#)).

2.3 ANÁLISIS FUNCIONAL

Introducimos algunos conceptos del Análisis Funcional, que son importantes para la demostración del Teorema de Aproximación Universal. Textos de referencia para esta área son [Kreyszig \(1989\)](#) y [Rudin \(1991\)](#).

Definición 20 (Espacio de Funciones Continuas $C(K)$). Sea K un espacio topológico compacto Hausdorff (en nuestro caso, $I_n = [0, 1]^n \subset \mathbb{R}^n$). Definimos el espacio $C(K)$ como el conjunto de todas las funciones continuas $f : K \rightarrow \mathbb{R}$ (o $\mathbb{C}$). Dotamos a $C(K)$ de la estructura de espacio de Banach con la **norma del supremo** (o norma uniforme) ([Kreyszig, 1989](#)):

$$\|f\|_{\infty} = \sup_{\mathbf{x} \in K} |f(\mathbf{x})|$$

Definición 21 (Densidad). Decimos que un subconjunto S de un espacio normado X (e.g., $C(K)$) es **denso** en X si la clausura de S , denotada $\bar{S}$, es igual a X . Esto equivale a decir que para todo $f \in X$ y todo $\epsilon > 0$, existe un $g \in S$ tal que $\|f - g\|_\infty < \epsilon$ (usando la norma del espacio ambiente) (Rudin, 1991).

Definición 22 (Funcional Lineal Acotado). Sea X un espacio vectorial normado sobre $\mathbb{K}$ ($\mathbb{R}$ o $\mathbb{C}$). Definimos un **funcional lineal** como una aplicación $L : X \rightarrow \mathbb{K}$ tal que para todo $x, y \in X$ y todo $\alpha \in \mathbb{K}$:

1. $L(x + y) = L(x) + L(y)$ (aditividad)
2. $L(\alpha x) = \alpha L(x)$ (homogeneidad)

Decimos que un funcional lineal L es **acotado** si existe una constante $M \geq 0$ tal que $|L(x)| \leq M \|x\|$ para todo $x \in X$. El conjunto de todos los funcionales lineales acotados sobre X conforma el espacio dual X^* (Kreyszig, 1989).

Teorema 2 (Teorema de Hahn-Banach (Corolario para subespacios)). (Kreyszig, 1989, Thm. 4.3-2). Sea S un subespacio de un espacio vectorial normado X . Si $f \in X$ y la distancia de f a S , $d(f, S) = \inf_{s \in S} \|f - s\| = \delta > 0$ (i.e., $f \notin \bar{S}$), entonces existe un funcional lineal acotado $L \in X^*$ tal que:

1. $L(s) = 0$ para todo $s \in S$ (i.e., L anula a S).
2. $L(f) = \delta$ (o, más generalmente, $L(f) \neq 0$).
3. $\|L\| = 1$.

En particular, si S es un subespacio propio cerrado de X , existe un $L \in X^*$ no nulo tal que $L(s) = 0$ para todo $s \in S$.

Definición 23 (Medida de Borel Signada Regular). Sea K un espacio topológico compacto Hausdorff. Una **medida de Borel signada** μ sobre K es una función de conjunto $\mu : \mathcal{B}(K) \rightarrow \mathbb{R} \cup \{-\infty, \infty\}$ (donde $\mathcal{B}(K)$ es la σ -álgebra de Borel de K) que es σ -aditiva. Decimos que una medida de Borel signada μ es **regular** si para todo conjunto de Borel B :

$$|\mu(B)| = \inf\{\mu(U) : B \subseteq U, U \text{ abierto}\} \\ = \sup\{\mu(C) : C \subseteq B, C \text{ compacto}\} \quad (\text{para medidas positivas, con ajuste para signadas})$$

El espacio de todas las medidas de Borel signadas regulares finitas sobre K lo denotamos $M(K)$ (Rudin, 1987, Definición 6.2, 6.4).

Teorema 3 (Teorema de Representación de Riesz-Markov-Kakutani). (Rudin, 1987, Thm. 6.19). Sea K un espacio topológico compacto Hausdorff. El espacio dual de $C(K)$, denotado $(C(K))^*$, es isométricamente isomorfo al espacio $M(K)$ de las medidas de Borel signadas regulares finitas sobre K . Es decir, para cada funcional lineal acotado $L \in (C(K))^*$, existe una única medida $\mu \in M(K)$ tal que:

$$L(f) = \int_K f(\mathbf{x}) d\mu(\mathbf{x}) \quad \text{para todo } f \in C(K)$$

y $\|L\| = \|\mu\|_{TV}$ (norma de variación total de μ).

Teorema 4 (Teorema de Convergencia Dominada de Lebesgue). (*Rudin, 1987, Thm. 1.34*). Sea $(X, \mathcal{M}, \mu)$ un espacio de medida. Si $\{f_k\}_{k=1}^{\infty}$ es una secuencia de funciones medibles $f_k : X \rightarrow \mathbb{R}$ tal que $f_k(\mathbf{x}) \rightarrow f(\mathbf{x})$ casi en todas partes para $\mathbf{x} \in X$ cuando $k \rightarrow \infty$, y existe una función integrable $g : X \rightarrow [0, \infty]$ tal que $|f_k(\mathbf{x})| \leq g(\mathbf{x})$ casi en todas partes para todo k , entonces f es integrable y

$$\lim_{k \rightarrow \infty} \int_X f_k d\mu = \int_X f d\mu$$

2.4 PROBABILIDAD Y ESTADÍSTICA

La Probabilidad y la Estadística nos permiten modelar la incertidumbre, describir conjuntos de datos y realizar inferencias. La probabilidad proporciona el lenguaje para describir fenómenos aleatorios, mientras que la estadística se ocupa de la recolección, análisis, interpretación y presentación de datos. Textos de referencia incluyen *Casella and Berger (2002)*, *Ross (2014)*, *DeGroot and Schervish (2012)* y *Rohatgi and Saleh (2015)*.

2.4.1 Estadística Descriptiva

La estadística descriptiva resume y describe las características principales de un conjunto de datos.

Definición 24 (Media Aritmética). Dada una colección finita de N números reales $\{x_1, \dots, x_N\} \subset \mathbb{R}$, definimos la **media aritmética** como $\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$ (*Casella and Berger, 2002*).

Definición 25 (Mediana). Dada una colección finita de N números reales $\{x_1, \dots, x_N\} \subset \mathbb{R}$, y su versión ordenada no decreciente $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(N)}$, definimos la **mediana** Me como el valor central. Formalmente, si N es impar, $Me = x_{((N+1)/2)}$. Si N es par, $Me = \frac{1}{2}(x_{(N/2)} + x_{(N/2+1)})$ (*DeGroot and Schervish, 2012*).

La media y la mediana son medidas de centralización que buscan representar el valor típico de un conjunto de datos. Para medir la dispersión o variabilidad de los datos alrededor de la media, utilizamos la varianza y la desviación estándar.

Definición 26 (Varianza Muestral). Dada una colección finita de N números reales $\{x_1, \dots, x_N\} \subset \mathbb{R}$ con media aritmética $\bar{x}$, definimos la **varianza muestral**, denotada s^2 o $\sigma_{\text{muestra}}^2$, como el promedio de las diferencias al cuadrado con respecto a la media:

$$s^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2$$

El uso del denominador $N - 1$ en lugar de N proporciona un estimador insesgado de la varianza poblacional cuando los datos se consideran una muestra de una población mayor (Casella and Berger, 2002; Ross, 2014). Si el conjunto de N datos constituye la totalidad de la población, se utiliza N en el denominador para calcular la varianza poblacional σ^2 .

Definición 27 (Desviación Estándar Muestral). Dada una colección finita de N números reales $\{x_1, \dots, x_N\} \subset \mathbb{R}$, la **desviación estándar muestral**, denotada s o σ_{muestra} , es la raíz cuadrada positiva de la varianza muestral:

$$s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$$

La desviación estándar se expresa en las mismas unidades que los datos originales, lo que facilita su interpretación como una medida típica de la distancia de las observaciones a la media (DeGroot and Schervish, 2012).

Mientras que la media (Definición 24) y la mediana son medidas de centralización que buscan representar el valor típico de un conjunto de datos, la varianza y la desviación estándar cuantifican la dispersión o variabilidad de esos datos alrededor de la media.

2.4.2 Fundamentos de Probabilidad

La teoría de la probabilidad proporciona un marco para cuantificar la incertidumbre, basada en los axiomas de Kolmogorov (Ross, 2014).

Definición 28 (Espacio de Probabilidad y Eventos). Definimos un **espacio de probabilidad** como una terna $(\Omega, \mathcal{F}, P)$, donde Ω es el **espacio muestral** (el conjunto de todos los posibles resultados de un experimento aleatorio), $\mathcal{F}$ es una σ -álgebra de **eventos** (subconjuntos de Ω a los que podemos asignar una probabilidad), y $P : \mathcal{F} \rightarrow [0, 1]$ es una **medida de probabilidad** que satisface los axiomas de Kolmogorov (Ross, 2014).

Definición 29 (Probabilidad de un Evento). Definimos la **probabilidad** de un evento $A \in \mathcal{F}$, denotada $P(A)$, como un número en el intervalo $[0, 1]$ que cuantifica la verosimilitud de que ocurra A .

Definición 30 (Variable Aleatoria). Definimos una **variable aleatoria** (v.a.) X como una función medible $X : \Omega \rightarrow \mathbb{R}$. Esta transforma los resultados del espacio muestral en valores numéricos. Podemos considerar también vectores aleatorios ($X : \Omega \rightarrow \mathbb{R}^n$) o matrices aleatorias ($X : \Omega \rightarrow \mathbb{R}^{m \times n}$) (Casella and Berger, 2002).

Clasificamos las variables aleatorias como discretas o continuas, dependiendo del conjunto de valores que pueden tomar. A continuación, definiremos cada una de estas variables con el objetivo de caracterizar su comportamiento.

Definición 31 (Variable Aleatoria Discreta). Decimos que una variable aleatoria X es **discreta** si su conjunto de posibles valores es finito o contablemente infinito. Describimos la distribución de probabilidad de una v.a. discreta mediante su **función de masa de probabilidad** (FMP), $p_X(x) = P(X = x)$, que asigna una probabilidad a cada valor posible x . La FMP satisface $p_X(x) \geq 0$ para todo x y $\sum_x p_X(x) = 1$ (Ross, 2014).

Definición 32 (Variable Aleatoria Continua). Decimos que una variable aleatoria X es **continua** si toma valores en un intervalo de números reales (o $\mathbb{R}$). Describimos la distribución de probabilidad de una v.a. continua mediante su **función de densidad de probabilidad** (FDP), $f_X(x)$, tal que $P(a \leq X \leq b) = \int_a^b f_X(x)dx$ para cualesquiera $a \leq b$. La FDP satisface $f_X(x) \geq 0$ para todo x y $\int_{-\infty}^{\infty} f_X(x)dx = 1$ (DeGroot and Schervish, 2012).

Un concepto clave es cómo la información sobre un evento afecta la probabilidad de otro.

Definición 33 (Probabilidad Condicionada). Dados dos eventos $A, B \in \mathcal{F}$ con $P(B) > 0$, definimos la **probabilidad condicionada** de A dado B , denotada $P(A|B)$, como:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Esta representa la probabilidad de que ocurra A sabiendo que B ha ocurrido (Ross, 2014).

Relacionamos la probabilidad condicionada con el concepto de independencia.

Definición 34 (Independencia Estadística). Decimos que dos eventos $A, B \in \mathcal{F}$ son **estadísticamente independientes** si la ocurrencia de uno no altera la probabilidad del otro. Formalizamos esto como $P(A \cap B) = P(A)P(B)$. Si $P(B) > 0$, una definición equivalente es $P(A|B) = P(A)$. Decimos que dos variables aleatorias X e Y son independientes si para cualesquiera conjuntos medibles $A, B \subseteq \mathbb{R}$, los eventos $\{X \in A\}$ e $\{Y \in B\}$ son independientes (Casella and Berger, 2002).

2.4.3 Distribuciones de Probabilidad Discretas Comunes

Presentamos algunas distribuciones discretas fundamentales (Ross, 2014; Casella and Berger, 2002).

Definición 35 (Distribución Binomial). La distribución Binomial describe el número de éxitos k en una secuencia de n ensayos de Bernoulli independientes e idénticamente distribuidos, cada uno con una probabilidad de éxito p .

Sea X una variable aleatoria que sigue una distribución Binomial, que denotamos como $X \sim \text{Binomial}(n, p)$ o $X \sim B(n, p)$. La **función de masa de probabilidad** (FMP) de X para obtener exactamente k éxitos en n ensayos es:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

para $k = 0, 1, 2, \dots, n$, donde:

- n es el número total de ensayos (un entero no negativo).
- k es el número de éxitos (un entero no negativo, $0 \leq k \leq n$).
- p es la probabilidad de éxito en un solo ensayo ($0 \leq p \leq 1$).
- $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ es el coeficiente binomial.

La probabilidad de fracaso en un solo ensayo es $q = 1 - p$ (Ross, 2014).

Definición 36 (Distribución Multinomial). La distribución Multinomial generaliza la distribución Binomial. Describe la probabilidad de obtener un conjunto específico de conteos para K categorías mutuamente excluyentes, después de realizar n ensayos independientes.

Sean $X_1, X_2, \dots, X_K$ variables aleatorias que representan el número de veces que ocurre cada una de las K categorías en n ensayos. El vector de conteos $\mathbf{x} = (x_1, x_2, \dots, x_K)^T$ sigue una distribución Multinomial con parámetros n (número de ensayos) y $\mathbf{p} = (p_1, p_2, \dots, p_K)^T$ (vector de probabilidades de cada categoría), que denotamos como $\mathbf{X} \sim \text{Multinomial}(n, \mathbf{p})$. Las condiciones son:

- x_i es el número de ocurrencias de la categoría i (entero no negativo).
- $\sum_{i=1}^K x_i = n$.
- p_i es la probabilidad de que un ensayo resulte en la categoría i .
- $\sum_{i=1}^K p_i = 1$ y $p_i \geq 0$ para todo $i = 1, \dots, K$.

La **función de masa de probabilidad** (FMP) conjunta de $X_1, \dots, X_K$ es:

$$P(X_1 = x_1, X_2 = x_2, \dots, X_K = x_K) = \frac{n!}{x_1! x_2! \dots x_K!} p_1^{x_1} p_2^{x_2} \dots p_K^{x_K}$$

donde $\frac{n!}{x_1! x_2! \dots x_K!}$ es el coeficiente multinomial. Si $K = 2$, la distribución Multinomial se reduce a la [Definición 35](#) (con $x_1 = k$, $x_2 = n - k$, $p_1 = p$, $p_2 = 1 - p$) (Casella and Berger, 2002).

Definición 37 (Distribución Categórica). La distribución Categórica (también conocida como distribución de Bernoulli generalizada o Multinoulli) es un caso especial de la [Definición 36](#) donde realizamos un solo ensayo, es decir, $n = 1$. Describe la probabilidad de que un solo evento pertenezca a una de K categorías mutuamente excluyentes (Bishop, 2006).

Sea Y una variable aleatoria que sigue una distribución Categórica con parámetro $\mathbf{p} = (p_1, p_2, \dots, p_K)^T$, que denotamos como $Y \sim \text{Categorical}(\mathbf{p})$. Las condiciones son:

- K es el número de categorías posibles.
- p_i es la probabilidad de que el resultado sea la categoría i .

- $\sum_{i=1}^K p_i = 1$ y $p_i \geq 0$ para todo $i = 1, \dots, K$.

Estos fundamentos probabilísticos son indispensables para que comprendamos la aleatoriedad inherente en los datos y en los procesos de modelado.

2.5 LA FUNCIÓN SOFTMAX

Presentamos la función Softmax, ampliamente utilizada en clasificación multiclase (Bishop, 2006). La **Softmax** (también llamada función Softmax) transforma un vector de números reales en un vector de números positivos que suman 1.

Definición 38 (Softmax). Definimos la **Softmax** $\sigma : \mathbb{R}^K \rightarrow \mathbb{R}^K$ como la función que toma un vector de entrada $\mathbf{z} = (z_1, \dots, z_K)^T \in \mathbb{R}^K$ y devuelve un vector de salida $\sigma(\mathbf{z}) = (p_1, \dots, p_K)^T$ con componentes:

$$p_i = [\sigma(\mathbf{z})]_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

para $i = 1, \dots, K$

La Softmax posee dos propiedades clave que la relacionan con las distribuciones de probabilidad discretas.

Proposición 5 (Propiedades de la Softmax). Para el vector de salida $\mathbf{p} = \sigma(\mathbf{z})$, se cumplen (Bishop, 2006):

1. Positividad: $p_i > 0$ para todo $i = 1, \dots, K$.
2. Normalización: $\sum_{i=1}^K p_i = 1$.

Demostración de la Propiedad 5.1 (Positividad). Para cualquier número real z_i , la función exponencial e^{z_i} es estrictamente positiva. La suma en el denominador, $\sum_{j=1}^K e^{z_j}$, es una suma de términos estrictamente positivos, por lo que también es estrictamente positiva. El cociente de dos números estrictamente positivos es estrictamente positivo. Por lo tanto, concluimos que $p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} > 0$ para todo i . $\square$

Demostración de la Propiedad 5.2 (Normalización). Calculamos la suma de las componentes del vector de salida:

$$\sum_{i=1}^K p_i = \sum_{i=1}^K \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Dado que el denominador $\sum_{j=1}^K e^{z_j}$ es una constante con respecto al índice de suma i , podemos sacarlo fuera de la suma:

$$\sum_{i=1}^K p_i = \frac{1}{\sum_{j=1}^K e^{z_j}} \left(\sum_{i=1}^K e^{z_i} \right)$$

Observamos que la suma en el numerador $\sum_{i=1}^K e^{z_i}$ es idéntica a la suma en el denominador $\sum_{j=1}^K e^{z_j}$ (solo cambia el nombre del índice de sumación, lo cual no altera el resultado de la suma sobre todos los componentes). Por lo tanto, simplificamos la expresión a:

$$\frac{\sum_{i=1}^K e^{z_i}}{\sum_{j=1}^K e^{z_j}} = 1$$

Esto completa la demostración. $\square$

2.5.1 Apunte sobre estabilidad numérica

Para mejorar la estabilidad numérica al calcular la Softmax, especialmente cuando los valores de entrada x_i son grandes (lo que podría llevar a que e^{x_i} cause un desbordamiento), restamos el valor máximo del vector de entrada de cada elemento *antes* de aplicar la exponencial.

Definimos $m = \max\{x_1, x_2, \dots, x_N\}$. Entonces, calculamos:

$$\text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_{j=1}^N e^{x_j - m}} = \frac{e^{-m} \sum_{i=1}^N e^{x_i}}{e^{-m} \sum_{j=1}^N e^{x_j}} = \frac{\sum_{i=1}^N e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$

Esto es matemáticamente equivalente pero numéricamente estable. Al restar m , el exponente más grande será 0 (ya que para algún k , $x_k = m$, entonces $x_k - m = 0$), y $e^0 = 1$. Todos los demás exponentes serán negativos, resultando en valores $e^{x_j - m}$ entre $(0, 1]$. Esto evita el desbordamiento de los términos exponenciales y también ayuda a prevenir el subdesbordamiento en el denominador (ya que al menos un término en la suma será 1).

Ejemplo 3 (Estabilidad de Softmax con $K = 3$). Consideremos un vector de logits (valores de entrada) $\mathbf{x} = (1, 2, 5)^T$.

■ **Cálculo estándar (potencialmente inestable):**

- $e^1 \approx 2,718$
- $e^2 \approx 7,389$
- $e^5 \approx 148,413$

Suma de exponenciales: $S_E = 2,718 + 7,389 + 148,413 = 158,520$.

$$\text{softmax}(x_1) = \frac{2,718}{158,520} \approx 0,017$$

$$\text{softmax}(x_2) = \frac{7,389}{158,520} \approx 0,047$$

$$\text{softmax}(x_3) = \frac{148,413}{158,520} \approx 0,936$$

Vector de salida: $(0,017, 0,047, 0,936)^T$. (Suma $\approx 1,000$)

- **Cálculo numéricamente estable:** El valor máximo es $m = \max(1, 2, 5) = 5$. Restamos m de cada elemento: $\mathbf{x}' = (1 - 5, 2 - 5, 5 - 5)^T = (-4, -3, 0)^T$. Calculamos las exponenciales de $\mathbf{x}'$:

- $e^{-4} \approx 0,0183$
- $e^{-3} \approx 0,0498$
- $e^0 = 1,0000$

Suma de nuevas exponenciales: $S'_E = 0,0183 + 0,0498 + 1,0000 = 1,0681$.

$$\text{softmax}(x_1) = \frac{0,0183}{1,0681} \approx 0,017$$

$$\text{softmax}(x_2) = \frac{0,0498}{1,0681} \approx 0,047$$

$$\text{softmax}(x_3) = \frac{1,0000}{1,0681} \approx 0,936$$

Vector de salida: $(0,017, 0,047, 0,936)^T$. (Suma $\approx 1,000$)

Ambos métodos producen el mismo resultado (dentro de la precisión de redondeo). Sin embargo, si los valores de entrada fueran, por ejemplo, $(100, 102, 105)^T$, e^{105} excedería el límite de flotantes estándar, mientras que el método estable operaría con e^0, e^{-3}, e^{-5} , que son numéricamente manejables.

El vector de salida de la función Softmax, que denotamos como $\mathbf{y} = (y_1, \dots, y_K)^T$ donde cada componente $y_i = \text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_{j=1}^K e^{x_j - m}}$, tiene elementos y_i que son no negativos ($y_i \geq 0$) y suman a uno ($\sum_{i=1}^K y_i = 1$). Estas propiedades fundamentales (no negatividad y suma igual a uno) permiten que interpretemos el vector $\mathbf{y}$ como los parámetros de una distribución de probabilidad categórica (Definición 37) sobre K posibles resultados mutuamente excluyentes (Bishop, 2006).

2.6 MÉTODOS NUMÉRICOS

Los métodos numéricos desarrollan algoritmos para resolver problemas matemáticos que no podemos resolver analíticamente de manera sencilla o eficiente. Estos métodos son esenciales en la práctica computacional, como detallan Burden and Faires (2011) y Kincaid and Cheney (2002). Una técnica común es la interpolación, que estima valores desconocidos a partir de valores conocidos.

2.6.1 Interpolación lineal

Definimos la interpolación lineal como la forma más simple de interpolación, que aproxima una función entre dos puntos conocidos mediante una línea recta (Burden and Faires, 2011).

Definición 39 (Interpolación Lineal). *Dados dos puntos (x_0, y_0) y (x_1, y_1) en el plano cartesiano $\mathbb{R}^2$, con $x_0 \neq x_1$, la función de **interpolación lineal** que pasa por estos puntos es la única línea recta. Para cualquier valor x en el intervalo $[x_0, x_1]$, calculamos el valor interpolado $f(x)$ como:*

$$f(x) = y_0 + \frac{x - x_0}{x_1 - x_0}(y_1 - y_0)$$

Podemos reescribir esta fórmula como una combinación convexa de y_0 e y_1 :

$$f(x) = (1 - \alpha)y_0 + \alpha y_1$$

donde $\alpha = \frac{x - x_0}{x_1 - x_0}$. Notamos que α varía de 0 a 1 a medida que x varía de x_0 a x_1 .

Derivación de la fórmula de interpolación lineal. Buscamos la ecuación de la recta $y = mx + c$ que pasa por los puntos (x_0, y_0) y (x_1, y_1) . Calculamos la pendiente m de la recta como:

$$m = \frac{y_1 - y_0}{x_1 - x_0}$$

Usamos la forma punto-pendiente de la ecuación de una recta con el punto (x_0, y_0) :

$$y - y_0 = m(x - x_0)$$

Sustituyendo el valor de m :

$$y - y_0 = \frac{y_1 - y_0}{x_1 - x_0}(x - x_0)$$

Despejando y , que representa el valor interpolado $f(x)$:

$$f(x) = y = y_0 + \frac{x - x_0}{x_1 - x_0}(y_1 - y_0)$$

Esta es la primera fórmula dada en la [Definición 39](#).

Para obtener la segunda forma, definimos $\alpha = \frac{x - x_0}{x_1 - x_0}$. Entonces:

$$f(x) = y_0 + \alpha(y_1 - y_0)$$

$$f(x) = y_0 + \alpha y_1 - \alpha y_0$$

$$f(x) = (1 - \alpha)y_0 + \alpha y_1$$

Esta es la segunda fórmula. Cuando $x = x_0$, $\alpha = \frac{x_0 - x_0}{x_1 - x_0} = 0$, y $f(x_0) = (1 - 0)y_0 + 0 \cdot y_1 = y_0$. Cuando $x = x_1$, $\alpha = \frac{x_1 - x_0}{x_1 - x_0} = 1$, y $f(x_1) = (1 - 1)y_0 + 1 \cdot y_1 = y_1$. La fórmula interpola correctamente los puntos dados. $\square$

Utilizamos este método comúnmente para estimar valores intermedios cuando solo conocemos puntos discretos.

2.6.2 Limitaciones de la Interpolación Lineal

Pese a su simplicidad, la interpolación lineal presenta varias limitaciones (Burden and Faires, 2011; Kincaid and Cheney, 2002):

- **No diferenciabilidad en nodos:** La función interpolante $P(x)$ es C^0 pero no C^1 en los nodos x_i , resultando en quiebres de pendiente.
- **Linealidad local asumida:** Presupone linealidad de la función subyacente $f(x)$ entre nodos; es imprecisa si $f''(x)$ es significativa.
- **Imprecisión en extrapolación:** Inadecuada para estimar valores fuera del rango de datos $[\min(x_i), \max(x_i)]$, pudiendo generar errores grandes.
- **Error proporcional a $h^2 \max |f''(x)|$:** El error depende del espaciado h entre puntos y de la curvatura de $f(x)$.
- **Evita el fenómeno de Runge:** A diferencia de polinomios globales de alto grado, la interpolación lineal por partes no sufre de oscilaciones espurias (Burden and Faires, 2011).

A pesar de ello, es útil para aproximaciones rápidas si la función es localmente lineal. Para mayor suavidad o complejidad, se prefieren métodos de orden superior (polinomios globales, splines, interpolación trigonométrica) (Kincaid and Cheney, 2002).

2.7 TEOREMA DE APROXIMACIÓN UNIVERSAL

Establecemos el Teorema de Aproximación Universal de Cybenko (1989), que es un resultado fundamental en la teoría de redes neuronales. Sea $I_n = [0, 1]^n$ el hipercubo unitario n -dimensional. Nuestro objetivo es demostrar que las funciones de la forma:

$$G(\mathbf{x}) = \sum_{j=1}^N \alpha_j \sigma(\mathbf{y}_j^T \mathbf{x} + \theta_j)$$

donde $\mathbf{x}, \mathbf{y}_j \in \mathbb{R}^n$, $\alpha_j, \theta_j \in \mathbb{R}$, $N \in \mathbb{N}$, y σ es una función de activación adecuada, son densas en $C(I_n)$.

Definición 40 (Función Sigmoidal (según Cybenko)). Definimos una función $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ como *sigmoidal* (Cybenko, 1989) si

$$\sigma(t) \rightarrow \begin{cases} 1 & \text{si } t \rightarrow +\infty \\ 0 & \text{si } t \rightarrow -\infty \end{cases}$$

Cybenko (1989) considera funciones sigmoideas continuas.

Definición 41 (Función Discriminatoria (según Cybenko)). Definimos una función continua $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ como **discriminatoria** (Cybenko, 1989) si para cualquier medida $\mu \in M(I_n)$ (medida de Borel signada regular finita sobre I_n), la condición

$$\int_{I_n} \sigma(\mathbf{y}^T \mathbf{x} + \theta) d\mu(\mathbf{x}) = 0 \quad \text{para todo } \mathbf{y} \in \mathbb{R}^n, \theta \in \mathbb{R}$$

implica que $\mu = 0$ (la medida nula).

Lema 1 (Lema 1 de Cybenko: Sigmoidal $\implies$ Discriminatoria). (Cybenko, 1989, Lemma 1). Toda función $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ continua y sigmoidal (según Def. 40) es discriminatoria (según Def. 41).

Esquema de la Demostración del Lema 1 (basado en Cybenko (1989)). 1. Asumimos que

$$\int_{I_n} \sigma(\mathbf{y}^T \mathbf{x} + \theta) d\mu(\mathbf{x}) = 0 \quad \text{para todo } \mathbf{y}, \theta.$$

2. Consideramos la familia de funciones $\sigma_{\lambda, \phi}(u) = \sigma(\lambda u + \phi)$ para $\lambda > 0$. Entonces $\int_{I_n} \sigma(\lambda(\mathbf{y}^T \mathbf{x} + \theta_0) + \phi) d\mu(\mathbf{x}) = 0$ para $\mathbf{y}, \theta_0, \phi$ fijos y todo λ . (Obtenemos esto de la hipótesis tomando un $\mathbf{y}'$ adecuado y θ').
3. Sea $u = \mathbf{y}^T \mathbf{x} + \theta_0$. A medida que $\lambda \rightarrow \infty$:

$$\sigma(\lambda u + \phi) \rightarrow \gamma_\phi(u) = \begin{cases} 1 & \text{si } u > 0 \\ 0 & \text{si } u < 0 \\ \sigma(\phi) & \text{si } u = 0 \end{cases}$$

Esta convergencia es puntual. Como σ es continua y sigmoidal, es acotada. Por el Teorema de Convergencia Dominada de Lebesgue (Teorema 4, Rudin 1987), podemos intercambiar el límite y la integral:

$$\int_{I_n} \gamma_\phi(\mathbf{y}^T \mathbf{x} + \theta_0) d\mu(\mathbf{x}) = 0$$

Esto es $\mu(H_{\mathbf{y}, \theta_0}^+) + \sigma(\phi)\mu(\Pi_{\mathbf{y}, \theta_0}) = 0$, donde $H_{\mathbf{y}, \theta_0}^+ = \{\mathbf{x} \in I_n : \mathbf{y}^T \mathbf{x} + \theta_0 > 0\}$ y $\Pi_{\mathbf{y}, \theta_0} = \{\mathbf{x} \in I_n : \mathbf{y}^T \mathbf{x} + \theta_0 = 0\}$.

4. Tomando el límite cuando $\phi \rightarrow -\infty$, tenemos $\sigma(\phi) \rightarrow 0$. Entonces $\mu(H_{\mathbf{y}, \theta_0}^+) = 0$.
5. De forma similar, tomando $\phi \rightarrow +\infty$, $\sigma(\phi) \rightarrow 1$. Usando una transformación adecuada ($1 - \sigma$ en lugar de σ para definir una función sigmoidal que va de 1 a 0, y ajustando los argumentos), podemos mostrar que $\mu(H_{\mathbf{y}, \theta_0}^-) = 0$, donde $H_{\mathbf{y}, \theta_0}^- = \{\mathbf{x} \in I_n : \mathbf{y}^T \mathbf{x} + \theta_0 < 0\}$.
6. Dado que $\mu(I_n) = \mu(H_{\mathbf{y}, \theta_0}^+) + \mu(H_{\mathbf{y}, \theta_0}^-) + \mu(\Pi_{\mathbf{y}, \theta_0})$, deducimos que $\mu(\Pi_{\mathbf{y}, \theta_0})$ también debe ser cero para casi todo θ_0 si $\mathbf{y} \neq \mathbf{0}$.
7. El hecho de que la medida de todos los semiplanos abiertos sea cero ($\mu(H_{\mathbf{y}, \theta_0}^+) = 0$ y $\mu(H_{\mathbf{y}, \theta_0}^-) = 0$ para todo $\mathbf{y} \neq \mathbf{0}$ y casi todo θ_0) implica que la Transformada de Fourier de la medida μ es idénticamente cero para $\xi \neq \mathbf{0}$.

$$\hat{\mu}(\xi) = \int_{I_n} e^{-2\pi i \xi^T \mathbf{x}} d\mu(\mathbf{x}) = 0 \quad \text{para todo } \xi \in \mathbb{R}^n \setminus \{\mathbf{0}\}$$

(Este paso requiere un argumento técnico que [Cybenko \(1989\)](#) detalla, mostrando que la integral de $e^{i\boldsymbol{\xi}^T \mathbf{x}}$ contra $d\mu(\mathbf{x})$ es cero usando la relación con las funciones indicadoras de semiplanos. La anulación para $\boldsymbol{\xi} = \mathbf{0}$ se sigue de $\mu(I_n) = 0$, que se deriva de los pasos anteriores).

8. Si la Transformada de Fourier de una medida finita es cero, entonces la medida misma es cero ($\mu = 0$) por la unicidad de la Transformada de Fourier ([Rudin, 1987](#), Capítulo 9).
9. Por lo tanto, concluimos que σ es discriminatoria.

□

Teorema 5 (Teorema de Aproximación Universal). ([Cybenko, 1989](#)). Sea $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ una función continua y discriminatoria (por Lema 1, esto incluye a las sigmoides continuas). Entonces, las sumas finitas de la forma

$$G(\mathbf{x}) = \sum_{j=1}^N \alpha_j \sigma(\mathbf{y}_j^T \mathbf{x} + \theta_j)$$

son densas en $C(I_n)$ con respecto a la norma del supremo.

Esquema de la Demostración del Teorema 5 (basado en [Cybenko \(1989\)](#)). 1. Sea S el conjunto de todas las funciones $G(\mathbf{x})$ de la forma dada. S es un subespacio vectorial de $C(I_n)$ (no necesariamente cerrado).

2. Queremos demostrar que $\bar{S} = C(I_n)$. Procedemos por contradicción.
3. Suponemos que $\bar{S} \neq C(I_n)$. Entonces $\bar{S}$ es un subespacio propio cerrado de $C(I_n)$.
4. Por el Teorema de Hahn-Banach ([Teorema 2, Kreyszig 1989](#)), existe un funcional lineal acotado $L \in (C(I_n))^*$ tal que $L \neq 0$ (el funcional nulo) y $L(g) = 0$ para todo $g \in \bar{S}$ (y por lo tanto para todo $g \in S$).
5. Por el Teorema de Representación de Riesz-Markov-Kakutani ([Teorema 3, Rudin 1987](#)), existe una medida de Borel signada regular finita no nula $\mu \in M(I_n)$ (ya que $L \neq 0$) tal que para toda función $h \in C(I_n)$:

$$L(h) = \int_{I_n} h(\mathbf{x}) d\mu(\mathbf{x})$$

6. En particular, para cualquier función de la forma $s(\mathbf{x}) = \sigma(\mathbf{y}^T \mathbf{x} + \theta)$ (que es un elemento de S tomando $N = 1, \alpha_1 = 1$), tenemos que $L(s(\mathbf{x})) = 0$. Es decir:

$$\int_{I_n} \sigma(\mathbf{y}^T \mathbf{x} + \theta) d\mu(\mathbf{x}) = 0 \quad \text{para todo } \mathbf{y} \in \mathbb{R}^n, \theta \in \mathbb{R}$$

7. Dado que asumimos que σ es una función discriminatoria ([Definición 41](#)), la condición anterior implica que la medida μ debe ser la medida nula ($\mu = 0$).
8. Si $\mu = 0$, entonces el funcional L debe ser el funcional nulo ($L(h) = \int h d\mu = 0$ para todo h). Esto contradice el paso 4, donde establecimos que $L \neq 0$.

9. Por lo tanto, la suposición inicial (que $\bar{S} \neq C(I_n)$) debe ser falsa.
10. Concluimos que $\bar{S} = C(I_n)$, es decir, las sumas finitas de la forma $G(\mathbf{x})$ son densas en $C(I_n)$.

□

DEL APRENDIZAJE AUTOMÁTICO AL APRENDIZAJE PROFUNDO

Este capítulo define inicialmente el ámbito del ML y lo sitúa como marco de referencia para el DL. A continuación, explica qué implica que una máquina aprenda y expone los principales paradigmas de aprendizaje. Después, presenta los modelos y algoritmos fundamentales que han servido de cimiento a las técnicas modernas. Por último, examina la evolución de estos fundamentos hasta dar origen a arquitecturas actuales, como los Transformers.

3.1 ¿QUÉ ES EL APRENDIZAJE AUTOMÁTICO?

El ML es una rama de la IA y de las ciencias de la computación que se centra en el uso de datos y algoritmos para imitar la forma en que los humanos aprenden, mejorando gradualmente su precisión. Dicho de otro modo, dota a los sistemas de la capacidad de aprender y mejorar a partir de la experiencia (conjuntos de datos) sin ser explícitamente programados para cada tarea específica ([Goodfellow et al., 2016a](#)).

La esencia del ML reside en descubrir patrones subyacentes en los datos. Dada una tarea y un conjunto de datos relevantes, el objetivo es desarrollar un modelo matemático que pueda generalizar su aprendizaje a nuevos datos no vistos previamente, permitiendo realizar predicciones o tomar decisiones informadas. Los componentes fundamentales de un problema de aprendizaje son, como se detalla en [Abu-Mostafa et al. \(2012a\)](#):

- El **conjunto de datos** (*dataset*): Colección de ejemplos (observaciones, instancias) que sirven como base empírica para el aprendizaje. Cada ejemplo suele consistir en un conjunto de características (*features*) y, en algunos paradigmas, una etiqueta o valor objetivo asociado.
- El **modelo o hipótesis** (*model/hypothesis*): Una representación matemática (por ejemplo, una función o una distribución de probabilidad) que intenta capturar la relación entre las entradas y las salidas, o la estructura inherente de los datos. El modelo pertenece a un conjunto predefinido de posibles modelos, denominado espacio de hipótesis ($\mathcal{H}$).

- El **algoritmo de aprendizaje** (*learning algorithm*): El procedimiento utilizado para ajustar los parámetros del modelo (o seleccionar la mejor hipótesis dentro del espacio $\mathcal{H}$) basándose en los patrones identificados en los datos de entrenamiento.

Este capítulo se centrará en los fundamentos de estos componentes, estableciendo el marco necesario para comprender técnicas aprendizaje automático.

3.2 PARADIGMAS DEL APRENDIZAJE AUTOMÁTICO

El ML comprende diversos enfoques o paradigmas, clasificados según la naturaleza de los datos de entrenamiento y el tipo de retroalimentación disponible. La taxonomía más extendida distingue tres categorías principales: aprendizaje supervisado, aprendizaje no supervisado y aprendizaje por refuerzo (Bishop, 2006). Este trabajo se centrará predominantemente en los dos primeros, que son los más relevantes para el problema de imputación de datos.

3.2.1 Aprendizaje Supervisado

En el aprendizaje supervisado, el algoritmo aprende a partir de un conjunto de datos de entrenamiento donde cada instancia $(\mathbf{x}_n, y_n)$ se compone de un vector de características de entrada $\mathbf{x}_n \in \mathcal{X}$ y una etiqueta o valor objetivo conocido $y_n \in \mathcal{Y}$ (Goodfellow et al., 2016a; Hastie et al., 2009). Se proporciona, por tanto, el resultado esperado para cada entrada durante la fase de entrenamiento. El objetivo principal es inferir una función de mapeo $g : \mathcal{X} \rightarrow \mathcal{Y}$ que aproxime de manera óptima la función objetivo subyacente f (desconocida) y que sea capaz de generalizar, prediciendo la salida para nuevas entradas $\mathbf{x}$ no observadas previamente.

Dentro del aprendizaje supervisado, se distinguen principalmente dos tipos de tareas (Alpaydin, 2020):

- **Clasificación:** La variable objetivo y_n es una etiqueta categórica discreta, perteneciente a un conjunto finito de clases predefinidas. El modelo aprende a asignar nuevas instancias a una de estas categorías. Ejemplos de algoritmos son:
 - **Regresión Logística:** A pesar de su nombre, es un modelo lineal para clasificación binaria (y extensible a multiclase) que modela la probabilidad de pertenencia a una clase mediante una función logística (sigmoide) (Cox, 1958; Jr. et al., 2013).
 - **Máquinas de Soporte Vectorial (SVM):** Buscan encontrar un hiperplano óptimo que separe las clases en el espacio de características, maximizando el margen entre ellas. Pueden usar el truco del kernel para manejar datos no separables linealmente (Cortes and Vapnik, 1995; Boser et al., 1992).

- **Árboles de Decisión y Bosques Aleatorios (Random Forests):** Los árboles de decisión construyen un modelo predictivo basado en una serie de reglas de división jerárquicas. Los Bosques Aleatorios son un conjunto (ensemble) de múltiples árboles de decisión, que mejoran la robustez y reducen el sobreajuste (Breiman et al., 1984; Breiman, 2001).
- **K-Vecinos más Cercanos (K-NN):** Un método no paramétrico que clasifica una nueva instancia basándose en la clase mayoritaria de sus k vecinos más cercanos en el espacio de características (Cover and Hart, 1967).
- **Redes Neuronales Artificiales (ANN):** Incluyendo Perceptrones Multicapa (MLP) y arquitecturas más profundas, que aprenden representaciones jerárquicas para mapear entradas a clases (e.g., Rumelhart et al., 1986b; LeCun et al., 2015).
- **Regresión:** La variable objetivo y_n es un valor continuo o real. El modelo aprende a predecir un valor numérico para nuevas instancias. Algoritmos representativos de regresión son:
 - **Regresión Lineal:** Asume una relación lineal entre las características de entrada y la variable objetivo. Sus parámetros se estiman típicamente mediante mínimos cuadrados (Galton, 1886; Montgomery et al., 2021).
 - **Regresión Polinomial:** Una extensión de la regresión lineal que permite modelar relaciones no lineales añadiendo términos polinomiales de las características de entrada (Hastie et al., 2009).
 - **Árboles de Regresión y Bosques Aleatorios de Regresión:** Análogos a sus contrapartes de clasificación, pero adaptados para predecir valores continuos (Breiman et al., 1984; Breiman, 2001).
 - **Máquinas de Soporte Vectorial para Regresión (SVR):** Una adaptación de las SVM para tareas de regresión, que intenta ajustar una función a los datos dentro de un cierto margen de error (Drucker et al., 1997).
 - **Redes Neuronales Artificiales (ANN):** También aplicables a regresión, donde la capa de salida predice un valor continuo (e.g., Goodfellow et al., 2016a).

3.2.2 Aprendizaje No Supervisado

En el aprendizaje no supervisado, el algoritmo procesa un conjunto de datos de entrada $\{\mathbf{x}_n\}$ que carecen de etiquetas o valores objetivo predefinidos asociados (Barlow, 1989; Hastie et al., 2009). El propósito fundamental es descubrir patrones intrínsecos, estructuras latentes, o relaciones inherentes en los propios datos sin una guía explícita sobre cuál debería ser la salida esperada.

Las tareas más comunes dentro del aprendizaje no supervisado incluyen (Bishop, 2006; James et al., 2013):

- **Agrupamiento (Clustering):** Consiste en organizar los datos en grupos o clústeres, de manera que las instancias dentro de un mismo clúster presenten una alta similitud entre sí y una baja similitud con las instancias de otros clústeres. Ejemplos de algoritmos y aplicaciones son:
 - El algoritmo K-Means (MacQueen, 1967; Lloyd, 1982), utilizado para segmentar clientes según su comportamiento de compra.
 - DBSCAN (Density-Based Spatial Clustering of Applications with Noise) (Ester et al., 1996), capaz de encontrar clústeres de forma arbitraria y manejar ruido, útil en la detección de anomalías geoespaciales.
 - Agrupamiento jerárquico (Johnson, 1967), que construye una jerarquía de clústeres, aplicado en taxonomías biológicas.
- **Reducción de Dimensionalidad:** Busca transformar los datos desde un espacio de alta dimensionalidad a uno de menor dimensión, intentando preservar la máxima cantidad de información relevante o la estructura esencial de los datos originales. Técnicas destacadas son:
 - Análisis de Componentes Principales (PCA) (Pearson, 1901; Hotelling, 1933), ampliamente usado para compresión de datos y visualización.
 - t-distributed Stochastic Neighbor Embedding (t-SNE) (van der Maaten and Hinton, 2008), una técnica popular para la visualización de datos de alta dimensión en espacios de baja dimensión (generalmente 2D o 3D).
 - Autoencoders (Hinton and Salakhutdinov, 2006; Bengio, 2009), redes neuronales entrenadas para reconstruir su entrada, cuya capa intermedia aprende una representación comprimida.
- **Estimación de Densidad:** Persigue construir un modelo de la distribución de probabilidad subyacente a partir de la cual se generaron los datos observados. Enfoques comunes incluyen:
 - Estimación de Densidad por Kernel (KDE) (Parzen, 1962; Rosenblatt, 1956), un método no paramétrico para estimar la función de densidad de probabilidad.
 - Modelos de Mezcla Gaussiana (GMM) (Dempster et al., 1977), que asumen que los datos son generados a partir de una mezcla de varias distribuciones gaussianas.

3.3 EVALUACIÓN DE MODELOS Y TRATAMIENTO DE DATOS FALTANTES

Una vez que un algoritmo de aprendizaje ha generado una hipótesis final g , es importante evaluar su rendimiento. Esta evaluación se centra en la **generalización**, que se refiere a la capacidad del modelo para realizar predicciones precisas sobre datos nuevos y no vistos previamente, en lugar de simplemente ajustarse bien a los datos con los que fue entrenado (Goodfellow et al., 2016a). Una buena generalización es el objetivo último de la mayoría de los modelos de ML .

3.3.1 Métricas de Evaluación

La elección de la métrica de evaluación depende de la tarea (clasificación o regresión) y de los objetivos específicos del problema (Bishop, 2006). Un buen modelo no solo debe ajustarse bien a los datos de entrenamiento (minimizar el error *in-sample*, E_{in}), sino, y más importante, generalizar bien a datos no vistos (minimizar el error *out-of-sample*, E_{out}) (Abu-Mostafa et al., 2012a).

Métricas para clasificación

En tareas de clasificación (especialmente binaria: clases positiva/negativa), el rendimiento se evalúa a partir de cuatro resultados posibles, resumidos en la matriz de confusión (Fawcett, 2006):

- **Verdadero Positivo (VP o TP):** Clase positiva correctamente clasificada como positiva.
- **Verdadero Negativo (VN o TN):** Clase negativa correctamente clasificada como negativa.
- **Falso Positivo (FP):** También denominado error de Tipo I. Clase negativa incorrectamente clasificada como positiva.
- **Falso Negativo (FN):** También denominado error de Tipo II. Clase positiva incorrectamente clasificada como negativa.

Algunas métricas comunes, presentadas aquí con sus formulaciones básicas, son:

- **Matriz de Confusión:** Tabla que desglosa los aciertos y errores por clase (VP, VN, FP, FN), proporcionando una visión detallada del rendimiento del clasificador (Fawcett, 2006).
- **Accuracy (Exactitud):** Proporción de predicciones correctas sobre el total de predicciones. Simple pero potencialmente engañosa en datasets desbalanceados (Powers, 2011).

$$\text{Accuracy} = \frac{\text{VP} + \text{VN}}{\text{VP} + \text{VN} + \text{FP} + \text{FN}}$$

- **Precisión:** Fracción de predicciones positivas que fueron correctas (Goutte and Gaussier, 2005).

$$\text{Precisión} = \frac{\text{VP}}{\text{VP} + \text{FP}}$$

- **Recall (Sensibilidad o Exhaustividad):** Fracción de positivos reales que se identificaron correctamente (Goutte and Gaussier, 2005).

$$\text{Recall} = \frac{\text{VP}}{\text{VP} + \text{FN}}$$

- *F1-Score*: Media armónica de la precisión y el recall, buscando un balance entre ambas (Goutte and Gaussier, 2005).

$$F1\text{-Score} = 2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}}$$

- *Área Bajo la Curva ROC (AUC-ROC)*: Evalúa la capacidad del modelo para discriminar entre clases a diferentes umbrales de decisión. Un valor cercano a 1 indica un excelente poder discriminatorio (Bradley, 1997). La curva ROC representa la Tasa de Verdaderos Positivos frente a la Tasa de Falsos Positivos a varios umbrales, como se ilustra en la Figura 1.



Figura 1: Ejemplo de una curva ROC (Badat, 2020)

Métricas para regresión

Sea y_i el valor real y $\hat{y}_i$ el valor predicho para la i -ésima observación, con N observaciones en total.

- *Error Cuadrático Medio* (MSE, Mean Squared Error): Promedio de los errores al cuadrado. Penaliza fuertemente los errores grandes y es sensible a outliers (Lehmann and Casella, 1998).

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- *Raíz del Error Cuadrático Medio* (RMSE, Root Mean Squared Error): Raíz cuadrada del MSE, expresada en las mismas unidades que la variable objetivo, lo que facilita su interpretación (Chai and Draxler, 2014a).

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} = \sqrt{\text{MSE}}$$

- *Error Absoluto Medio* (MAE, Mean Absolute Error): Promedio de los errores absolutos. Más robusto a outliers que el MSE (Willmott and Matsuura, 2005).

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- *Error Relativo Medio* (MRE, Mean Relative Error): Calcula el error como una proporción del valor real, útil para entender la magnitud del error respecto a la escala de los datos (Hyndman and Koehler, 2006). Para estabilidad numérica, se añade una pequeña constante ϵ al denominador. Una formulación común es:

$$\text{MRE} = \frac{1}{N} \sum_{i=1}^N \frac{|y_i - \hat{y}_i|}{|y_i| + \epsilon}$$

- *Coefficiente de Determinación* (R^2): Proporción de la varianza de la variable objetivo que es predecible a partir de las variables independientes. Varía entre 0 y 1 (o incluso negativo para modelos muy malos) (Nagelkerke, 1991).

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} = 1 - \frac{\text{MSE}}{\text{Var}(y)}$$

donde $\bar{y}$ es la media de los valores reales y_i .

La selección y correcta interpretación de estas métricas es fundamental para el desarrollo y despliegue de modelos de ML (Japkowicz and Shah, 2011).

3.4 DE MODELOS LINEALES A ARQUITECTURAS DE ATENCIÓN PROFUNDA

Los modelos lineales, aunque fundamentales, a menudo no capturan la complejidad inherente de muchos problemas del mundo real, especialmente aquellos con relaciones no lineales o interacciones complejas entre variables (Bishop, 2006; Hastie et al., 2009). Esta sección traza una evolución desde estos modelos básicos hacia arquitecturas más sofisticadas, capaces de aprender representaciones jerárquicas y manejar dependencias complejas en los datos, como las que se encuentran en secuencias.

3.4.1 Modelos Lineales Fundamentales

Los modelos lineales son un pilar fundamental del ML y la estadística (Montgomery et al., 2021; Weisberg, 2005). Su simplicidad matemática permite un análisis profundo, una interpretación relativamente sencilla de los coeficientes y, a menudo, soluciones analíticas o algoritmos de optimización convexos con garantías de convergencia (Boyd and Vandenberghe, 2004). Sirven además como bloques de construcción y puntos de referencia esenciales para el desarrollo y la evaluación de modelos más complejos (Goodfellow et al., 2016a).

El Perceptrón

El Perceptrón es un modelo histórico para clasificación binaria (Abu-Mostafa et al., 2012a). Representa una neurona artificial que calcula una suma ponderada de sus entradas $\mathbf{x}$ y aplica una función de activación escalón (la función signo) para producir una salida binaria: $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$. Aunque limitado a problemas linealmente separables, introdujo conceptos clave como el aprendizaje iterativo basado en errores.

Regresión Lineal

La regresión lineal modela una variable objetivo real y como una combinación lineal de características: $h(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$. Los pesos $\mathbf{w}$ se encuentran minimizando el error cuadrático medio (MSE), lo que tiene una solución analítica mediante las ecuaciones normales (Bishop, 2006).

Descenso del Gradiente

Para modelos donde no existe solución analítica, como la regresión logística o las redes neuronales, el *descenso del gradiente* es un algoritmo iterativo clave. Actualiza los pesos $\mathbf{w}$ en la dirección opuesta al gradiente del error: $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \nabla E(\mathbf{w}^{(t)})$.

El descenso de gradiente estocástico (SGD) es una variante eficiente, esencial para el entrenamiento de modelos de AP.

3.4.2 De Perceptrones Simples a Redes Neuronales Profundas

El modelo del Perceptrón se enfrenta a un muro con datos no linealmente separables. La necesidad de modelar relaciones más complejas impulsó la idea de combinar múltiples perceptrones, dando origen al **Perceptrón Multicapa (MLP)**. En un MLP, las salidas de una capa de unidades (neuronas) se convierten en las entradas de otra. Esta arquitectura, donde la información fluye unidireccionalmente desde la entrada hacia la salida, sin ciclos, se conoce formalmente como una **Feed-Forward Neural Network (FFNN)**, como se ilustra en la Figura 2.

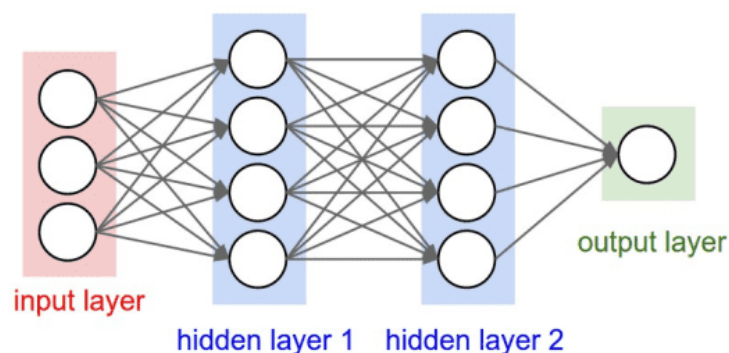


Figura 2: Diagrama de una Red Neuronal Feed-Forward (FFNN) (Reyes-Ochoa, 2020)

En una FFNN, cada neurona típicamente reemplaza la función escalón por una función de activación no lineal suave y diferenciable (e.g., sigmoide, ReLU). Esta diferenciable permite aplicar el Descenso del Gradiente para ajustar todos los pesos de la red mediante el algoritmo de **retropropagación (backpropagation)**, una aplicación eficiente de la regla de la cadena. La profundidad (número de capas ocultas) permite aprender jerarquías de características, siendo la base del **Deep Learning**.

Si bien las FFNN son potentes, su estructura totalmente conectada puede no ser óptima para datos secuenciales, donde arquitecturas como las Redes Neuronales Recurrentes (RNNs) fueron inicialmente preferidas. Sin embargo, las RNNs tenían dificultades con secuencias largas, tendiendo a olvidar el inicio y su naturaleza recurrente impide la paralelización, haciéndolas lentas de entrenar. Estas limitaciones prepararon el terreno para una nueva arquitectura.

3.4.3 Redes Neuronales Profundas con atención: Transformer

Los modelos **Transformer** (Vaswani et al., 2017b) surgieron como una solución disruptiva, especialmente en el Procesamiento del Lenguaje Natural (NLP), al solo basarse

en mecanismos de atención para recordar, abandonando la recurrencia. La **atención** se refiere a la capacidad de un modelo para prestar atención a la parte importante de una entrada. Esto permite que el modelo alimente todas las palabras de la oración al mismo tiempo a la red, facilitando una paralelización y eficiencia sin precedentes. La Figura 3 ilustra la arquitectura conceptual.

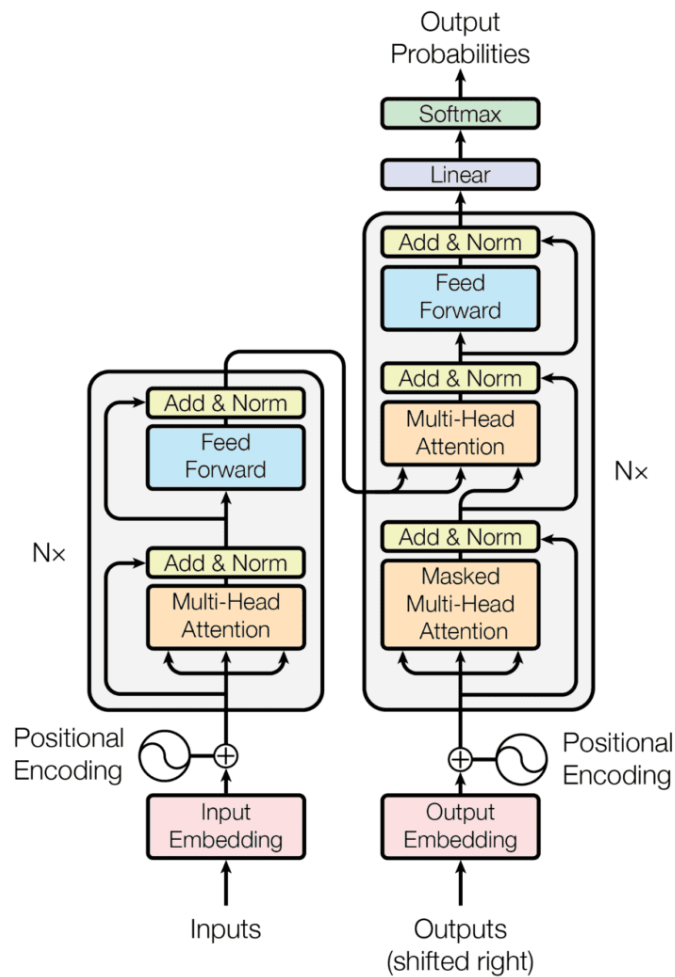


Figura 3: Diagrama conceptual de la arquitectura Transformer (Vaswani et al., 2017b)

Componentes Fundamentales del Transformer

Antes de detallar la arquitectura Encoder-Decoder, definiremos las diferentes capas que la componen: Embeddings y Encodings Posicionales, Mecanismo de Auto-Atención, Atención Multi-Cabeza, Redes Feed-Forward, Conexiones Residuales y Normalización de Capas.

EMBEDDINGS Y ENCODINGS POSICIONALES

En el procesamiento del lenguaje natural con Transformers, las secuencias de entrada, típicamente palabras o tokens, primero se transforman en representaciones vectoriales densas y de dimensión fija.

Los **embeddings de palabras** (o *tokens*) convierten cada palabra de un vocabulario V en un vector de alta dimensionalidad, usualmente d_{model} . Si w es una palabra, su embedding se representa como $\mathbf{e}_w \in \mathbb{R}^{d_{\text{model}}}$. Estos embeddings se aprenden durante el entrenamiento y están diseñados para capturar información semántica y sintáctica sobre las palabras (Mikolov et al., 2013; Pennington et al., 2014). Así, una secuencia de entrada de L tokens $(x_1, x_2, \dots, x_L)$ se mapea a una secuencia de vectores de embedding $(\mathbf{e}_{x_1}, \mathbf{e}_{x_2}, \dots, \mathbf{e}_{x_L})$.

Dado que la arquitectura Transformer, a diferencia de las Redes Neuronales Recurrentes (RNN) o las Redes Convolucionales (CNN), no procesa los tokens secuencialmente ni posee una noción inherente del orden o la posición de los tokens en la secuencia, es necesario inyectar explícitamente esta información. Para este fin, se utilizan los **encodings posicionales** (PE, Positional Encodings). Estos encodings se suman a los embeddings de los tokens correspondientes antes de ser alimentados a la primera capa del codificador o decodificador (Vaswani et al., 2017a).

El encoding posicional para un token en la posición pos de la secuencia y para la dimensión i del vector de embedding se define mediante funciones seno y coseno de diferentes frecuencias, como sigue:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (3)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (4)$$

donde pos es la posición del token en la secuencia (desde 0 hasta $L - 1$), i es el índice de la dimensión dentro del vector de encoding posicional (desde 0 hasta $d_{\text{model}}/2 - 1$), y d_{model} es la dimensionalidad de los embeddings (y, por tanto, de los encodings posicionales). Cada dimensión del encoding posicional corresponde a una senoide. Las longitudes de onda forman una progresión geométrica desde 2π hasta $10000 \cdot 2\pi$.

Esta elección de funciones sinusoidales permite al modelo aprender a atender a posiciones relativas, ya que para cualquier desfase fijo k , PE_{pos+k} puede ser representado

como una función lineal de PE_{pos} (Vaswani et al., 2017a). Además, estas funciones permiten al modelo extrapolar a longitudes de secuencia mayores que las vistas durante el entrenamiento.

El vector de embedding posicional $\mathbf{p}_{pos} \in \mathbb{R}^{d_{\text{model}}}$ para la posición pos se obtiene concatenando o intercalando estas componentes. La representación final del token x_t en la posición pos_t antes de entrar en la primera capa del Transformer es la suma de su embedding de palabra y su encoding posicional:

$$\mathbf{x}_{\text{input},t} = \mathbf{e}_{x_t} + \mathbf{p}_{pos_t} \quad (5)$$

Esta suma permite que la información posicional se integre directamente con la representación semántica del token.

MECANISMO DE AUTO-ATENCIÓN (SELF-ATTENTION)

La otra idea clave es la auto-atención, específicamente la **Scaled Dot-Product Attention**. Para cada palabra (embedding + encoding posicional), se generan tres vectores aprendibles: **Consulta (Query, Q)**, **Clave (Key, K)** y **Valor (Value, V)** mediante proyecciones lineales. La atención se calcula como:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (6)$$

donde d_k es la dimensión de las claves. El producto QK^T calcula puntuaciones de similitud, el escalado por $\sqrt{d_k}$ estabiliza los gradientes, softmax normaliza estas puntuaciones a pesos de atención, y finalmente estos pesos ponderan los vectores V para producir una representación contextualizada de cada palabra.

ATENCIÓN MULTI-CABEZA (MULTI-HEAD ATTENTION)

En lugar de realizar una única función de atención con consultas Q , claves K y valores V de dimensión d_{model} , el mecanismo de Atención Multi-Cabeza (MHA) proyecta linealmente estas entradas h veces (donde h es el número de cabezas de atención) con diferentes conjuntos de pesos aprendidos. Luego, la función de atención (Scaled Dot-Product Attention) se aplica en paralelo a cada una de estas proyecciones. Este enfoque permite al modelo atender conjuntamente a información de diferentes subespacios de representación y en diferentes posiciones (Vaswani et al., 2017a).

Formalmente, para cada cabeza $j \in \{1, \dots, h\}$, se calculan las proyecciones:

$$Q_j = QW_j^Q \quad (7)$$

$$K_j = KW_j^K \quad (8)$$

$$V_j = VW_j^V \quad (9)$$

donde $Q, K, V \in \mathbb{R}^{L \times d_{\text{model}}}$ son las matrices de entrada (con L como la longitud de la secuencia), y las matrices de pesos de proyección son $W_j^Q \in \mathbb{R}^{d_{\text{model}} \times d_q}$, $W_j^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, y $W_j^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. Típicamente, las dimensiones de las proyecciones para cada cabeza son menores que la dimensión del modelo: $d_q = d_k = d_v = d_{\text{model}}/h$.

Luego, la función de atención (ecuación 6) se aplica independientemente a cada cabeza:

$$\text{head}_j = \text{Attention}(Q_j, K_j, V_j) = \text{softmax} \left(\frac{Q_j K_j^T}{\sqrt{d_k}} \right) V_j \quad (10)$$

donde $\text{head}_j \in \mathbb{R}^{L \times d_v}$.

Las salidas de las h cabezas de atención se concatenan para formar una única matriz:

$$\text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) = [\text{head}_1 || \text{head}_2 || \dots || \text{head}_h] \quad (11)$$

donde la matriz concatenada tiene dimensiones $L \times (h \cdot d_v)$. Dado que $h \cdot d_v = h \cdot (d_{\text{model}}/h) = d_{\text{model}}$, la dimensión de la matriz concatenada es $L \times d_{\text{model}}$.

Finalmente, esta matriz concatenada se proyecta linealmente una vez más mediante otra matriz de pesos aprendida $W^O \in \mathbb{R}^{(h \cdot d_v) \times d_{\text{model}}}$ (es decir, $W^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$) para producir la salida final del bloque de atención multi-cabeza:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (12)$$

El resultado, $\text{MultiHead}(Q, K, V)$, tiene dimensiones $L \times d_{\text{model}}$.

Este mecanismo permite al modelo, como se mencionó, prestar atención no solo a una palabra o aspecto, sino a múltiples palabras o, más precisamente, a diferentes subespacios de representación y tipos de relaciones de forma simultánea, enriqueciendo la capacidad del modelo para capturar dependencias complejas.

REDES FEED-FORWARD POSICIONALES

Cada capa del Transformer, tanto en el codificador como en el decodificador, incluye una subcapa de red neuronal feed-forward (FFNN) (Ver Sección 3.4.2) completamente conectada, que se aplica de forma idéntica e independiente a cada posición (token) de la secuencia tras el mecanismo de atención (Vaswani et al., 2017a).

Esta FFNN consiste típicamente en dos transformaciones lineales con una función de activación no lineal entre ellas. La entrada $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ (para cada posición) se transforma según:

$$\text{FFNN}(\mathbf{x}) = \text{ReLU}(\mathbf{x}W_1 + \mathbf{b}_1)W_2 + \mathbf{b}_2 \quad (13)$$

donde $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $\mathbf{b}_1 \in \mathbb{R}^{d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, y $\mathbf{b}_2 \in \mathbb{R}^{d_{\text{model}}}$ son parámetros aprendibles, y d_{ff} es la dimensionalidad interna (e.g., $4 \cdot d_{\text{model}}$).

La función de activación comúnmente empleada es la Unidad Lineal Rectificada (ReLU) (Nair and Hinton, 2010; Glorot et al., 2011), definida como $\text{ReLU}(z) = \max(0, z)$.

ReLU es computacionalmente eficiente y ayuda a mitigar el desvanecimiento de gradientes. Aunque los pesos de la FFNN son compartidos entre posiciones, esta opera sobre representaciones de tokens individuales, añadiendo capacidad de modelado no lineal a cada capa del Transformer.

CONEXIONES RESIDUALES Y NORMALIZACIÓN DE CAPAS

Para facilitar el entrenamiento de arquitecturas profundas como los Transformers y mitigar problemas como el desvanecimiento o explosión de gradientes, cada subcapa (ya sea la de atención multi-cabeza o la red neuronal feed-forward) dentro de un bloque del codificador o decodificador está envuelta por una **conexión residual** (He et al., 2016) seguida de una **normalización de capas** (Layer Normalization, LN) (Ba et al., 2016).

Conexión Residual: Si $\mathbf{x}$ es la entrada a una subcapa y $\text{SubLayer}(\mathbf{x})$ es la función implementada por la propia subcapa (por ejemplo, el mecanismo de atención multi-cabeza o la red feed-forward), la salida de la conexión residual es:

$$\mathbf{x}_{\text{residual}} = \mathbf{x} + \text{SubLayer}(\mathbf{x}) \quad (14)$$

Esta estructura permite que los gradientes fluyan más directamente a través de la red durante la retropropagación, facilitando el aprendizaje de funciones de identidad si es necesario y ayudando al modelo a no olvidar información de capas anteriores. La dimensionalidad de $\mathbf{x}$ y $\text{SubLayer}(\mathbf{x})$ debe ser la misma para permitir la suma directa, lo cual se cumple en la arquitectura Transformer estándar donde todas las subcapas y embeddings operan con la dimensión d_{model} .

Normalización de Capas (Layer Normalization): Después de la conexión residual, se aplica la normalización de capas. A diferencia de la normalización por lotes (Batch Normalization), la normalización de capas calcula la media y la varianza a lo largo de las características (dimensiones del embedding) para cada muestra de entrenamiento individualmente, lo que la hace independiente del tamaño del lote y adecuada para datos secuenciales de longitud variable (Ba et al., 2016).

Para un vector de entrada $\mathbf{z} = (z_1, z_2, \dots, z_{d_{\text{model}}})$ (que en este caso sería $\mathbf{x}_{\text{residual}}$ de la ecuación 14), la normalización de capas se calcula de la siguiente manera: Primero, se calculan la media (μ) y la varianza (σ^2) sobre todas las d_{model} componentes del vector $\mathbf{z}$ para esa muestra específica:

$$\mu = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} z_j \quad (15)$$

$$\sigma^2 = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} (z_j - \mu)^2 \quad (16)$$

Luego, cada componente z_j del vector se normaliza:

$$\hat{z}_j = \frac{z_j - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad (17)$$

donde ϵ es una constante pequeña (e.g., 10^{-5}) añadida a la varianza para mejorar la estabilidad numérica y evitar la división por cero.

Finalmente, la salida de la normalización de capas se obtiene aplicando una transformación afín con dos parámetros aprendibles por capa, una ganancia (scale) $\gamma \in \mathbb{R}^{d_{\text{model}}}$ y un sesgo (bias) $\beta \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{LayerNorm}(\mathbf{z})_j = \gamma_j \hat{z}_j + \beta_j \quad (18)$$

O en forma vectorial:

$$\text{LayerNorm}(\mathbf{z}) = \gamma \odot \frac{\mathbf{z} - \boldsymbol{\mu}_{\text{vec}}}{\sqrt{\sigma_{\text{vec}}^2 + \epsilon}} + \boldsymbol{\beta} \quad (19)$$

donde $\odot$ denota la multiplicación elemento a elemento, y $\boldsymbol{\mu}_{\text{vec}}$ y σ_{vec}^2 son vectores donde cada componente es la media y varianza calculadas respectivamente (se replican para la operación vectorial). Los parámetros γ y $\boldsymbol{\beta}$ permiten a la capa de normalización aprender a escalar y desplazar las activaciones normalizadas, preservando la capacidad representacional de la red.

En resumen, la salida de un bloque que contiene una subcapa se puede expresar como:

$$\text{SalidaBloque}(\mathbf{x}) = \text{LayerNorm}(\mathbf{x} + \text{SubLayer}(\mathbf{x})) \quad (20)$$

Este proceso se repite para cada subcapa dentro de los bloques del codificador y del decodificador, contribuyendo significativamente a la estabilidad y el rendimiento de los modelos Transformer profundos.

ARQUITECTURA ENCODER-DECODER

La arquitectura Transformer canónica utiliza una pila de seis encoders y seis decoders.

Encoder: Cada encoder consta de dos subcapas principales: una capa de auto-atención multi-cabeza y una red feed-forward posicional. Cada subcapa tiene una conexión residual y normalización de capa. El encoder procesa la secuencia de entrada completa en paralelo, generando representaciones enriquecidas contextualizadas.

Decoder: El decoder también tiene una estructura similar, pero con una subcapa adicional:

1. **Auto-Atención Multi-Cabeza Enmascarada:** Similar a la del encoder, pero con un enmascaramiento para asegurar que, al predecir una palabra, solo se pueda atender a las palabras previas en la secuencia de salida (naturaleza auto-regresiva).

2. **Atención Encoder-Decoder Multi-Cabeza:** Esta capa permite al decoder atender a la salida del encoder. Las consultas (Q) provienen de la capa de auto-atención enmascarada del decoder, mientras que las claves (K) y valores (V) provienen de la salida del último encoder. Es aquí donde la información de la secuencia de entrada se integra para guiar la generación de la secuencia de salida.
3. **Red Feed-Forward Posicional.**

Igualmente, se utilizan conexiones residuales y normalización de capas.

FLUJO DE INFORMACIÓN Y SALIDA

La secuencia de entrada (embeddings + encodings posicionales) pasa por la pila de encoders. La salida del último encoder se alimenta a todas las capas de atención encoder-decoder en cada uno de los decoders. El decoder genera la secuencia de salida palabra por palabra. La salida final del decoder pasa por una capa lineal y una función softmax para producir una distribución de probabilidad sobre el vocabulario para la siguiente palabra.

Entrenamiento del Transformer

El entrenamiento de un Transformer, como cualquier modelo de DL f_{θ} , se basa en minimizar una función de pérdida $L(\theta)$ (e.g., entropía cruzada para clasificación de palabras) utilizando descenso de gradiente. Los parámetros θ (pesos W y sesgos b de todas las capas lineales, incluyendo las de las proyecciones Q, K, V y las FFNN) se actualizan mediante $\theta^{(t+1)} = \theta^{(t)} - \eta \nabla L(\theta^{(t)})$. El gradiente $\nabla L(\theta)$ se calcula eficientemente mediante **retropropagación**, como se describe en el Algoritmo 1.

Algorithm 1 Algoritmo de Retropropagación Generalizado

```

1: Entrada: Lote de datos de entrada  $X = A^0$ , Lote de datos objetivo  $Y$ , Parámetros
   actuales  $\theta = \{W^l, b^l\}_{l=1}^{N_L}$ , Funciones de activación  $\{g^l\}_{l=1}^{N_L}$ , Función de Pérdida  $\mathcal{L}$ .
2: Salida: Gradiente  $\nabla_{\theta} \mathcal{L} = \{\nabla_{W^l} \mathcal{L}, \nabla_{b^l} \mathcal{L}\}_{l=1}^{N_L}$ .
   // — Fase 1: Pase Hacia Adelante (Forward Pass) —
3: for  $l = 1$  to  $N_L$  do
4:    $Z^l \leftarrow f_{\text{op}}^l(A^{l-1}, \text{params}^l)$  ▷ Operación de la capa  $l$ 
5:    $A^l \leftarrow g^l(Z^l)$  ▷ Función de activación (si aplica)
6:   Almacenar  $A^{l-1}$  y  $Z^l$ .
7:  $\hat{Y} \leftarrow A^{N_L}$ 
8:  $L \leftarrow \mathcal{L}(\hat{Y}, Y)$ 
   // — Fase 2: Pase Hacia Atrás (Backward Pass) —
9:  $\delta^{N_L} \leftarrow \frac{\partial \mathcal{L}}{\partial \hat{Y}} \odot (g^{N_L})'(Z^{N_L})$  ▷  $\odot \frac{\partial \mathcal{L}}{\partial Z^{N_L}}$  si no hay activación final explícita
10: for  $l = N_L$  down to 1 do
11:    $\nabla_{\text{params}^l} \mathcal{L} \leftarrow \frac{\partial \mathcal{L}}{\partial \text{params}^l}$  ▷ Usando  $\delta^l$  y  $A^{l-1}$ 
12:   if  $l > 1$  then
13:      $\nabla_{A^{l-1}} \mathcal{L} \leftarrow (\frac{\partial Z^l}{\partial A^{l-1}})^T \delta^l$ 
14:      $\delta^{l-1} \leftarrow \nabla_{A^{l-1}} \mathcal{L} \odot (g^{l-1})'(Z^{l-1})$  ▷  $\odot \nabla_{A^{l-1}} \mathcal{L}$  si  $Z^{l-1}$  es la entrada a una op.
       no lineal
15: Devolver  $\{\nabla_{\text{params}^l} \mathcal{L}\}_{l=1}^{N_L}$ 

```

NOTAS SOBRE EL ALGORITMO DE RETROPROPAGACIÓN: El Algoritmo 1 presenta una visión generalizada. Las expresiones exactas para las derivadas parciales dependen de las operaciones específicas de cada capa del Transformer (atención, FFNN, normalización). δ^l representa $\frac{\partial \mathcal{L}}{\partial Z^l}$, el error retropropagado antes de la activación de la capa l . La eficiencia radica en reutilizar los valores del pase hacia adelante.

CONSIDERACIONES DE ENTRENAMIENTO PARA TRANSFORMERS: El costo computacional de la atención es cuadrático con la longitud de la secuencia ($O(L^2)$), siendo un cuello de botella para secuencias muy largas. Las conexiones residuales y la normalización de capas son esenciales para la estabilidad del gradiente. La paralelización inherente tanto en el pase hacia adelante como en el de retropropagación es una ventaja clave.

3.4.4 Self-Attention-based Imputation for Time Series (SAITS)

(Du et al., 2023b) es un modelo de aprendizaje profundo propuesto específicamente para la imputación de valores faltantes en series temporales multivariantes. Se fun-

damenta en el mecanismo de autoatención de la arquitectura Transformer (Vaswani et al., 2017a), pero introduce un enfoque de optimización conjunta y modificaciones estructurales para abordar eficazmente esta tarea. El modelo está diseñado para aprender a estimar los valores faltantes a partir de una combinación ponderada de dos bloques de autoatención con enmascaramiento diagonal, Diagonally-Masked Self-Attention (DMSA), como se ilustra en la Figura 4.

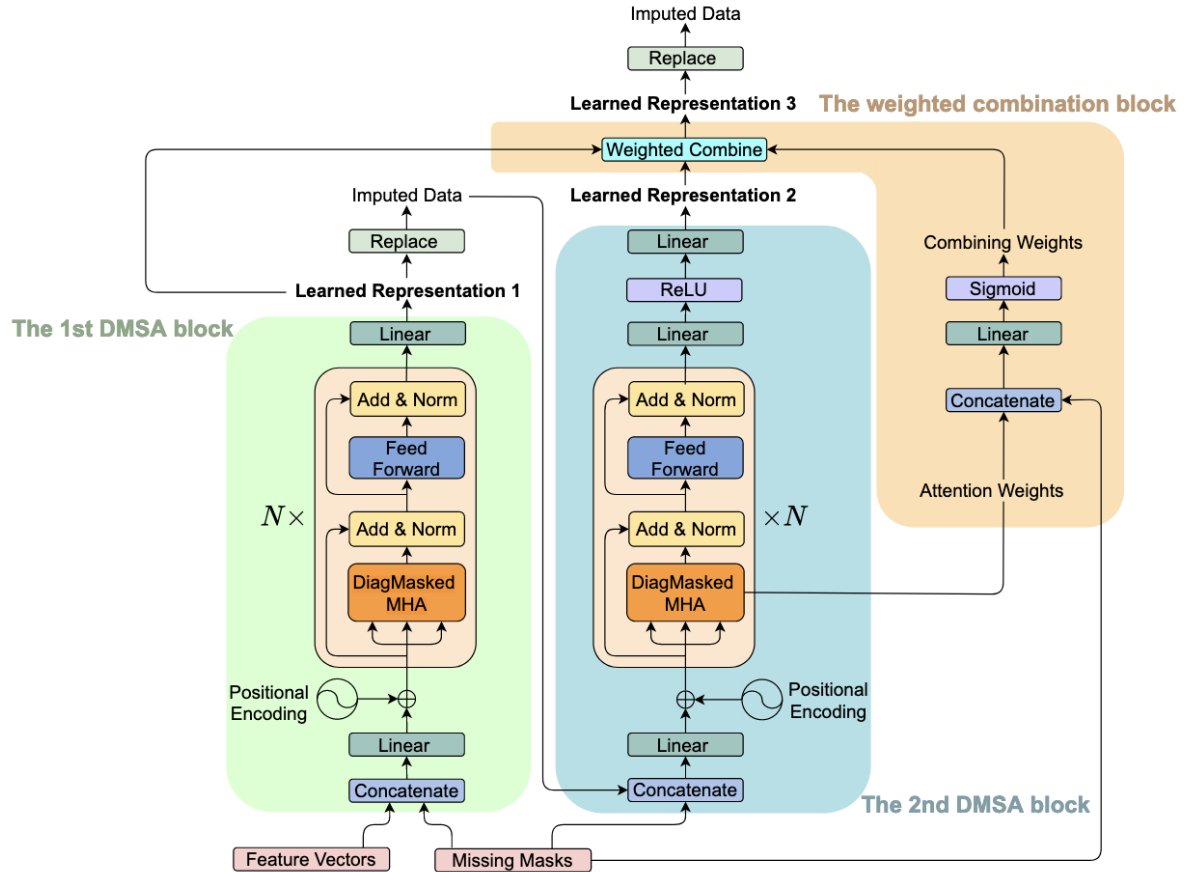


Figura 4: Diagrama conceptual de la arquitectura SAITS, mostrando los dos bloques DMSA y el bloque de combinación ponderada. Adaptado de Du et al. (2023b).

Las diferencias y adaptaciones clave de SAITS respecto a un Transformer estándar, así como sus componentes matemáticos, se detallan a continuación, siguiendo la metodología propuesta en Du et al. (2023b):

Enfoque de Optimización Conjunta (Joint-Optimization Training Approach)

SAITS se entrena mediante un enfoque de optimización conjunta que involucra dos tareas de aprendizaje principales, con el objetivo de forzar al modelo a predecir con precisión los valores faltantes y, al mismo tiempo, preservar la distribución de los datos observados:

1. **Tarea de Imputación Enmascarada (Masked Imputation Task, MIT):** Esta tarea fuerza explícitamente al modelo a predecir valores que fueron artificialmente enmascarados a partir de los datos observados. Dada una serie de entrada $\mathbf{X}$ (ground truth) y su máscara de ausencia original $\mathbf{M}$ (donde $M_{td} = 1$ si X_{td} es observado, 0 si es faltante), se introduce una máscara indicadora $\mathbf{I}$ donde $I_{td} = 1$ si X_{td} fue observado y se enmascaró artificialmente, y 0 en caso contrario. Si $\hat{\mathbf{X}}_c$ es la serie completada (imputada) por el modelo, la pérdida para esta tarea (L_{MIT}) se calcula como el Error Absoluto Medio (MAE) entre los valores originales $\mathbf{X}$ y los imputados $\hat{\mathbf{X}}_c$, únicamente en las posiciones artificialmente enmascaradas:

$$L_{MIT} = \text{MAE}(\hat{\mathbf{X}}_c, \mathbf{X}, \mathbf{I}) = \frac{\|(\hat{\mathbf{X}}_c - \mathbf{X}) \odot \mathbf{I}\|_1}{\|\mathbf{I}\|_1} \quad (21)$$

Esto es análogo al modelado de lenguaje enmascarado (MLM) en NLP.

2. **Tarea de Reconstrucción Observada (Observed Reconstruction Task, ORT):** Esta tarea asegura que el modelo reconstruya correctamente los valores que ya estaban observados en la entrada. La pérdida para esta tarea (L_{ORT}) se calcula como el MAE entre los valores originales observados $\mathbf{X}$ y sus correspondientes reconstrucciones en $\hat{\mathbf{X}}_c$, utilizando la máscara original $\mathbf{M}$:

$$L_{ORT} = \text{MAE}(\hat{\mathbf{X}}_c, \mathbf{X}, \mathbf{M}) \quad (22)$$

En SAITS, la L_{ORT} final es un promedio de las pérdidas de reconstrucción de tres representaciones aprendidas internas ($\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3$, que se describen más adelante):

$$L_{ORT} = \frac{1}{3} (\text{MAE}(\mathbf{X}_1, \mathbf{X}, \mathbf{M}) + \text{MAE}(\mathbf{X}_2, \mathbf{X}, \mathbf{M}) + \text{MAE}(\mathbf{X}_3, \mathbf{X}, \mathbf{M})) \quad (23)$$

La función de pérdida total (L) es una suma ponderada de estas dos pérdidas:

$$L = L_{ORT} + \lambda L_{MIT} \quad (24)$$

donde λ es un hiperparámetro que balancea la importancia de ambas tareas.

Arquitectura SAITS: Bloques DMSA y Combinación Ponderada

La arquitectura de SAITS se compone de dos bloques principales de Autoatención con Enmascaramiento Diagonal (DMSA) y un bloque de combinación ponderada:

- **Autoatención con Enmascaramiento Diagonal:** A diferencia de la autoatención estándar, DMSA aplica una máscara diagonal a la matriz de atención antes de la función softmax. Esto evita que un punto temporal t se atienda a sí mismo

directamente al calcular su propia representación imputada, forzándolo a depender de otros puntos temporales $(T - 1)$. La atención con enmascaramiento diagonal se formula como:

$$\text{DiagMaskedSelfAttention}(Q, K, V) = \text{softmax} \left(\text{DiagMask} \left(\frac{QK^T}{\sqrt{d_k}} \right) \right) V \quad (25)$$

donde $[\text{DiagMask}(\mathbf{x})]_{ij} = -\infty$ si $i = j$ y x_{ij} si $i \neq j$. Esto asegura que los pesos de atención diagonales tiendan a cero después del softmax. Cada bloque DMSA, al igual que un encoder Transformer, contiene capas de DMSA Multi-Cabeza y redes feed-forward posicionales, con conexiones residuales y normalización de capa.

- **Primer Bloque DMSA:** La entrada a este bloque es la concatenación de la serie temporal original $\mathbf{X}$ (con faltantes usualmente rellenos con ceros o un valor específico) y su máscara de ausencia $\mathbf{M}$. Esta entrada combinada se proyecta linealmente y se le suma el encoding posicional $\mathbf{p}$ para formar $\mathbf{e}$:

$$\mathbf{e} = [\text{Concat}(\mathbf{X}, \mathbf{M})\mathbf{W}_e + \mathbf{b}_e] + \mathbf{p} \quad (26)$$

Luego, $\mathbf{e}$ pasa a través de N capas de FFNN($\text{DiagMaskedMHA}(\cdot)$) para producir $\mathbf{z}$. Finalmente, $\mathbf{z}$ se proyecta linealmente para obtener la primera representación aprendida $\mathbf{X}_1$:

$$\mathbf{z} = \{\text{FFNN}(\text{DiagMaskedMHA}(\mathbf{e}))\}^N \quad (27)$$

$$\mathbf{X}_1 = \mathbf{z}\mathbf{W}_z + \mathbf{b}_z \quad (28)$$

La serie completada después de este bloque, $\mathbf{X}'$, se obtiene reemplazando los valores faltantes en $\mathbf{X}$ (según $\mathbf{M}$) con los valores correspondientes de $\mathbf{X}_1$:

$$\mathbf{X}' = \mathbf{M} \odot \mathbf{X} + (1 - \mathbf{M}) \odot \mathbf{X}_1 \quad (29)$$

donde $\odot$ es el producto de Hadamard (elemento a elemento).

- **Segundo Bloque DMSA:** Toma como entrada la serie completada $\mathbf{X}'$ del primer bloque, concatenada nuevamente con la máscara $\mathbf{M}$, y sigue un proceso similar para producir α y luego la segunda representación aprendida $\mathbf{X}_2$:

$$\alpha = [\text{Concat}(\mathbf{X}', \mathbf{M})\mathbf{W}_a + \mathbf{b}_a] + \mathbf{p} \quad (30)$$

$$\beta = \{\text{FFNN}(\text{DiagMaskedMHA}(\alpha))\}^N \quad (31)$$

$$\mathbf{X}_2 = \text{ReLU}(\beta\mathbf{W}_\beta + \mathbf{b}_\beta)\mathbf{W}_y + \mathbf{b}_y \quad (32)$$

Nótese la capa ReLU adicional en la proyección para $\mathbf{X}_2$, lo que hace este bloque ligeramente más profundo que el primero.

- **Bloque de Combinación Ponderada (Weighted Combination Block):** Para obtener la representación final $\mathbf{X}_3$, SAITS combina dinámicamente $\mathbf{X}_1$ y $\mathbf{X}_2$ usando

pesos η derivados del promedio de los mapas de atención $\mathbf{A}$ de la última capa del segundo bloque DMSA y la máscara $\mathbf{M}$:

$$\bar{\mathbf{A}} = \frac{1}{h} \sum_{i=1}^h \mathbf{A}_i \quad (33)$$

$$\eta = \text{Sigmoid}(\text{Concat}(\bar{\mathbf{A}}, \mathbf{M}) \mathbf{W}_\eta + \mathbf{b}_\eta) \quad (34)$$

$$\mathbf{X}_3 = (1 - \eta) \odot \mathbf{X}_1 + \eta \odot \mathbf{X}_2 \quad (35)$$

Finalmente, la serie imputada $\hat{\mathbf{X}}_c$ se obtiene reemplazando los valores faltantes en $\mathbf{X}$ con los de $\mathbf{X}_3$:

$$\hat{\mathbf{X}}_c = \mathbf{M} \odot \mathbf{X} + (1 - \mathbf{M}) \odot \mathbf{X}_3 \quad (36)$$

En conclusión, SAITS especializa la arquitectura Transformer para la imputación mediante el uso de DMSA para evitar que un punto se atienda a sí mismo, un enfoque de optimización conjunta con pérdidas específicas de imputación y reconstrucción, y un mecanismo de combinación ponderada de representaciones aprendidas de dos bloques DMSA. Esto le permite capturar eficazmente dependencias temporales y entre características para una imputación precisa.

Parte III

BLOQUE III: METODOLOGÍA Y CASO DE ESTUDIO: IMPUTACIÓN DE DATOS

IMPUTACIÓN DE DATOS EN SERIES TEMPORALES

Los conjuntos de datos del mundo real, especialmente las series temporales, frecuentemente contienen valores faltantes (*missing data*). Abordar incorrectamente estos datos puede introducir sesgos significativos en los análisis o reducir drásticamente la cantidad de datos utilizables para la modelización (Bishop, 2006, Capítulo 8). La **imputación de datos** es el proceso de estimar y rellenar estos valores ausentes, con el objetivo de restaurar la completitud del conjunto de datos y mitigar los efectos adversos de la información faltante. La elección y aplicación de una técnica de imputación adecuada es, por tanto, un paso clave para la validez de cualquier análisis o modelo posterior construido sobre los datos.

4.1 DEFINICIÓN DEL PROBLEMA DE IMPUTACIÓN EXPERIMENTAL Y EVALUACIÓN

El problema abordado en este trabajo consiste en desarrollar y evaluar funciones de imputación f que, aplicadas al conjunto de datos de prueba con los valores faltantes, denotado como $(\mathcal{X}_{\text{missing}}^{\text{test}}, \mathbf{M}^{\text{test}})$, generen una estimación completada del conjunto de datos, $\hat{\mathcal{X}}^{\text{test}}$:

$$\hat{\mathcal{X}}^{\text{test}} = f(\mathcal{X}_{\text{missing}}^{\text{test}}, \mathbf{M}^{\text{test}})$$

Para los métodos de imputación basados en modelos predictivos, f_{θ} , la función de imputación f depende de un conjunto de parámetros θ^* que son optimizados durante una fase de entrenamiento previa.

El **objetivo principal de la evaluación** es cuantificar la precisión con la que las estimaciones $\hat{\mathcal{X}}^{\text{test}}$ reconstruyen los valores verdaderos originales, $\mathcal{X}_{\text{orig}}^{\text{test}}$. Para ello, se elimina deliberadamente un porcentaje de los valores observados del conjunto de datos, estableciendo así una categorización entre **datos faltantes naturales** (aquellos que están ausentes de forma inherente en el conjunto de datos original) y **datos faltantes artificiales** (aquellos que se eliminan intencionalmente para propósitos de evaluación). La evaluación se centra exclusivamente en aquellas posiciones donde los datos faltantes fueron eliminados artificialmente, según lo indicado por la máscara $\mathbf{M}_{\text{artificial}}^{\text{test}}$.

permitiendo una comparación directa entre los valores verdaderos conocidos y las estimaciones generadas por el método de imputación.

$$\text{Error} = \text{Metric}(\hat{\mathcal{X}}^{\text{test}}, \mathcal{X}_{\text{orig}}^{\text{test}}, \mathbf{M}_{\text{artificial}}^{\text{test}})$$

Se busca identificar la función de imputación f que minimice este error de reconstrucción, medido a través de diversas métricas como el Error Absoluto Medio (MAE), el Error Cuadrático Medio (MSE), la Raíz del Error Cuadrático Medio (RMSE) y el Error Relativo Medio (MRE), en el conjunto de datos de prueba.

4.2 MÉTODOS DE IMPUTACIÓN EN SERIES TEMPORALES

Las estrategias de imputación para series temporales varían ampliamente según la información que utilizan y los supuestos subyacentes sobre la naturaleza de los datos faltantes y la propia serie temporal (Little and Rubin, 2019; Schafer, 1997). En este estudio, adoptamos una taxonomía que clasifica los métodos según el alcance de la información utilizada y la complejidad del modelo subyacente. Sea una serie temporal univariante $X = (x_1, x_2, \dots, x_T)$, donde algunos x_t pueden estar faltantes. Denotaremos el valor imputado para x_t como $\hat{x}_t$.

- **Métodos de Imputación con Información Local (Basados en Propagación o Interpolación):** Estos métodos estiman los valores faltantes x_t basándose predominantemente en los puntos de datos temporalmente adyacentes. Explotan la autocorrelación temporal, partiendo del supuesto de que las observaciones cercanas en el tiempo tienden a presentar valores similares (Molenberghs et al., 2015). Son computacionalmente eficientes pero pueden ser imprecisos si los huecos son grandes o la serie es muy volátil.
 - *Forward Fill (Ffill)* o Last Observation Carried Forward (LOCF): Si x_t es faltante, se imputa con el último valor observado que lo precede. Sea $t_0 < t$ el índice temporal más reciente tal que x_{t_0} está observado. Entonces:

$$\hat{x}_t = \begin{cases} x_{t_0} & \text{si } \{\tau \mid \tau < t \text{ y } x_\tau \neq \text{NaN}\} \text{ no es vacío} \\ 0 & \text{en caso contrario} \end{cases} \quad (37)$$

donde $t_0 = \max\{\tau \in \{1, \dots, T\} \mid \tau < t \text{ y } x_\tau \neq \text{NaN}\}$.

Este método es simple y comúnmente usado, especialmente en datos médicos o financieros donde se asume persistencia del estado (Shao and Zhong, 1992).

- *Backward Fill (Bfill)* o Next Observation Carried Backward (NOCB): Si x_t es faltante, se imputa utilizando el siguiente valor conocido en la secuen-

cia. Sea $t_1 > t$ el índice temporal más cercano tal que x_{t_1} está observado. Entonces:

$$\hat{x}_t = \begin{cases} x_{t_1} & \text{si } \{\tau \mid \tau > t \text{ y } x_\tau \neq \text{NaN}\} \text{ no es vacío} \\ 0 & \text{en caso contrario} \end{cases} \quad (38)$$

donde $t_1 = \min\{\tau \in \{1, \dots, T\} \mid \tau > t \text{ y } x_\tau \neq \text{NaN}\}$.

Utilizado cuando se espera que el futuro inmediato influya en el valor actual (Allison, 2001).

- *Interpolación Lineal*: Si x_t es faltante y se encuentra entre dos valores observados x_{t_a} (anterior, $t_a < t$) y x_{t_p} (posterior, $t_p > t$), con todos los valores entre x_{t_a} y x_{t_p} (excluyendo x_t) también faltantes o siendo estos los más cercanos observados, se asume una trayectoria lineal. El valor imputado $\hat{x}_t$ se calcula como:

$$\hat{x}_t = x_{t_a} + (x_{t_p} - x_{t_a}) \frac{t - t_a}{t_p - t_a} \quad (39)$$

Este método es una mejora sobre LOCF/NOCB si hay fluctuaciones entre los puntos conocidos (Hastie et al., 2009). Para huecos al inicio o final de la serie, se puede recurrir a extrapolación o a LOCF/NOCB.

Estos métodos son generalmente eficientes para rellenar huecos cortos, pero su rendimiento puede decaer ante lagunas de datos más extensas o patrones no lineales.

- **Métodos de Imputación Estacionarios (Basados en Estadísticas Globales o de Ventana)**: Se fundamentan en el supuesto de que ciertas propiedades estadísticas de la serie temporal (o segmentos de ella), como la media o la mediana, son representativas de los valores faltantes. La **estacionariedad** implica que las propiedades estadísticas de la serie (media, varianza, autocorrelación) son invariantes en el tiempo (Box et al., 2015). Estos métodos utilizan estadísticas calculadas sobre los datos observados X_{obs} .

- *Imputación por Media*: Reemplaza los valores faltantes x_t con la media aritmética de todos los valores observados en la serie (o de una ventana temporal relevante W alrededor de t si se aplica localmente):

$$\hat{x}_t = \frac{1}{|X_{missing}^{test}|} \sum_{x_i \in X_{missing}^{test}} x_i \quad (40)$$

Este método es simple pero puede reducir la varianza de la serie e ignorar la estructura temporal y las relaciones con otras variables (Donders et al., 2006).

- *Imputación por Mediana*: Similar al anterior, pero utiliza la mediana de los valores observados, lo que lo hace más robusto a valores atípicos (outliers) (Enders, 2010):

$$\hat{x}_t = \text{mediana}(X_{missing}) \quad (41)$$

Comparte limitaciones similares a la imputación por media en cuanto a la distorsión de la estructura de la serie.

Aunque simples, estos métodos tienden a subestimar la varianza y aplanar la estructura temporal de la serie, y son generalmente inapropiados si la serie no es estacionaria o si los datos faltantes no son completamente al azar.

- **Métodos de Imputación Predictivos (Basados en Modelos):** Emplean modelos estadísticos o de ML para predecir los valores faltantes, aprendiendo patrones y relaciones a partir de los datos observados. Son capaces de capturar dependencias más complejas (Abu-Mostafa et al., 2012b). El objetivo es aprender una función f tal que $\hat{x}_t = f(X_{missing}^{test}, M^{test})$

- *Modelos de Regresión:* En este contexto, la función f representa el modelo de regresión (lineal, no lineal, etc.) ajustado sobre los datos observados. Para imputar un valor faltante $\hat{x}_t$, f toma como entradas los valores observados de otras variables. La función f se traduce en la ecuación del modelo de regresión. Por ejemplo, para una regresión lineal multivariante, donde $\hat{x}_t$ es la variable dependiente y $Z_t = (z_{t,1}, \dots, z_{t,p})$ son las variables predictoras observadas en el instante t (extraídas de $\mathcal{X}_{missing}^{test}$):

$$\hat{x}_t = \beta_0 + \sum_{j=1}^p \beta_j z_{t,j} \quad (42)$$

Los coeficientes β_j son parámetros aprendidos por el modelo de regresión. La máscara M^{test} es fundamental para identificar los x_t que deben ser imputados y para asegurar que solo se utilicen las variables predictoras disponibles y válidas en $\mathcal{X}_{missing}^{test}$.

- *Modelos Autorregresivos (e.g., ARIMA):* En los modelos autorregresivos, la función f captura la dependencia de un valor x_t con sus propios valores pasados. La función f representa el modelo autorregresivo (AR, MA, ARMA, ARIMA) con los parámetros estimados a partir de la serie observada. Para imputar $\hat{x}_t$, f utiliza los valores históricos observados de la misma serie. Por ejemplo, para un modelo autorregresivo AR(p), la función f se materializa como:

$$\hat{x}_t = c + \sum_{i=1}^p \phi_i x_{t-i} + \epsilon_t \quad (43)$$

donde los x_{t-i} son los valores de la serie en instantes anteriores (observados y extraídos de $\mathcal{X}_{missing}^{test}$), los ϕ_i son los coeficientes autorregresivos aprendidos, c es una constante y ϵ_t es un término de error. La máscara M^{test} informa a f qué valores de x están realmente observados y cuáles necesitan imputación, y el modelo manejaría internamente la situación si alguno de los x_{t-i} también fuera faltante (por ejemplo, mediante un proceso iterativo o la imputación previa de esas dependencias).

- *Modelos Basados en Matrices:* Estos modelos transforman la serie temporal en una matriz (e.g., Hankel, Toeplitz) y luego aplican técnicas de completado de matrices para imputar los valores faltantes.

En el caso de la Imputación Hankel (HI) (Poudevigne and Jones, 2024), la serie temporal $x(1), \dots, x(n)$ en $\mathbb{R}^d$ se reestructura en una matriz de Hankel por bloques, H , de tamaño $(n - k + 1) \times (kd)$, donde k es el retardo (lag) fijo y d la dimensión de la serie (para series univariadas, $d = 1$). Esta matriz H es construida de tal manera que cada fila representa un segmento de la serie temporal:

$$H = \begin{pmatrix} x(1) & x(2) & \cdots & x(k) \\ x(2) & x(3) & \cdots & x(k+1) \\ \vdots & \vdots & \ddots & \vdots \\ x(n-k+1) & x(n-k+2) & \cdots & x(n) \end{pmatrix}$$

La tarea de imputación se reformula como un problema de completado de matriz de bajo rango. El objetivo es encontrar una matriz X de bajo rango que coincida con los valores observados de H en las posiciones conocidas. El problema se formula como una minimización de la norma nuclear de la matriz X (sumatoria de sus valores singulares, $\|X\|_*$), sujeta a las restricciones de que los valores observados en las posiciones Ω deben ser preservados. La norma nuclear es una relajación convexa de la función de rango:

$$\min_X \|X\|_* \quad \text{s.t.} \quad X(i, j) = M(i, j) \quad \forall (i, j) \in \Omega \quad (44)$$

Donde M es la matriz Hankel original con valores faltantes, y Ω es el conjunto de índices de los valores observados (es decir, las entradas conocidas de M). La matriz X obtenida al resolver este problema es la matriz Hankel completada.

Los valores imputados $\hat{x}_t$ corresponden a la reconstrucción de la serie temporal a partir de esta matriz X completada. El valor imputado $\hat{x}_t$ se obtiene usualmente promediando las entradas correspondientes en la matriz X completada.

La máscara $\mathbf{M}^{test}$ en este contexto se traduce en el conjunto Ω , que identifica las entradas observadas de la matriz Hankel que deben ser preservadas durante el proceso de completado. Las entradas fuera de Ω son las que el algoritmo de completado de matriz calculará para imputar los valores faltantes en la serie temporal subyacente.

- *Modelos de Aprendizaje Profundo:* Arquitecturas avanzadas como las Redes Neuronales Recurrentes (RNNs, e.g., LSTM (Hochreiter and Schmidhuber, 1997), Gated Recurrent Unit (GRU) (Cho et al., 2014)) o los Transformers (e.g., SAITS (Du et al., 2023a)) son especialmente adecuadas para series temporales. Dada su capacidad para modelar dependencias temporales complejas y de largo alcance, así como interacciones no lineales entre múltiples series, pueden aprender representaciones ricas de los datos observados para inferir los valores faltantes (Goodfellow et al., 2016a, Capítulos 10 y 12).

Por ejemplo, un modelo podría ser entrenado para predecir x_t dado un contexto de valores observados $X_{missing}^{test} = (x_{t-k}, \dots, x_{t-1}, x_{t+1}, \dots, x_{t+k})$. Estos métodos son generalmente más potentes y pueden proporcionar imputaciones más precisas, especialmente en presencia de patrones complejos o cuando el mecanismo de generación de los datos faltantes depende de otras variables observadas o no observadas. (Rubin, 1976; Little and Rubin, 2019).

$$f_{\theta}(X_{missing}^{test}, M_{missing}^{test}) = \text{Transformer}_{\theta}(X_{missing}^{test}, M_{missing}^{test}) \quad (45)$$

Específicamente, pueden ser más efectivos en escenarios donde los datos son **Faltantes Aleatoriamente (MAR, Missing At Random)** –es decir, la probabilidad de que un valor falte depende solo de otras variables observadas– o incluso **Faltantes No Aleatoriamente (MNAR, Missing Not At Random)** –donde la probabilidad de ausencia depende del propio valor que falta o de otras variables no observadas–, siempre y cuando el modelo sea capaz de capturar o se le proporcione información sobre dicho mecanismo de ausencia Sin embargo, estos enfoques predictivos pueden ser computacionalmente más costosos, requerir conjuntos de datos más grandes para un entrenamiento efectivo y son susceptibles al sobreajuste si no se regularizan adecuadamente.

La elección del método de imputación más apropiado depende de múltiples factores, incluyendo las características de los datos (univariante/multivariante, longitud, estacionalidad, tendencia), el mecanismo, patrón y proporción de valores faltantes, y los recursos computacionales disponibles (Rubin, 1976; Van Buuren, 2018).

4.3 MÉTRICAS DE EVALUACIÓN PARA LOS MÉTODOS DE IMPUTACIÓN

Para cuantificar el rendimiento de los métodos de imputación, se emplean métricas de error estándar como el Error Cuadrático Medio (MSE), la Raíz del Error Cuadrático Medio (RMSE), el Error Absoluto Medio (MAE) y el Error Relativo Medio (MRE). Estas métricas, ya introducidas en la Subsección 3.3.1, requieren una adaptación específica para el problema de datos faltantes. La evaluación del error se realiza *exclusivamente* en las posiciones donde los datos originales fueron artificialmente eliminados. Para ello, comparamos los valores imputados $\hat{\mathcal{X}}^{test}$ con los valores verdaderos $\mathcal{X}_{orig}^{test}$ únicamente en las posiciones indicadas por la máscara $\mathbf{M}_{artificial}^{test}$. Estas adaptaciones se formalizan matemáticamente a continuación.

Sean:

- $\hat{\mathcal{X}} = \hat{\mathcal{X}}^{test} \in \mathbb{R}^{N \times T \times D}$: Tensor de valores imputados (conjunto de prueba).
- $\mathbf{X}_{orig} = \mathcal{X}_{orig}^{test} \in \mathbb{R}^{N \times T \times D}$: Tensor de valores verdaderos originales (conjunto de prueba).
- $\mathbf{M}_{ind} = \mathbf{M}_{artificial}^{test} \in \{0, 1\}^{N \times T \times D}$: Máscara binaria que indica las posiciones donde los valores fueron artificialmente eliminados y posteriormente imputados (1 si fue imputada, 0 si no).

Las métricas se definen como:

Definición 42 (Error Absoluto Medio (MAE)). *El Error Absoluto Medio (MAE) se calcula como:*

$$MAE(\hat{\mathcal{X}}, \mathbf{X}_{orig}, \mathbf{M}_{ind}) = \frac{|||\hat{\mathcal{X}} - \mathbf{X}_{orig}|| \odot \mathbf{M}_{ind}||_1}{||\mathbf{M}_{ind}||_1} \quad (46)$$

según (Willmott and Matsuura, 2005) donde $|\cdot|$ denota el valor absoluto elemento a elemento, $\odot$ es el producto de Hadamard (Ver Definición 8), y $||\cdot||_1$ es la norma ℓ_1 (suma de los valores absolutos de los elementos). El término $\odot \mathbf{M}_{ind}$ asegura que solo se consideren los errores en las posiciones imputadas. El denominador normaliza por el número total de puntos imputados.

Definición 43 (Raíz del Error Cuadrático Medio (RMSE)). *La Raíz del Error Cuadrático Medio (RMSE) se define por:*

$$RMSE(\hat{\mathcal{X}}, \mathbf{X}_{orig}, \mathbf{M}_{ind}) = \sqrt{\frac{||(\hat{\mathcal{X}} - \mathbf{X}_{orig})^{\odot 2} \odot \mathbf{M}_{ind}||_1}{||\mathbf{M}_{ind}||_1}} \quad (47)$$

según (Chai and Draxler, 2014a). Aquí, $(\hat{\mathcal{X}} - \mathbf{X}_{orig})^{\odot 2}$ representa el error cuadrático elemento a elemento. Similar al MAE, la máscara $\mathbf{M}_{ind}$ selecciona los errores cuadráticos en las posiciones relevantes, se suman, se promedian y finalmente se toma la raíz cuadrada.

Definición 44 (Error Relativo Medio (MRE)). *El Error Relativo Medio (MRE) se calcula como:*

$$MRE(\hat{\mathcal{X}}, \mathbf{X}_{orig}, \mathbf{M}_{ind}) = \frac{|||\hat{\mathcal{X}} - \mathbf{X}_{orig}|| \odot \mathbf{M}_{ind}||_1}{|||\mathbf{X}_{orig}|| \odot \mathbf{M}_{ind}||_1 + \epsilon} \quad (48)$$

según (Chai and Draxler, 2014b). El numerador corresponde a la suma de los errores absolutos enmascarados. El denominador es la suma de los valores absolutos verdaderos originales en esas mismas posiciones enmascaradas. Se añade una pequeña constante $\epsilon > 0$ al denominador para prevenir la división por cero en caso de que algún valor original en una posición imputada sea cero.

Adicionalmente, se reporta el Error Cuadrático Medio (MSE), que es simplemente el cuadrado del RMSE antes de tomar la raíz cuadrada.

4.4 APRENDIZAJE PROFUNDO PARA LA IMPUTACIÓN EN SERIES TEMPORALES

Las redes neuronales profundas son cada vez más usadas para la imputación en series temporales, superando a métodos tradicionales. Enfoques basados en RNN como BRITS (Cao et al., 2018), GRU-D (Che et al., 2018), o GAN para series temporales (Luo et al., 2018) han mostrado resultados prometedores. Sin embargo, arquitecturas basadas en autoatención, como el Transformer (Vaswani et al., 2017a) y modelos derivados como SAITS (Du et al., 2023b) para imputación, presentan ventajas distintivas:

- **Modelado Eficaz de Dependencias Temporales a Largo Plazo:** Los modelos basados en autoatención como SAITS destacan por su capacidad para modelar dependencias a largo plazo, superando una limitación clave de las RNN (e.g., LSTM (Hochreiter and Schmidhuber, 1997), GRU (Cho et al., 2014)). El procesamiento secuencial de las RNN puede llevar a la pérdida de información o al desvanecimiento de gradientes en secuencias largas, obstaculizando la captura de relaciones distantes (Bengio et al., 1994; Pascanu et al., 2013). En cambio, la autoatención ofrece un acceso directo entre todos los puntos de la secuencia (camino de $O(1)$ vs. $O(L)$ en RNNs), lo que es fundamental para imputar valores que dependen de información lejana (Vaswani et al., 2017a).
- **Captura Integral de Dependencias Conjuntas (Inter e Intra-Variable):** En series multivariantes, los modelos de autoatención pueden modelar holísticamente tanto la evolución individual de cada variable (intra-variable) como las interacciones entre diferentes variables (inter-variable) a lo largo del tiempo. Al permitir que cada variable en cada instante atienda a todas las demás en todos los instantes, se pueden aprender patrones de correlación complejos y dinámicos que modelos más simples podrían omitir.
- **Flexibilidad ante Patrones Complejos sin Supuestos Restrictivos:** Muchos métodos tradicionales requieren supuestos como la **estacionariedad** (propiedades estadísticas constantes en el tiempo (Box et al., 2015)). Los modelos basados en Transformers, como arquitecturas de DL, aprenden patrones no lineales y no estacionarios directamente de los datos, adaptándose a dinámicas temporales diversas sin necesidad de preprocesamientos extensivos o supuestos restrictivos sobre la estructura de los datos.
- **Potencial de Robustez frente a Ruido y Outliers:** El mecanismo de atención, al ponderar la importancia de diferentes segmentos de la secuencia, tiene el potencial de asignar pesos menores a partes ruidosas o con outliers. Esto podría llevar a imputaciones más robustas, aunque la efectividad de esta propiedad depende del aprendizaje adecuado de dichas ponderaciones.

Frente a los modelos mencionados inicialmente y considerando el conjunto de ventajas expuestas, para este trabajo hemos seleccionado SAITS (Du et al., 2023b). La principal razón de esta elección radica en su demostrada capacidad para el **modelado eficaz de dependencias temporales a largo plazo** (como se detalló en el primer punto), una característica esencial para la imputación precisa en series temporales complejas donde la información relevante puede estar significativamente separada en el tiempo. Esta fortaleza, combinada con su habilidad para manejar dependencias multivariantes y su flexibilidad, lo posiciona como un candidato ideal para los desafíos de imputación abordados en este estudio.

IMPUTACIÓN DE DATOS DE TRÁFICO Y CONTAMINACIÓN

En este capítulo, el lector encontrará el desarrollo del caso de estudio práctico: la imputación de datos de tráfico y contaminación del proyecto Smart Poqueira ([Universidad de Granada, 2021](#)). Comenzaremos describiendo el contexto y los datos. Luego, presentaremos un Análisis Exploratorio (EDA) y detallaremos el entorno experimental, incluyendo el software `pampaneira_imputation` y la implementación de los métodos de imputación. Finalmente, se discutirán los resultados de la evaluación.

5.1 INFRAESTRUCTURA DEL SISTEMA IOT

La infraestructura IoT consta, entre otros sensores, de cámaras con reconocimiento de matrículas License Plate Recognition (LPR) estratégicamente ubicadas en los accesos a los municipios de Pampaneira, Bubión y Capileira (Figura 5), y una unidad móvil, camión, en el Cementerio Municipal de Pampaneira (★) equipada con sensores para medir diversos contaminantes atmosféricos y variables meteorológicas.

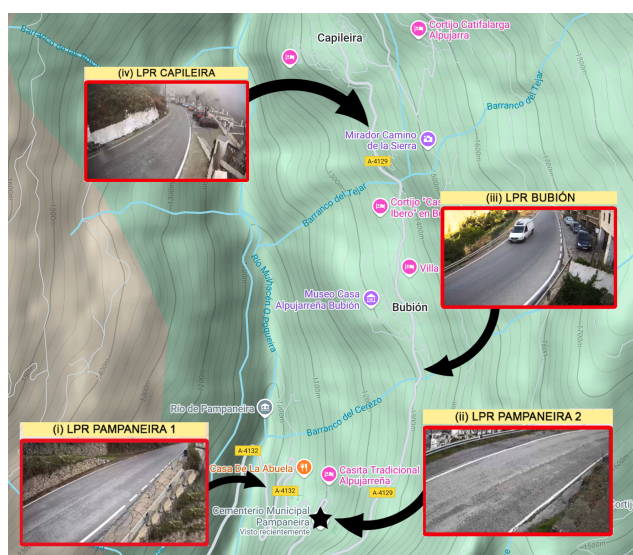


Figura 5: Distribución geográfica de las cámaras LPR en los municipios de Poqueira. Elaborada a partir de ([Bolaños-Martínez et al., 2024](#))

La finalidad última es establecer las relaciones entre los patrones de tráfico vehicular y los niveles de contaminación atmosférica en esta región. Sin embargo, surge un desafío fundamental inherente a la naturaleza móvil de la unidad de medición de contaminantes: el camión de meteorología y contaminación opera en ubicaciones específicas (Pampaneira-Bubión) solo durante periodos de tiempo definidos:

- **Capileira:** 16/11/2022 - 15/01/2023 (Estos datos no se tratan en este análisis específico de imputación por no estar el camión próximo a las ubicaciones de las cámaras.).
- **Pampaneira-Bubión (Cementerio):** 17/01/2023 - 14/03/2023.
- **Pampaneira-Bubión (Cementerio):** 06/06/2023 - 27/06/2023.

La operativa intrínsecamente discontinua de la unidad móvil, agravada por problemas prácticos como cortes de suministro eléctrico, robo de cableado, o el impacto de condiciones meteorológicas adversas (por ejemplo, lluvia de barro cegando las cámaras), además de fallos intermitentes de los sensores, genera **datos faltantes** significativos en las series temporales combinadas de tráfico y contaminación. Consecuentemente, la ausencia de estas mediciones impide un análisis continuo y completo de la relación tráfico-contaminación, lo que subraya la necesidad de abordar estos vacíos mediante técnicas de imputación.

Dada la presencia significativa de datos faltantes, un paso crucial antes de emprender análisis exhaustivos sobre la relación tráfico-contaminación es mejorar la calidad e integridad del dataset. Por ello, este trabajo se centra en la **aplicación y evaluación de diversas técnicas de imputación**. El propósito es reconstruir los valores ausentes de la manera más robusta y precisa posible, aprovechando la información contenida en los datos existentes. Al completar estas series temporales, se busca facilitar análisis posteriores más completos y, fundamentalmente, permitir el desarrollo de modelos predictivos o explicativos más fiables y con mayor poder inferencial.

5.2 CONJUNTOS DE DATOS (DATASETS)

Para llevar a cabo este estudio, se utilizan dos conjuntos de datos principales, derivados de la infraestructura IoT descrita.

El primer dataset consolida los registros agregados por hora del flujo vehicular detectado por las cuatro cámaras LPR del proyecto Smart Poqueira, abarcando un periodo extenso desde febrero de 2022 hasta agosto de 2023. Cada fila del dataset representa una hora específica, y las columnas cuantifican el número de vehículos observados, categorizados según diversos criterios que se detallan a continuación. Este dataset proporciona una visión continua y detallada de la movilidad en la zona de estudio.

Las variables de este dataset se pueden agrupar en las siguientes categorías:

identificación de flujo por cámara y dirección (vehicles_id_camera_in/out):

Estas columnas cuantifican el número de vehículos detectados por una cámara específica y en una dirección determinada. El nombre de la columna sigue el patrón 'vehicles_IDCAM_DIRECCION'.

- IDCAM: Identificador de la cámara LPR. Puede tomar los siguientes valores:
 - PAM_1: Cámara LPR 1 ubicada en Pampaneira.
 - PAM_2: Cámara LPR 2 ubicada en Pampaneira (e.g., cerca del cementerio, cubriendo el acceso a Bubión).
 - BUB: Cámara LPR ubicada en Bubión.
 - CAP: Cámara LPR ubicada en Capileira.
- DIRECCION: Indica el sentido del flujo vehicular. Puede ser:
 - IN: Vehículos entrando al núcleo urbano o zona monitorizada por la cámara.
 - OUT: Vehículos saliendo del núcleo urbano o zona monitorizada por la cámara.

Ejemplo: La columna vehicles_PAM_1_IN contabiliza el número de vehículos que entraron por la cámara PAM_1 durante esa hora.

origen geográfico del vehículo (zona_...): Estas columnas agrupan los vehículos según la provincia o comunidad autónoma de origen de su matrícula a partir de los datos de la Dirección General de Tráfico (DGT).

- Zona_Catalunia_y_Otras: Vehículos procedentes de Cataluña, La Rioja, Aragón, Illes Balears, Euskadi, Navarra, Galicia, Asturias, Cantabria, o Castilla y León.
- Zona_Comunidad_de_Madrid: Vehículos procedentes de la Comunidad de Madrid.
- Zona_Andalucia_no_GR: Vehículos procedentes de las provincias de Andalucía, excluyendo Granada (i.e., Almería, Cádiz, Córdoba, Huelva, Jaén, Málaga, Sevilla).
- Zona_Granada: Vehículos procedentes de la provincia de Granada.
- Zona_Extremadura_y_Otras: Vehículos procedentes de Extremadura, Castilla-La Mancha, Región de Murcia, Comunitat Valenciana, Canarias, Ceuta o Melilla.

características del vehículo: emisiones de co2 (co2_intervalo_...): Estas columnas clasifican los vehículos según el intervalo de emisiones de CO₂ (en g/km) asociado a su modelo, obtenido a partir de la información de la DGT.

- co2_0-100: Vehículos con emisiones de CO₂ menores o iguales a 100 g/km.
- co2_101-200: Vehículos con emisiones de CO₂ entre 101 y 200 g/km (ambos inclusive).
- co2_201-300: Vehículos con emisiones de CO₂ entre 201 y 300 g/km (ambos inclusive).
- co2_+300: Vehículos con emisiones de CO₂ superiores a 300 g/km.

características del vehículo: número de asientos (seats_...): Clasificación de los vehículos según el número de asientos.

- seats_5_seats: Vehículos con exactamente 5 asientos.
- seats_+5_seats: Vehículos con más de 5 asientos.
- seats_-5_seats: Vehículos con menos de 5 asientos.

características el vehículo: etiqueta ambiental dgt (label_...): Categorización de los vehículos según la etiqueta ambiental oficial de la DGT.

- label_Sin_Etiqueta: Vehículos que no disponen de etiqueta ambiental (generalmente los más antiguos y contaminantes).
- label_C: Vehículos con etiqueta ambiental C.
- label_B: Vehículos con etiqueta ambiental B.
- label_ECO: Vehículos con etiqueta ambiental ECO.
- label_0: Vehículos con etiqueta ambiental Cero Emisiones (o).

Nota: Esta información se obtiene mediante técnicas de web scraping de fuentes oficiales de la DGT a partir de la matrícula.

Dataset de Intersección Tráfico-Contaminación

El segundo dataset fusiona temporalmente los datos de tráfico con las mediciones del camión móvil, conteniendo únicamente registros horarios de los períodos y ubicaciones donde el camión estuvo operativo en Pampaneira/Bubión. Es un subconjunto temporal del dataset de tráfico, enriquecido con variables ambientales. Estas variables de contaminación son:

- C0 Monóxido de Carbono:** Concentración media horaria medida con el analizador Thermo Scientific mod. 48i. Es un trazador de combustión incompleta, que principalmente procede de vehículos de gasolina en condiciones de arranque en frío o ralentí, y también de la quema de biomasa. Unidades: miligramos por metro cúbico (mg/m³).
- N0 Monóxido de Nitrógeno:** Concentración media horaria medida con el analizador Thermo Scientific mod. 42i. Es un contaminante primario emitido directamente por procesos de combustión a alta temperatura, siendo un **trazador importante del tráfico diésel**. Unidades: microgramos por metro cúbico (µg/m³).
- N02 Dióxido de Nitrógeno:** Concentración media horaria medida con el analizador Thermo Scientific mod. 42i. Se forma principalmente por la oxidación del NO en la atmósfera, aunque también se emite directamente por los coches aunque en menor medida. Es, junto con el NO, un **trazador clave del tráfico diésel** y otros procesos de combustión. Unidades: microgramos por metro cúbico (µg/m³).

- 03 Ozono Troposférico:** Concentración media horaria medida con el analizador Thermo Scientific mod. 49i. Es un contaminante secundario, no emitido directamente, sino formado en la atmósfera por reacciones fotoquímicas entre óxidos de nitrógeno (NOx) y compuestos orgánicos volátiles (COVs) en presencia de luz solar. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- S02 Dióxido de Azufre:** Concentración media horaria medida con el analizador Thermo Scientific mod. 43i. Proviene principalmente de la combustión de combustibles fósiles que contienen azufre (carbón, fuelóleo). Su relevancia como trazador del tráfico rodado actual es limitada debido a las bajas concentraciones de azufre en gasolinas y gasóleos modernos. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- PM10 Material Particulado <10 μm :** Concentración media horaria de partículas en suspensión con un diámetro aerodinámico inferior a 10 micrómetros, obtenida con el Monitor TEOM 1405 (Thermo Scientific). Incluye una mezcla compleja de partículas de diversas fuentes: combustión (tráfico, industria, calefacción), polvo en suspensión, aerosoles secundarios, fuentes naturales, etc. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- ebc_tot Carbono Negro Total (Hollín):** Concentración media horaria de carbono negro total obtenida con el fotómetro de absorción Aethalometer AE31. El carbono negro es un producto de la combustión incompleta de material carbonoso y es un componente importante del material particulado fino. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- ebc_ff Carbono Negro de Combustibles Fósiles:** Concentración media horaria de carbono negro atribuible específicamente a la combustión de combustibles fósiles (gasolina, diésel). Se obtiene aplicando el modelo del etalómetro (Sandra-dewi et al., 2008). Es considerado un **trazador primario y específico del tráfico vehicular**. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- ebc_bb Carbono Negro de Quema de Biomasa:** Concentración media horaria de carbono negro atribuible específicamente a la quema de biomasa (leña para calefacción, quemas agrícolas, incendios forestales). Se obtiene aplicando el modelo del etalómetro (Sandra-dewi et al., 2008). Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).

Variables Meteorológicas

- temp Temperatura del Aire:** Temperatura media horaria ($^{\circ}\text{C}$) medida por el transmisor YOUNG 92000 ResponseONE. Afecta las tasas de reacción química atmosférica. Temperaturas invernales anómalamente elevadas pueden indicar la presencia de inversiones térmicas asociadas a condiciones anticiclónicas. Estas inversiones limitan la dispersión vertical y favorecen la acumulación de con-

taminantes en superficie, pudiendo generar episodios de alta contaminación (Wallace and Hobbs, 2006; Seinfeld and Pandis, 2016).

- rh Humedad Relativa:** Humedad relativa media horaria del aire medida por el transmisor YOUNG 92000. Afecta a la formación de aerosoles secundarios, la visibilidad y puede influir en la deposición de contaminantes. Alta humedad puede correlacionarse con condiciones de estancamiento o niebla. Unidades: porcentaje (%).
- ws Velocidad del Viento:** Velocidad media horaria del viento medida por el anemómetro de la torre meteorológica. Es un factor fundamental para la dispersión horizontal de los contaminantes; velocidades bajas favorecen la acumulación. Unidades: metros por segundo (m/s).
- wd Dirección del Viento:** Dirección media horaria desde la que sopla el viento, medida en la torre meteorológica. 0° corresponde al Norte geográfico. Es crucial para identificar el transporte de contaminantes desde posibles áreas fuente. Unidades: grados (°).
- pres Presión Atmosférica:** Presión atmosférica media horaria a nivel de la estación de medición. Altas presiones suelen asociarse con condiciones meteorológicas estables (anticiclones) que, combinadas con vientos bajos, dificultan la dispersión vertical y horizontal de los contaminantes, favoreciendo episodios de alta contaminación. Unidades: hectopascales (hPa).
- truck_pos Posición del Camión Meteorológico:** Variable categórica que indica la cámara LPR más cercana a la ubicación física del camión de medición durante la hora correspondiente. Los valores posibles en este dataset son 'CAP' (cuando estaba en Capileira) y 'PAM_2' (cuando estaba en la ubicación del cementerio de Pampaneira, monitorizado por la cámara PAM_2). Esta variable permite asociar las mediciones de contaminación con el flujo de tráfico específico de la cámara más próxima.

En la Figura 6 se puede observar los registros para cada localización distinta del camión.

Este dataset hereda también las columnas de tráfico vehicular correspondientes a la hora y cámara más cercana. Es el dataset principal para el análisis y la imputación.

5.3 ANÁLISIS EXPLORATORIO DE DATOS (EDA)

Se efectuó un Análisis Exploratorio de Datos (EDA) sobre el dataset de trafico-intersección para comprender las distribuciones, identificar patrones, valores atípicos, datos faltantes y correlaciones.

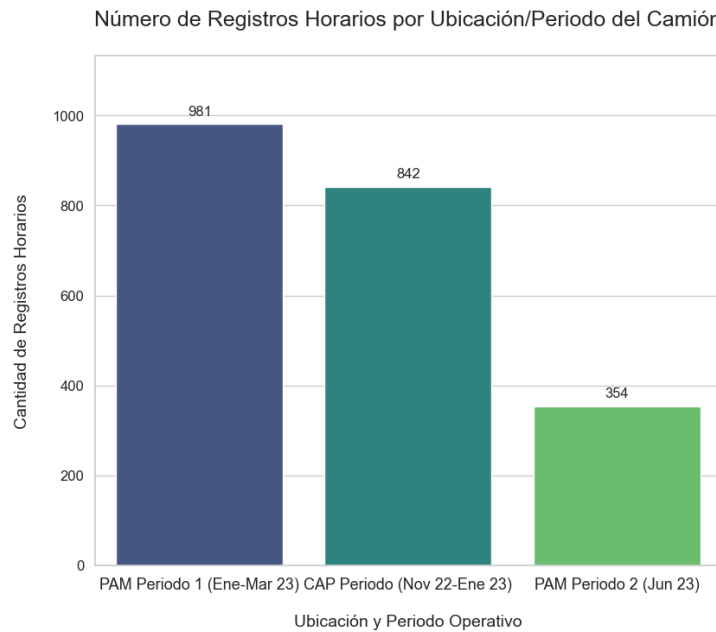


Figura 6: Distribución del número de registros horarios en el dataset de intersección (trafico_contamina_interseccion) según la ubicación y periodo operativo del camión móvil de medición

Se realizó un análisis de valores faltantes, distribución de variables y análisis de correlaciones.

5.3.1 Análisis del Porcentaje de Valores Faltantes

Análisis de la Figura 7: En este periodo, las variables con la mayor proporción de valores faltantes son **WS** (Velocidad del Viento) y **WD** (Dirección del Viento), ambas con un **60.6 %** de datos ausentes. Les siguen notablemente **eBC_ff** y **eBC_bb**, cada una con un **43.7 %** de valores faltantes. **PM10** presenta un **30.9 %** de ausencia, mientras que **O3**, **NO2**, **NO** y **CO** muestran porcentajes de valores faltantes aproximados al **27-28 %**. El resto de las variables de tráfico (como **vehicles_PAM_1_OUT**, **vehicles_PAM_1_OUT_Zona_Granada**, etc.) también se sitúan alrededor del **27.3 %** de datos faltantes.

Análisis de la Figura 8: En contraste con el Periodo 1, la variable **PM10** se destaca como la de mayor incidencia de datos faltantes en este periodo, alcanzando un notable **80.2 %**. Las variables **WS** y **WD** ahora presentan un **30.8 %** de ausencia, seguidas de cerca por **eBC_ff** y **eBC_bb** con **29.2 %**. Las variables de contaminación **CO**, **NO2**, **NO** y **O3** muestran porcentajes muy similares, alrededor del **28.3 %**. Las variables de tráfico también mantienen un nivel constante de datos faltantes, aproximadamente el **28.1 %**.

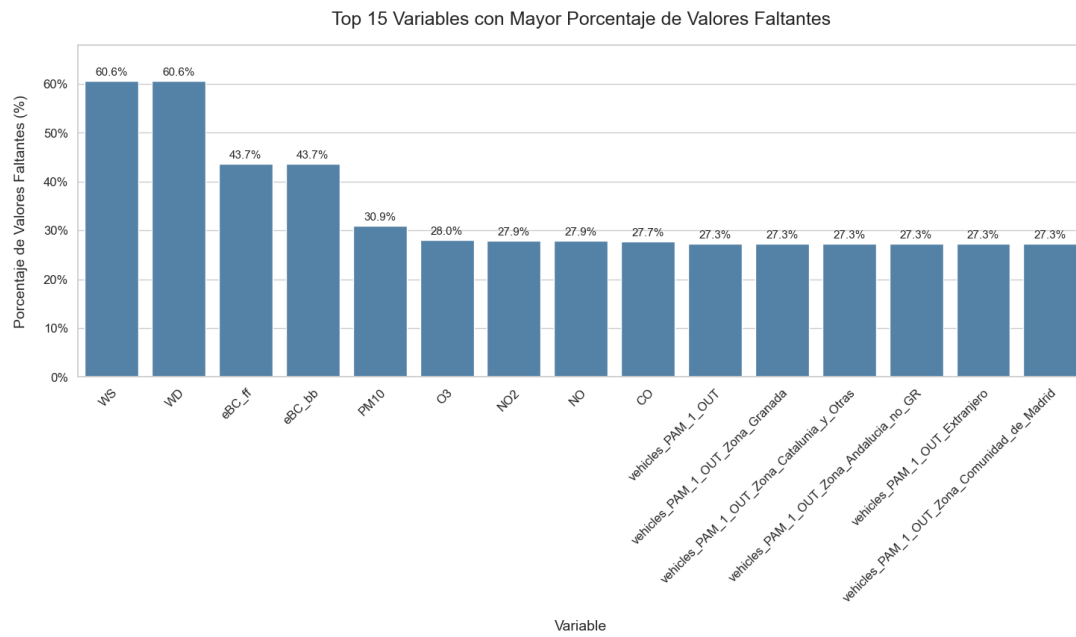


Figura 7: Porcentaje de valores faltantes para las 15 variables con mayor proporción de ausencias en el Periodo 1.

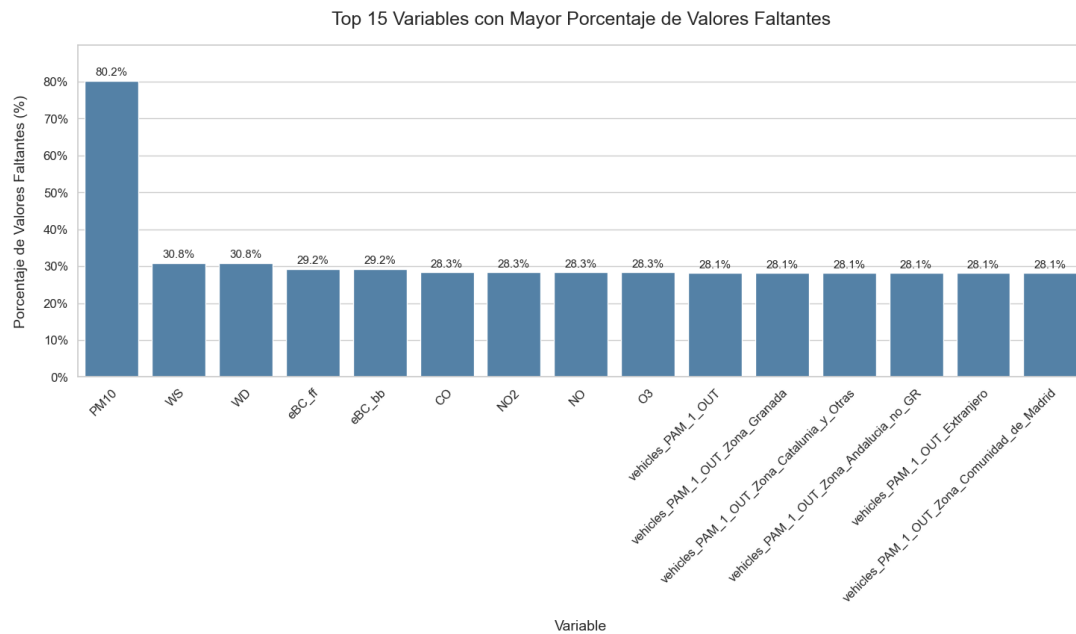


Figura 8: Porcentaje de valores faltantes para las 15 variables con mayor proporción de ausencias en el Periodo 2

Los histogramas de los **contaminantes (CO, NO, NO₂)**: (Figura 9) mostraron distribuciones con fuerte asimetría derecha, indicando predominio de concentraciones bajas con picos ocasionales altos, característico de fuentes como el tráfico.

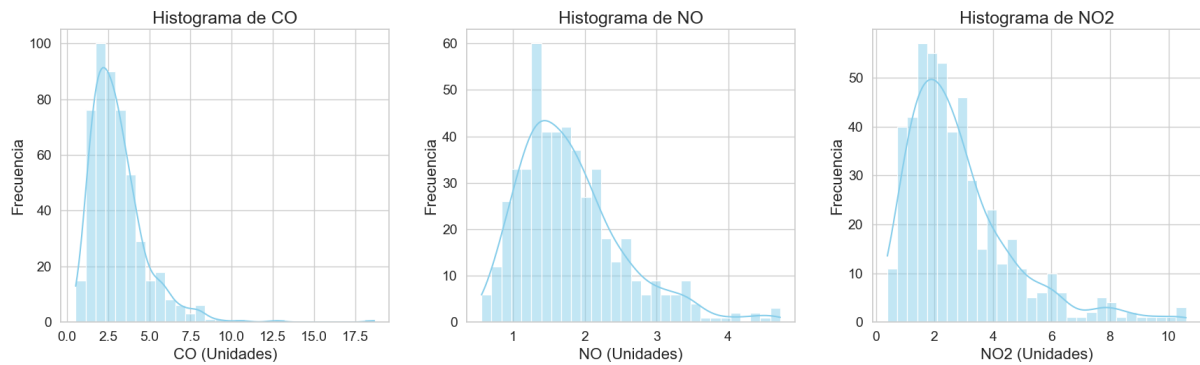


Figura 9: Histogramas de las concentraciones horarias de CO, NO y NO₂. Se observa una clara asimetría hacia la derecha en las tres distribuciones.

Los diagramas de caja de los **contaminantes (O₃, SO₂, PM₁₀, eBC)**: (Figura 10) revelaron valores atípicos, especialmente en O₃ y PM₁₀. La mediana de SO₂ fue muy baja. Las fracciones de eBC mostraron diferencias en sus rangos y medianas.

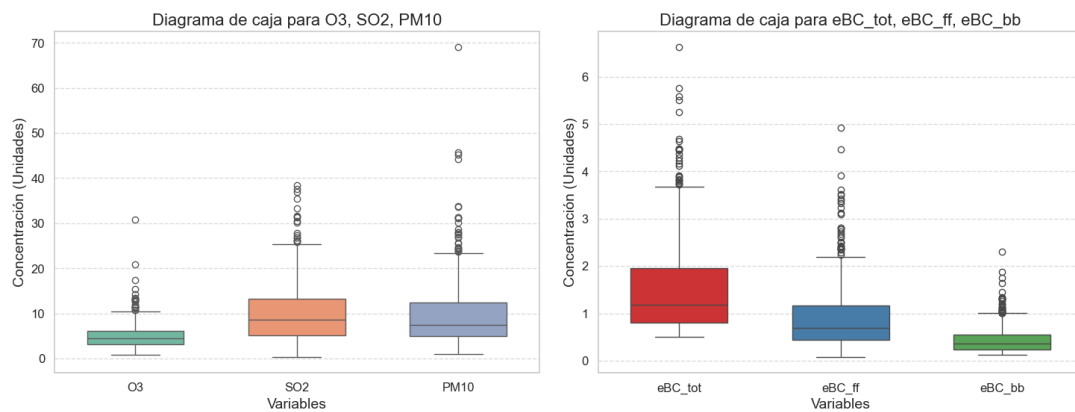


Figura 10: Diagramas de caja comparativos para variables de contaminación. A la izquierda se muestran O₃, SO₂ y PM₁₀. A la derecha se muestran las variables relacionadas con el carbono negro elemental (eBC_tot, eBC_ff, eBC_bb).

Los histogramas de **meteorología (TEMP, RH, WS)**: (Figura 11) mostraron distribuciones más variadas: TEMP bimodal (ciclos día/noche), RH más centrada, y WS con fuerte asimetría derecha (predominio de vientos bajos).

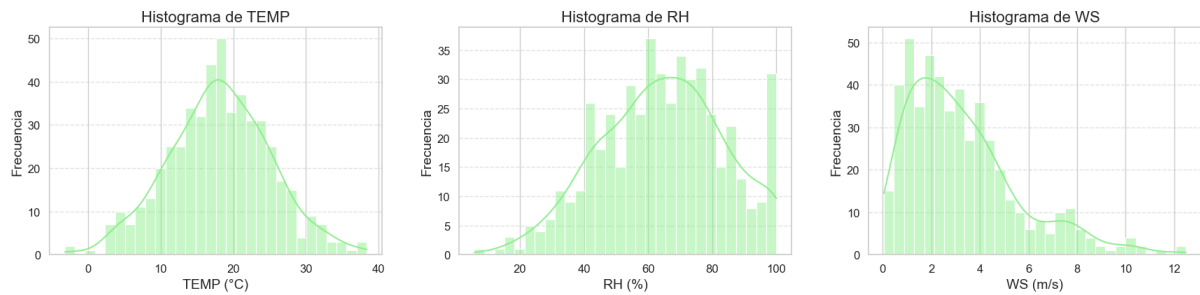


Figura 11: Histogramas de las variables meteorológicas TEMP (Temperatura), RH (Humedad Relativa) y WS (Velocidad del Viento).

La matriz de correlación (Figura 12) mostró relaciones significativas:

- **Positivas Fuertes:** eBC_tot con eBC_ff (0.76) y eBC_bb (0.64); NO2 con eBC_tot (0.69); NO con NO2 (0.64); O3 con TEMP (0.4).
- **Negativas Moderadas:** RH con O3 (-0.5); RH con TEMP (-0.5).
- **Otras:** Correlaciones negativas débiles entre contaminantes primarios (NOx, eBC) y O3. Correlaciones débiles entre meteorología y contaminantes primarios.

Estas correlaciones, especialmente las ligadas a trazadores de tráfico, son clave para la imputación basada en modelos predictivos.

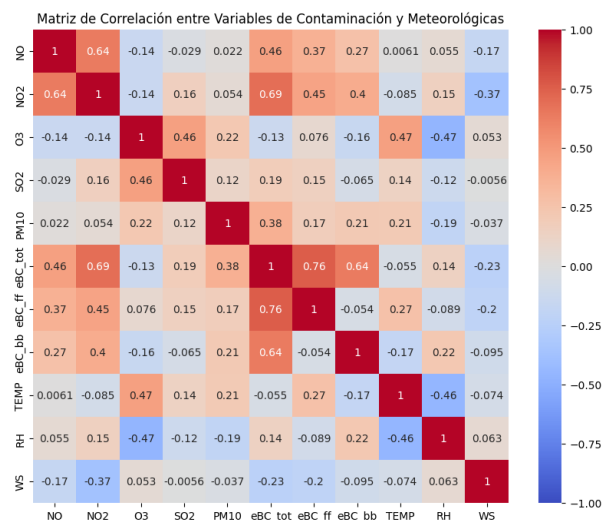


Figura 12: Matriz de correlación de Pearson entre las principales variables de contaminación

5.4 ENTORNO EXPERIMENTAL Y FLUJO DE TRABAJO

Para llevar a cabo la parte práctica de este trabajo, centrada en la imputación de datos faltantes en series temporales de contaminación atmosférica, se ha desarrollado una solución específica mediante la creación de un paquete de Python denominado `pampaneira_imputation`¹. Este paquete organiza de forma modular toda la lógica necesaria para el preprocesamiento de datos, la aplicación de los diversos métodos de imputación seleccionados y la evaluación sistemática de sus resultados.

5.4.1 Estructura del Paquete `pampaneira_imputation`

El paquete `pampaneira_imputation` ha sido estructurado en varios módulos, cada uno con responsabilidades bien definidas para facilitar el mantenimiento, la escalabilidad y la claridad del código. La estructura principal del paquete es la siguiente:

- **`__init__.py`**: Archivo estándar que permite que el directorio sea tratado como un paquete de Python.
- **`config.py`**: Centraliza todos los parámetros de configuración globales del proyecto. Esto incluye rutas de archivos de datos, fechas de inicio y fin de los periodos de estudio, hiperparámetros de los modelos de aprendizaje profundo, nombres de las columnas de características, entre otros. Esta centralización facilita la modificación de parámetros para diferentes ejecuciones y experimentos.
- **`data_loader.py`**: Contiene las funciones responsables de cargar los conjuntos de datos originales desde sus fuentes (e.g., archivos CSV). Este módulo también puede realizar una limpieza inicial o transformaciones básicas que sean comunes a todos los datos antes del preprocesamiento específico para la imputación.
- **`data_preprocessor.py`**: Implementa toda la lógica para el preprocesamiento de los datos. Sus funciones se encargan de la división de los datos en conjuntos de entrenamiento, validación y prueba según los periodos definidos; el escalado de las características numéricas (e.g., mediante `StandardScaler`); la creación de ventanas temporales o secuencias para los modelos que las requieran; y, crucialmente, la eliminación artificial de datos en los conjuntos de validación y prueba para permitir una evaluación controlada de los métodos de imputación.
- **`evaluation.py`**: Define las funciones para calcular las métricas de rendimiento (RMSE, MSE, MAE, MRE) que se utilizan para evaluar la precisión y la calidad de las imputaciones realizadas por los diferentes métodos.
- **`imputation_methods.py`**: Este módulo es el núcleo del proyecto donde se agrupan las implementaciones de los diferentes algoritmos de imputación evaluados. Incluye tanto métodos simples (media, mediana, interpolación lineal, forward fill, backward fill) como los modelos más avanzados basados en aprendizaje profundo (Transformer, SAITS). Para estos últimos, parte de la funcionalidad se ha

¹ https://github.com/eigenric/TFG/tree/main/pampaneira_imputation

construido aprovechando las capacidades de la librería **PyPOTS** (Du, 2023), una biblioteca especializada en el procesamiento de datos con patrones irregulares y datos faltantes en series temporales.

- **utils.py**: Recopila un conjunto de funciones auxiliares de propósito general que son utilizadas a lo largo de todo el paquete. Ejemplos de estas utilidades pueden ser funciones para la creación de ventanas deslizantes a partir de secuencias, la eliminación artificial de datos con patrones específicos, o el reformato de estructuras de datos.

5.4.2 Flujo de Trabajo y Experimentación

El flujo de trabajo principal (pipeline) para la ejecución de los experimentos y la obtención de los resultados comparativos entre los métodos de imputación se ha orquestado y documentado mediante un Jupyter Notebook principal, denominado `Notebook_Imputation.ipynb`. Este notebook guía al usuario o investigador a través de los siguientes pasos metodológicos:

1. **Configuración Inicial:** Carga de los parámetros del proyecto definidos en el módulo `config.py`.
2. **Carga de Datos:** Adquisición de los datos crudos de contaminación y meteorológicos utilizando las funciones provistas por `data_loader.py`.
3. **Preprocesamiento:** Aplicación de las rutinas de preprocesamiento de `data_preprocessor.py`, que incluyen la segmentación temporal, el escalado de características, la generación de secuencias apropiadas para los modelos y la eliminación controlada de datos en los subconjuntos designados para la evaluación.
4. **Aplicación de Métodos de Imputación:** Ejecución secuencial de los diferentes métodos de imputación definidos en `imputation_methods.py` sobre los datos preprocesados con datos eliminados artificialmente. Para los modelos de aprendizaje profundo, esto incluye la carga de modelos pre-entrenados o su entrenamiento si es la primera ejecución o si se han modificado los parámetros.
5. **Evaluación Cuantitativa:** Cálculo del rendimiento de cada método de imputación utilizando el conjunto de métricas implementadas en `evaluation.py`. Esta evaluación se realiza comparando los valores imputados con los valores originales verdaderos únicamente en las posiciones donde se eliminaron datos artificialmente.
6. **Análisis y Visualización de Resultados:** Presentación tabular de las métricas de error y generación de visualizaciones (como las series temporales comparativas) para un análisis cualitativo del comportamiento de los métodos.

Este enfoque estructurado mediante un notebook facilita la reproducibilidad de los experimentos y proporciona una documentación clara del proceso seguido para la obtención de los resultados.

5.4.3 Control de Versiones y Pruebas Unitarias

El desarrollo íntegro del código fuente del paquete `pampaneira_imputation`, así como del **Jupyter Notebook**² de experimentación, se ha gestionado mediante el sistema de control de versiones Git. El repositorio que contiene todo el código fuente, los datos (o enlaces a ellos) y los materiales complementarios del proyecto se encuentra disponible públicamente en la plataforma GitHub.

Adicionalmente, con el fin de asegurar la correcta funcionalidad y la fiabilidad de los componentes clave del paquete de software desarrollado, se ha implementado una serie de **pruebas unitarias**. Estos tests automatizados verifican el comportamiento esperado de funciones críticas, particularmente aquellas involucradas en el preprocesamiento de los datos (como la creación de ventanas o la introducción de datos faltantes) y en el cálculo de las métricas de evaluación. La inclusión de pruebas unitarias contribuye significativamente a la robustez del código, facilita la detección temprana de errores durante el desarrollo y permite realizar refactorizaciones con mayor confianza.

5.5 PREPARACIÓN DE DATOS Y CONFIGURACIÓN EXPERIMENTAL

El proceso se inicia con los datos crudos de series temporales multivariantes de Pampaneira-Bubión. Recordemos que el camión y la cámara PAM_2 se encuentran ubicados en el cementerio municipal de Pampaneira. Tratamos, por proximidad, los datos de Pampaneira y Bubión y descartamos Capileira.

5.5.1 Carga, Estructura y Preprocesamiento Inicial

Los datos originales, que pueden contener faltantes naturales, son cargados utilizando rutinas del módulo `pampaneira_imputation.data_loader`. A continuación, para garantizar una estructura de serie temporal multivariante completa y regular, una función del módulo `pampaneira_imputation.data_preprocessor` completa el índice temporal horario para los periodos de estudio. Si originalmente faltan registros para ciertas horas, se insertan filas nuevas para esos instantes, rellenando todas las características con valores NaN, asegurando así la continuidad requerida para los análisis posteriores. Los datos se organizan en una estructura tensorial 3D ($N \times T \times D$), donde N es el número de muestras (ventanas temporales), $T = 24$ es la longitud fija de la ventana, y D el número de características. El valor $T = 24horas$ se ha elegido para poder seguir los patrones diarios de vehículos y ambientales. Los datos se dividen por periodos temporales para análisis separados.

² https://github.com/eigenric/TFG/blob/main/notebooks/Notebook_Imputation.ipynb

5.5.2 División, Escalado y Creación del Escenario de Evaluación

El núcleo del preprocesamiento, gestionado por el módulo `pampaneira_imputation.data_preprocessor`, aplica los siguientes pasos a los datos de cada periodo:

1. **División Train/Validation/Test:** Segmentación cronológica de los datos en conjuntos de entrenamiento (80 %), validación (20 % del de entrenamiento) y prueba (20 % final), según las duraciones especificadas.
2. **Escalado (Normalización Z-score):** Aplicación de escalado estándar a las características seleccionadas, ajustando el escalador (*fitting*) solo sobre el conjunto de entrenamiento y aplicándolo (*transforming*) a los tres conjuntos. Para una característica j en el dataset, su valor estandarizado x'_{ij} para la i -ésima muestra se calcula como:

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \quad (49)$$

donde x_{ij} es el valor original de la característica j para la muestra i , μ_j es la media de la característica j , y σ_j es la desviación estándar de la característica j . Es crucial que los parámetros del escalador (la media μ_j y la desviación estándar σ_j para cada característica) se calculen (*fitting*) **únicamente** sobre el conjunto de entrenamiento. Posteriormente, esta transformación (con los parámetros aprendidos del entrenamiento) se aplica (*transforming*) a los tres conjuntos: entrenamiento, validación y prueba. Este procedimiento evita la fuga de información (data leakage) del conjunto de prueba al de entrenamiento, asegurando una evaluación más realista del rendimiento del modelo.

3. **Ventaneo (Windowing):** Creación de secuencias de longitud $T = 24$ mediante una ventana deslizante, resultando en la estructura tensorial 3D ($N \times T \times D$).
4. **Eliminación de datos para crearlos artificialmente:** Se eliminan artificialmente un 10 % de los valores previamente observados (después de escalar y ventanear) en cada split (train, val, test), siguiendo un patrón configurado. En este estudio, se consideran tres patrones para simular diferentes tipos de ausencia de datos:
 - **'point' (Puntual):** Introduce valores NaN de forma aleatoria e independiente en cualquier posición del tensor de datos, simulando fallos esporádicos. Este es el patrón que utilizamos por defecto.
 - **'subseq' (Ráfaga):** Genera huecos o ráfagas de valores NaN de longitud variable en posiciones aleatorias dentro de cada serie temporal individual (por característica y muestra), simulando interrupciones temporales de sensores.
 - **'block' (Bloque desde el inicio):** Crea bloques de NaN de longitud variable que comienzan siempre desde el inicio de la secuencia temporal (por característica y muestra), simulando, por ejemplo, datos ausentes al comienzo de un periodo de medición.

Este paso es crucial para poder comprobar la fiabilidad de los modelos de reconstrucción de la señal.

Esta fase de preprocesamiento define formalmente los siguientes conjuntos de datos y máscaras utilizados en el experimento:

- $\mathcal{X}_{\text{orig}}$: El tensor de datos escalados y ventaneados (windowing) *antes* de eliminar los faltantes artificiales. Este tensor carece de los faltantes naturales originales, pero no de los artificiales y sirve como verdad absoluta o **ground truth** para la evaluación.
- $\mathbf{M}_{\text{natural}}$: La máscara binaria que indica la posición de los faltantes naturales en $\mathcal{X}_{\text{orig}}$ (0 =faltante, 1 =observado).
- $\mathbf{M}_{\text{artificial}}$: La máscara binaria que indica las posiciones donde se eliminaron faltantes *artificialmente* (1 =eliminado artificialmente, 0 =no eliminado).
- $\mathcal{X}_{\text{missing}}$: El tensor de datos utilizado como **entrada** para los métodos de imputación. Le faltan tanto los naturales como los artificiales.

La imputación se realiza sobre $\mathcal{X}_{\text{missing}}$, pero la evaluación de la precisión se centra exclusivamente en la capacidad de los modelos para recuperar estos valores faltantes eliminados artificialmente $\mathbf{M}_{\text{artificial}}$, ya que para ellos se conoce el valor verdadero original ('ground truth').

5.6 IMPLEMENTACIÓN DE LAS FUNCIONES DE IMPUTACIÓN

Para este estudio, se han implementado y evaluado varios métodos de imputación, gestionados a través del módulo `pampaneira_imputation.imputation_methods`. A continuación, se describen conceptualmente, aplicados sobre el dataset con valores faltantes $\mathcal{X}_{\text{missing}}^{\text{test}}$.

5.6.1 Métodos Estadísticos Simples

Estos métodos operan sobre cada muestra de 24 horas (denotada como $\mathbf{X}^{(k)}$) de forma independiente.

Imputación por Media/Mediana Sample-Wise

Para cada característica dentro de una muestra $\mathbf{X}^{(k)}$, se calcula su media o mediana utilizando únicamente los valores observados en esa misma muestra. Los valores NaN de dicha característica en esa muestra se rellenan con el estadístico calculado. Se ha implementado una estrategia de respaldo (utilizar el estadístico global del conjunto de entrenamiento o un valor por defecto como 0) para los casos en que una carac-

terística esté completamente ausente en una muestra particular. Estos enfoques se clasifican dentro de los *métodos de imputación estacionarios* descritos en la [Sección 4.2](#).

5.6.2 *Métodos Basados en Propagación e Interpolación (Locales)*

Estos métodos también procesan cada muestra $\mathbf{X}^{(k)}$ de forma independiente, utilizando técnicas estándar para el tratamiento de series temporales, implementadas con el apoyo de bibliotecas como pandas. Se consideran *métodos de imputación con información local*, según la taxonomía de la [Sección 4.2](#).

Imputación Hacia Adelante (Forward Fill / LOCF)

Consiste en propagar la última observación válida hacia adelante en el tiempo (a lo largo del eje temporal, $\text{axis}=0$, dentro de cada muestra). Los valores NaN que puedan persistir al inicio de la secuencia o que no tengan un valor previo para propagar se rellenan con 0.

Imputación Hacia Atrás (Backward Fill / NOCB)

De forma análoga al forward fill, este método propaga la siguiente observación válida hacia atrás en el tiempo ($\text{axis}=0$). Los valores NaN al final de la secuencia o sin un valor posterior se rellenan con 0.

Imputación por Interpolación Lineal

Se estiman los valores faltantes mediante una interpolación lineal entre los puntos observados vecinos más cercanos en el tiempo ($\text{axis}=0$). Se contempla la extrapolación en los extremos de la secuencia si es viable. Los NaN restantes que no puedan ser interpolados se rellenan con 0.

5.6.3 *Métodos basados en Matrices*

Imputación Hankel

1. Representación Matricial: La serie temporal se transforma en una **matriz de Hankel**. Esta matriz se construye con filas que son ventanas sucesivas de la serie. La clave es que, debido a las dependencias temporales, se asume que esta matriz tiene un **rango bajo**.

2. Problema de Optimización: El objetivo es completar los valores faltantes en la matriz de Hankel. Esto se logra resolviendo un problema de optimización que minimiza la **norma nuclear** de la matriz (una relajación convexa de la minimización del rango).
3. Algoritmo SVT: La solución se encuentra iterativamente, comúnmente usando el algoritmo de **Umbralización de Valores Singulares (SVT)**. El SVT aplica un umbral suave a los valores singulares de la matriz para forzar el bajo rango, mientras se mantienen fijos los valores observados.
4. Reconstrucción: Una vez completada la matriz de Hankel, la serie temporal original se reconstruye promediando los valores de las diagonales de la matriz.

5.6.4 Métodos Basados en Modelos Predictivos

Estos métodos se encuadran en la categoría de *métodos de imputación predictivos* utilizando aprendizaje profundo, como se introdujo en la [Sección 4.2](#).

Imputación con Transformer y SAITS

Se han empleado las arquitecturas Transformer y SAITS, disponibles en la biblioteca PyPOTS ([Du, 2023](#)). La documentación detallada de estas implementaciones se puede consultar en la referencia oficial de la biblioteca³. El proceso de aplicación de estos modelos se divide en dos fases principales:

1. *Entrenamiento*: Una función específica dentro del módulo `imputation_methods` se encarga de instanciar el modelo PyPOTS seleccionado (SAITS o Transformer) con un conjunto de hiperparámetros predefinidos. Posteriormente, el modelo se entrena utilizando los conjuntos de datos de entrenamiento y validación, $(\mathcal{X}_{\text{missing}}^{\text{train/val}}, \mathbf{M}^{\text{train/val}})$. Durante el entrenamiento se utiliza un optimizador estándar como Adam, un planificador de la tasa de aprendizaje (e.g., ReduceLROnPlateau) y una función de pérdida adecuada para tareas de regresión (e.g., SmoothL1Loss). Esta fase tiene como resultado un modelo con parámetros optimizados, θ^* .
2. *Imputación (Inferencia)*: Una vez entrenado, otra función del módulo aplica el modelo f_{θ^*} al dataset de prueba $(\mathcal{X}_{\text{missing}}^{\text{test}}, \mathbf{M}^{\text{test}})$ para generar las estimaciones completadas $\hat{\mathcal{X}}^{\text{test}}$.

³ <https://docs.pypots.com/en/latest/pypots.imputation.html>

5.7 VISUALIZACIÓN DE SERIES TEMPORALES IMPUTADAS

Una vez preparados los conjuntos de datos con valores faltantes tanto naturales como artificiales (denominados genéricamente como $\mathcal{X}_{\text{missing}}$), se procedió a aplicar los diversos métodos de imputación descritos en la Sección 5.6. El objetivo de cada método fue estimar los valores en todas las posiciones donde existían datos ausentes. Para evaluar la efectividad de estas estimaciones, particularmente la del modelo SAITS que es de especial interés en este trabajo, ahora se comparan los valores imputados con la verdad absoluta ('ground truth', $\mathcal{X}_{\text{orig}}$), centrándose en aquellas posiciones donde los datos faltantes fueron introducidos artificialmente. Esta comparación nos permite cuantificar la precisión de la recuperación de los datos.

A continuación, se presentan las series temporales para algunas de las variables de contaminación clave, mostrando los datos originales (ground truth), los puntos donde se introdujo la ausencia de datos artificialmente, y los valores imputados por el método SAITS.

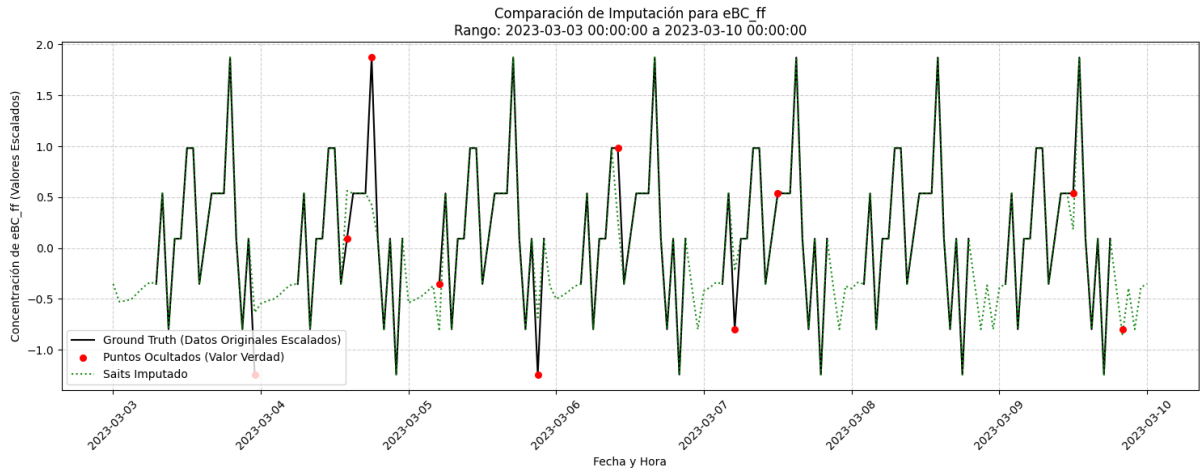


Figura 13: Comparación de imputación para la variable eBC_ff.

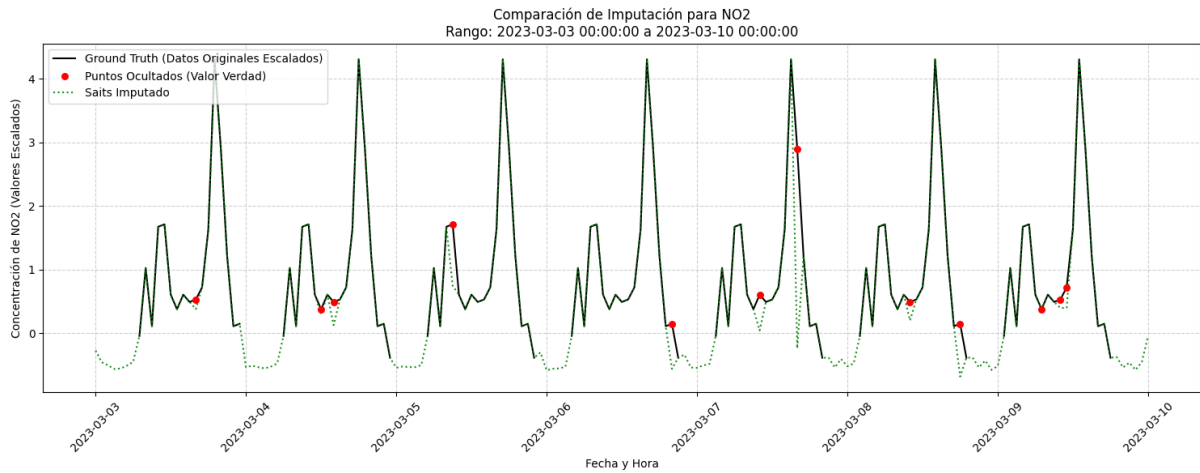


Figura 14: Comparación de imputación para la variable NO₂.

Eliminar faltantes en las mismas posiciones temporales para todas las variables simplificaría en cierto modo el problema de evaluación, ya que el 'patrón de ausencia' sería el mismo para todas. Al hacerlo de forma independiente, creamos un escenario más complejo y diverso de datos faltantes, lo que nos permite evaluar la generalización del modelo de imputación ante diferentes configuraciones de información disponible.

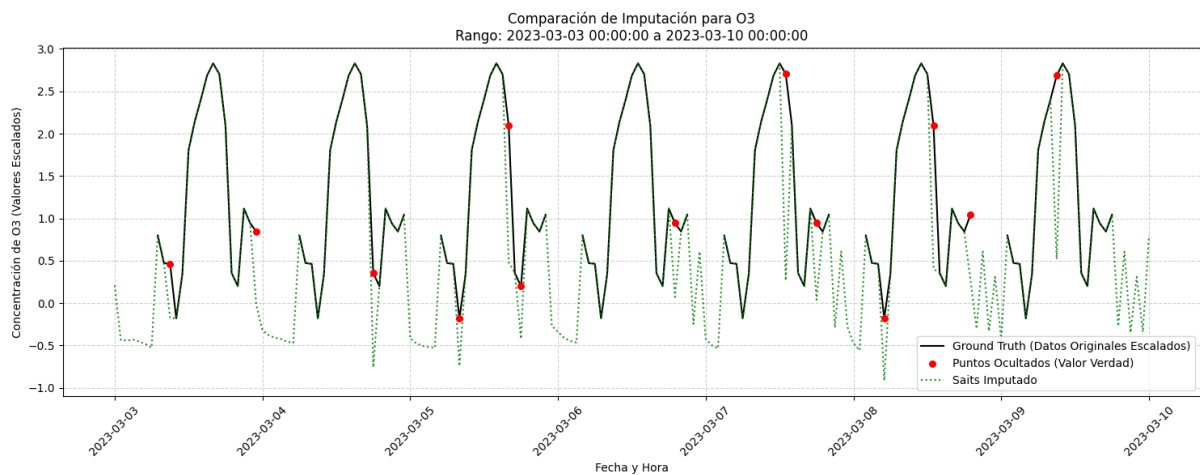


Figura 15: Comparación de imputación para la variable O₃.

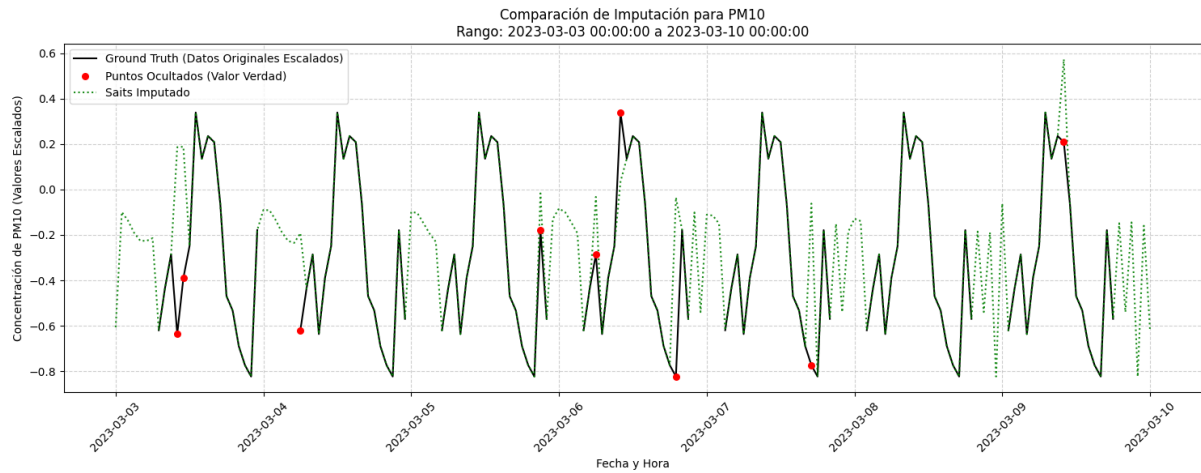


Figura 16: Comparación de imputación para la variable PM10.

En estas figuras, la línea continua negra representa el *ground truth* (datos originales escalados), los puntos rojos indican los valores que fueron ocultados artificialmente para la prueba del modelo, y la línea punteada verde corresponde a los valores imputados por el modelo SAITS. Se puede observar cómo el modelo de imputación sigue las tendencias generales de los datos originales y la precisión con la que reconstruye los valores en los puntos ocultados.

5.8 EVALUACIÓN DEL RENDIMIENTO

El rendimiento final de cada método se evalúa utilizando el módulo `pampaneira_imputation.evaluation`. Este calcula las métricas MAE, MSE, RMSE y MRE comparando las predicciones $\hat{\mathcal{X}}^{\text{test}}$ con el ground truth $\mathcal{X}_{\text{orig}}^{\text{test}}$ únicamente en las posiciones indicadas por la máscara artificial $\mathbf{M}_{\text{artificial}}^{\text{test}}$.

Las siguientes tablas presentan los resultados de la evaluación de ocho métodos de imputación de datos (Median, Mean, Linear, Ffill, Bfill, Hankel, Transformer, SAITS) en dos periodos distintos, utilizando cuatro métricas de error: RMSE, MSE, MAE y MRE.

Cuadro 2: Resultados de Evaluación - Periodo 1 (10 % ausencia, patrón 'point')

Métrica	Median	Mean	Linear	Ffill	Bfill	Hankel	Transformer	Saits
RMSE	1,1342	1,0521	0,8796	1,0604	1,0487	0,8071	0,6488	0,6360
MSE	1,2865	1,1069	0,7737	1,1244	1,0997	0,6514	0,4209	0,4045
MAE	0,6910	0,7346	0,4621	0,5618	0,5554	0,5432	0,3422	0,3263
MRE	0,9311	0,9898	0,6226	0,7569	0,7484	0,7319	0,4611	0,4397

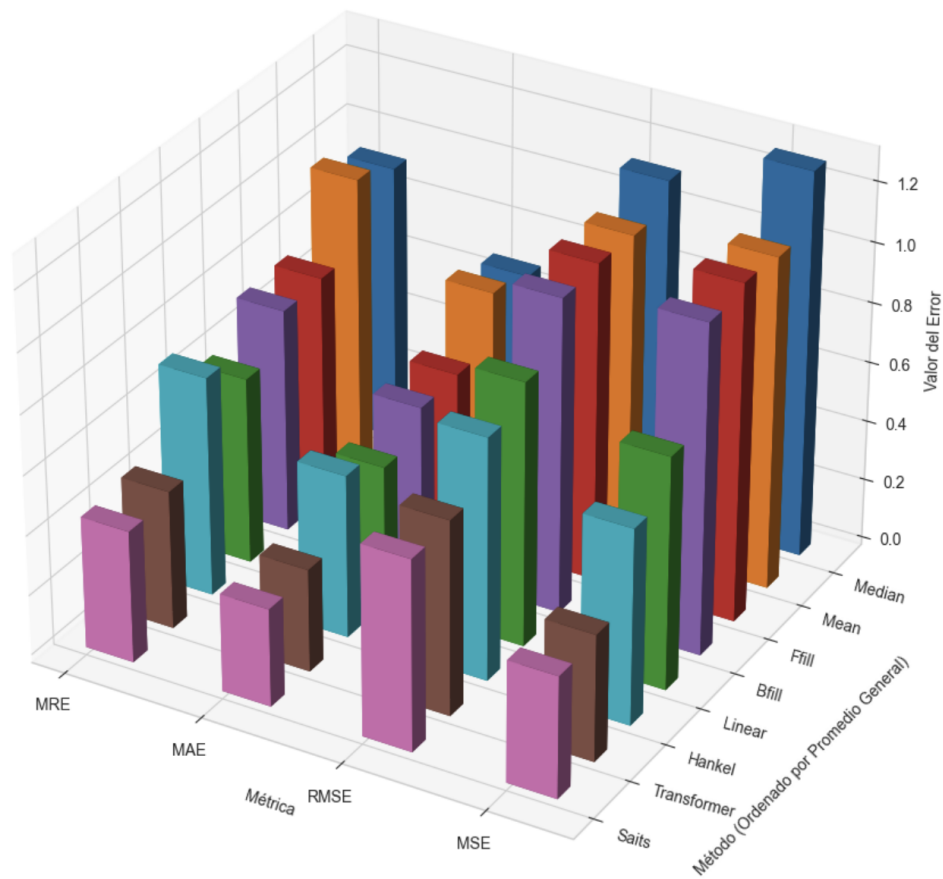


Figura 17: Comparativa tridimensional de las métricas de error (RMSE, MSE, MAE, MRE) para los diferentes métodos de imputación durante el Periodo 1 (10 % ausencia, patrón 'point'). Los métodos están ordenados globalmente según su rendimiento promedio.

Cuadro 3: Resultados de Evaluación - Periodo 2 (10 % ausencia, patrón 'point')

Métrica	Median	Mean	Linear	Ffill	Bfill	Hankel	Transformer	Saits
RMSE	1,1103	1,0609	0,9252	1,0754	1,1018	0,8754	0,7520	0,7428
MSE	1,2329	1,1255	0,8559	1,1565	1,2140	0,7663	0,5655	0,5517
MAE	0,7177	0,7637	0,4915	0,5836	0,5894	0,5971	0,4123	0,3980
MRE	0,9161	0,9747	0,6273	0,7470	0,7544	0,7621	0,5262	0,5079

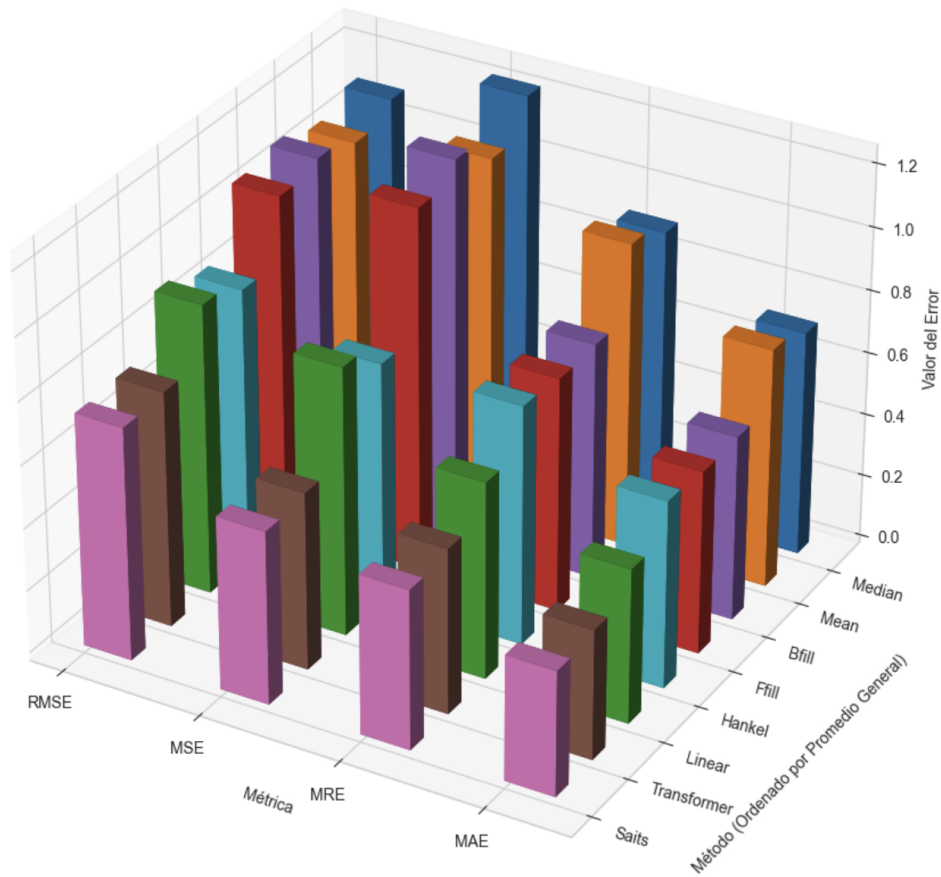


Figura 18: Comparativa tridimensional de las métricas de error (RMSE, MSE, MAE, MRE) para los diferentes métodos de imputación durante el Periodo 2 (10 % ausencia, patrón 'point'). Los métodos están ordenados globalmente según su rendimiento promedio.

Nuevos Experimentos con el Dataset de Poqueira

Para complementar el análisis, se realizaron experimentos adicionales en el dataset de Poqueira, variando el porcentaje de datos faltantes y el patrón de eliminación, permitiendo una evaluación más robusta del comportamiento de los métodos de imputación bajo diferentes condiciones.

Experimentos con patrón 'point' (puntual): Se introdujeron ausencias artificiales puntuales en los porcentajes del 5 %, 20 % y 50 % de los datos observados.

Cuadro 4: Resultados de Evaluación - Periodo 1 (5 % ausencia, patrón 'point')

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,1623	1,0886	0,9995	1,1590	1,1588	0,6881	0,6696
MSE	1,3510	1,1850	0,9989	1,3433	1,3429	0,4735	0,4484
MAE	0,7142	0,7495	0,5487	0,6480	0,6469	0,3637	0,3504
MRE	0,9600	1,0075	0,7376	0,8711	0,8696	0,4889	0,4710

Cuadro 5: Resultados de Evaluación - Periodo 2 (5 % ausencia, patrón 'point')

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,1392	1,0849	1,0187	1,1759	1,1799	0,7628	0,7521
MSE	1,2977	1,1771	1,0377	1,3828	1,3921	0,5819	0,5657
MAE	0,7378	0,7736	0,5642	0,6695	0,6668	0,4126	0,4058
MRE	0,9504	0,9965	0,7268	0,8643	0,8608	0,5316	0,5228

Cuadro 6: Resultados de Evaluación - Periodo 1 (20 % ausencia, patrón 'point')

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,1440	1,0602	0,8798	1,0541	1,0573	0,6498	0,6415
MSE	1,3086	1,1240	0,7741	1,1111	1,1179	0,4223	0,4115
MAE	0,6955	0,7377	0,4568	0,5535	0,5515	0,3364	0,3288
MRE	0,9302	0,9866	0,6110	0,7403	0,7376	0,4499	0,4397

Cuadro 7: Resultados de Evaluación - Periodo 2 (20 % ausencia, patrón 'point')

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,1061	1,0589	0,9045	1,0727	1,0736	0,7479	0,7336
MSE	1,2235	1,1212	0,8181	1,1507	1,1526	0,5593	0,5382
MAE	0,7187	0,7627	0,4822	0,5783	0,5744	0,4106	0,3896
MRE	0,9153	0,9713	0,6141	0,7379	0,7330	0,5229	0,4962

Cuadro 8: Resultados de Evaluación - Periodo 1 (50 % ausencia, patrón 'point')

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,1432	1,0595	0,8994	1,0735	1,0723	0,6606	0,6480
MSE	1,3070	1,1224	0,8090	1,1524	1,1499	0,4364	0,4199
MAE	0,6982	0,7369	0,4773	0,5754	0,5746	0,3453	0,3353
MRE	0,9398	0,9918	0,6425	0,7744	0,7734	0,4647	0,4513

Cuadro 9: Resultados de Evaluación - Periodo 2 (50 % ausencia, patrón 'point')

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,1172	1,0682	0,9492	1,1078	1,1238	0,7736	0,7508
MSE	1,2481	1,1411	0,9009	1,2273	1,2629	0,5985	0,5636
MAE	0,7182	0,7628	0,5055	0,6039	0,6055	0,4157	0,3985
MRE	0,9192	0,9763	0,6469	0,7746	0,7767	0,5320	0,5100

Experimentos con patrón 'subseq' (ráfaga) con longitud entre 8 y 12 horas: Se generaron ausencias artificiales en forma de ráfagas de longitud variable (entre 8 y 12 horas) en los porcentajes del 10 %, 20 % y 50 % de los datos observados.

Cuadro 10: Resultados de Evaluación - Periodo 1 (10 % ausencia, patrón 'subseq', longitud 8-12h)

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,3268	1,2476	1,2188	1,3704	1,3628	0,6819	0,6574
MSE	1,7604	1,5565	1,4855	1,8780	1,8573	0,4650	0,4322
MAE	0,8666	0,8681	0,7420	0,8574	0,8204	0,3522	0,3370
MRE	1,1536	1,1556	0,9877	1,1414	1,0922	0,4688	0,4486

Cuadro 11: Resultados de Evaluación - Periodo 2 (10 % ausencia, patrón 'subseq', longitud 8-12h)

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,3391	1,2408	1,1760	1,3823	1,3831	0,7859	0,7611
MSE	1,7931	1,5396	1,3830	1,9107	1,9130	0,6176	0,5792
MAE	0,9164	0,8981	0,7378	0,8742	0,8819	0,4015	0,3890
MRE	1,1609	1,1376	0,9346	1,1106	1,1204	0,5086	0,4927

Cuadro 12: Resultados de Evaluación - Periodo 1 (20 % ausencia, patrón 'subseq', longitud 8-12h)

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,3091	1,2399	1,2356	1,3988	1,3624	0,6653	0,6508
MSE	1,7137	1,5374	1,5266	1,9567	1,8562	0,4427	0,4235
MAE	0,8541	0,8630	0,7507	0,8736	0,8133	0,3420	0,3299
MRE	1,1450	1,1569	1,0064	1,1711	1,0903	0,4584	0,4423

Cuadro 13: Resultados de Evaluación - Periodo 2 (20 % ausencia, patrón 'subseq', longitud 8-12h)

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,3342	1,2423	1,2161	1,3747	1,3931	0,8019	0,7838
MSE	1,7801	1,5432	1,4789	1,8899	1,9406	0,6430	0,6144
MAE	0,9074	0,8944	0,7614	0,8757	0,8853	0,4247	0,4054
MRE	1,1687	1,1520	0,9806	1,1291	1,1415	0,5470	0,5222

Cuadro 14: Resultados de Evaluación - Periodo 1 (50 % ausencia, patrón 'subseq', longitud 8-12h)

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,2944	1,2242	1,2103	1,3939	1,3535	0,6743	0,6581
MSE	1,6755	1,4986	1,4647	1,9430	1,8319	0,4546	0,4332
MAE	0,8500	0,8590	0,7374	0,8662	0,8090	0,3518	0,3459
MRE	1,1430	1,1551	0,9917	1,1648	1,0880	0,4731	0,4652

Cuadro 15: Resultados de Evaluación - Periodo 2 (50 % ausencia, patrón 'subseq', longitud 8-12h)

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,3032	1,2123	1,1853	1,3597	1,3595	0,7713	0,7530
MSE	1,6982	1,4696	1,4049	1,8489	1,8482	0,5950	0,5671
MAE	0,8943	0,8846	0,7500	0,8656	0,8673	0,4150	0,4021
MRE	1,1591	1,1465	0,9720	1,1240	1,1261	0,5379	0,5212

Visualizaciones de PM₁₀ para los nuevos experimentos en Poqueira

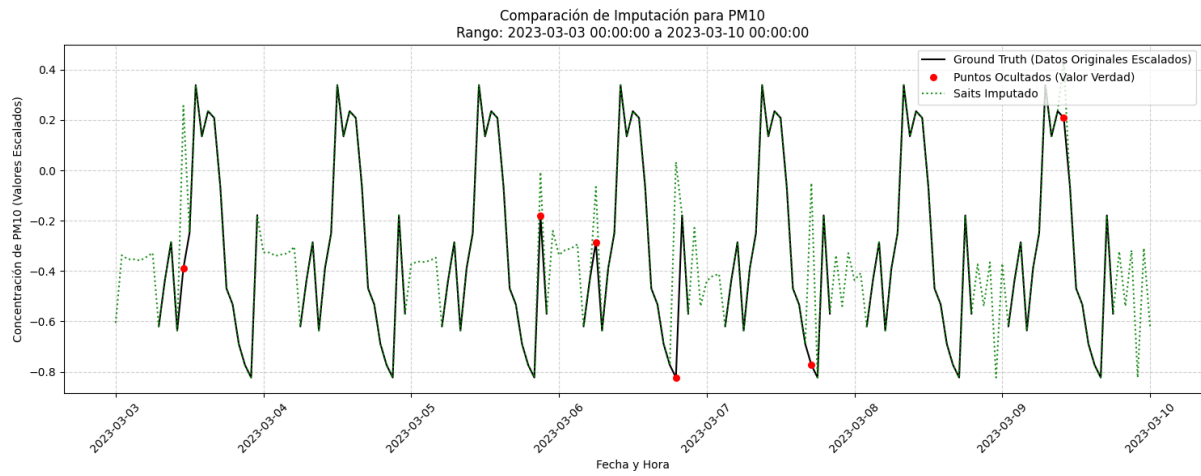


Figura 19: Comparación de imputación para la variable PM₁₀ en Poqueira (5 % ausencia, patrón 'point', Periodo 2).

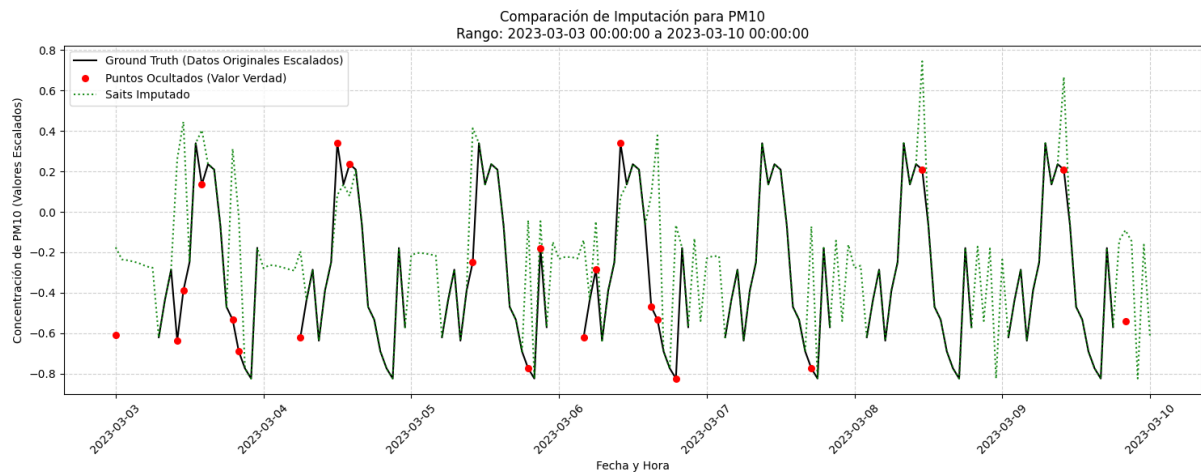


Figura 20: Comparación de imputación para la variable PM₁₀ en Poqueira (20 % ausencia, patrón 'point', Periodo 2).

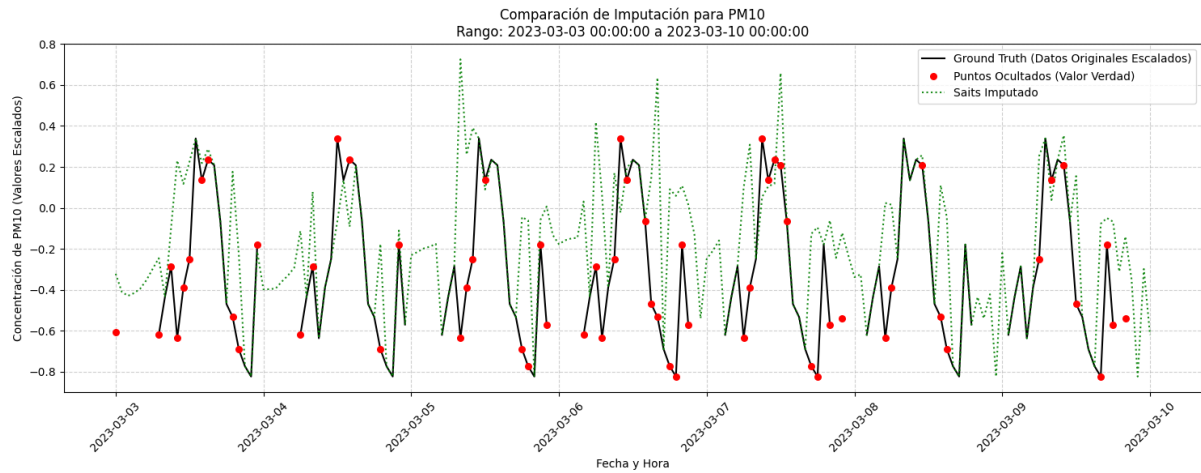


Figura 21: Comparación de imputación para la variable PM10 en Poqueira (50 % ausencia, patrón 'point', Periodo 2).

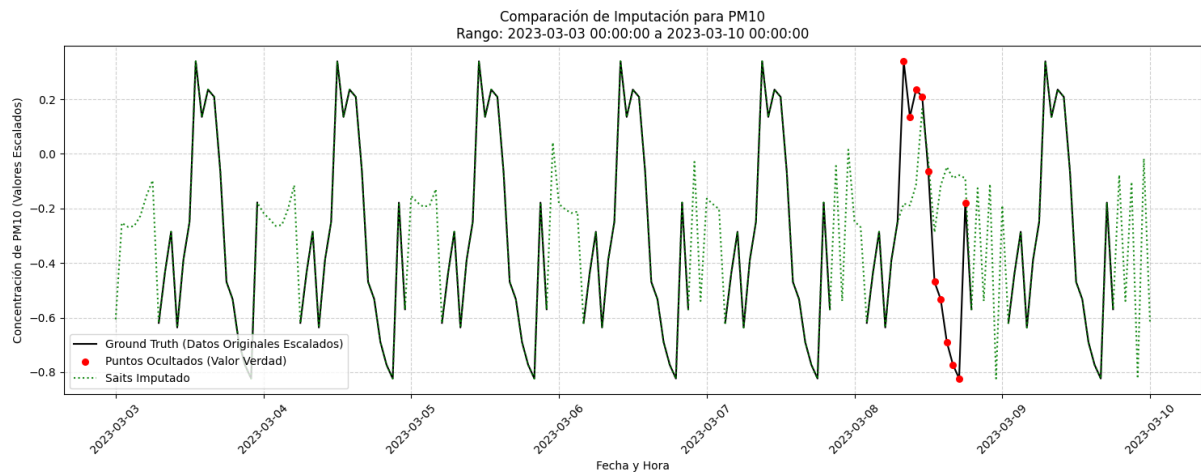


Figura 22: Comparación de imputación para la variable PM10 en Poqueira (10 % ausencia, patrón 'subseq', longitud 8-12h, Periodo 2).

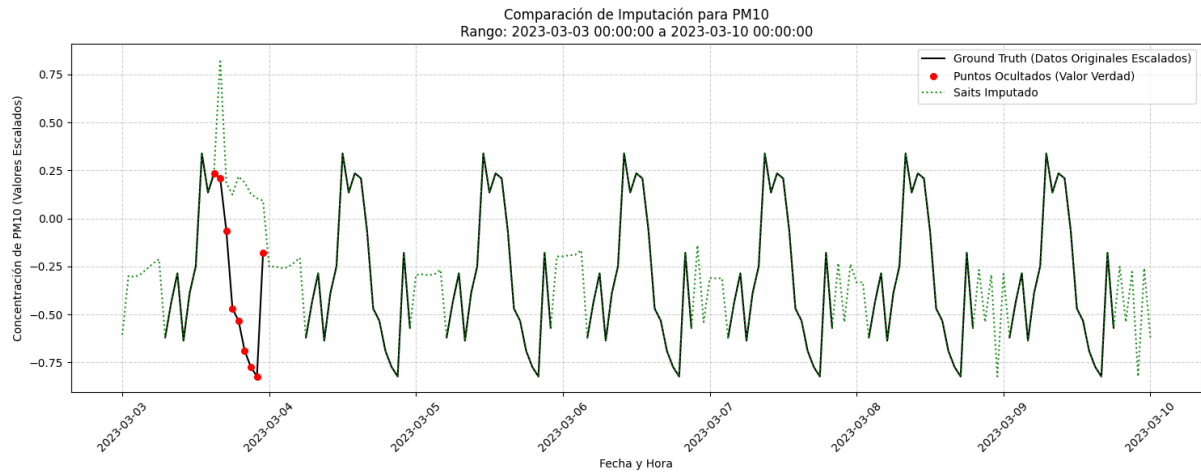


Figura 23: Comparación de imputación para la variable PM10 en Poqueira (20 % ausencia, patrón 'subseq', longitud 8-12h, Periodo 2).

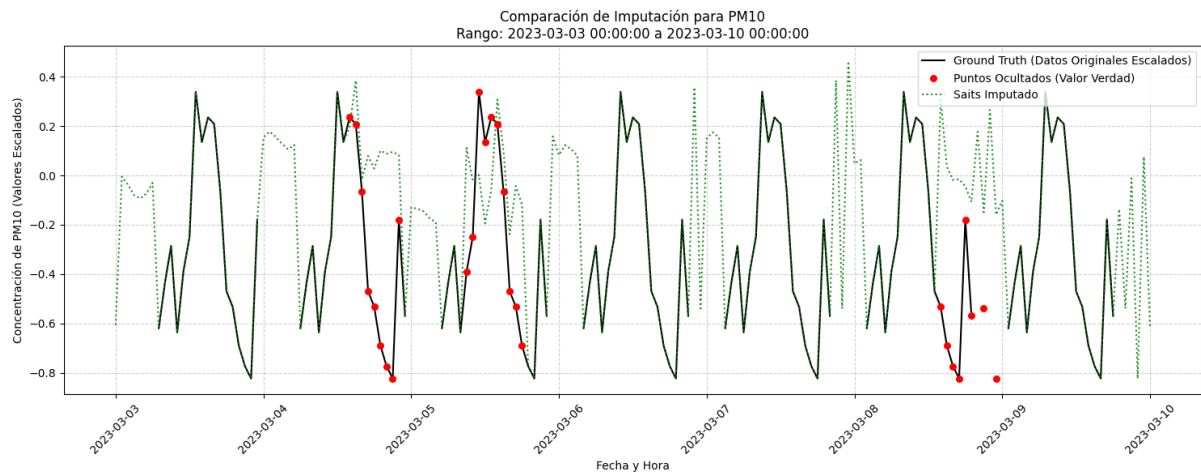


Figura 24: Comparación de imputación para la variable PM10 en Poqueira (50 % ausencia, patrón 'subseq', longitud 8-12h, Periodo 2).

INTERPRETACIÓN DE LOS RESULTADOS

Para todas estas métricas, **valores más bajos indican un mejor rendimiento** del método de imputación, ya que representan un menor error entre los valores imputados y los valores reales.

Discusión General del Rendimiento de los Métodos

- **Rendimiento de los Métodos Básicos y Tradicionales:** Los enfoques más elementales como la imputación por **Mean** y **Median** registraron consistentemen-

te los errores más elevados, demostrando una limitada capacidad para inferir datos faltantes con precisión. Los métodos de propagación, **Ffill** y **Bfill**, exhibieron un rendimiento inconsistente y variable, siendo sensibles a la posición de los datos faltantes en la secuencia. Entre los métodos tradicionales más simples, la imputación **Linear** (interpolación lineal) se mostró como la más efectiva, superando a los enfoques basados en media, mediana y propagación en la mayoría de los escenarios.

- **Imputación Hankel (HI): Un Intermedio Eficaz:** La **Imputación Hankel**, un método basado en la completación de matrices de bajo rango, se destacó por ofrecer un rendimiento notablemente superior al de los métodos básicos (Mean, Median, Ffill, Bfill) y a la interpolación lineal (Linear). Para el escenario de 10 % de ausencia puntual, tanto en el Periodo 1 (Cuadro 4) como en el Periodo 2 (Cuadro 9), **Imputación Hankel** mostró errores significativamente menores que Linear en todas las métricas, posicionándose claramente como el tercer método más preciso. Este resultado sugiere que la Imputación Hankel es capaz de capturar eficazmente las dependencias temporales y la estructura subyacente de la serie, lo que le permite imputar valores con una alta fidelidad comparado con los enfoques más simples.
- **Dominio Absoluto de Modelos Predictivos Avanzados:** Los modelos basados en arquitecturas Transformer, específicamente **SAITS** y **Transformer**, demostraron consistentemente una precisión superior en la imputación de datos faltantes en comparación con todos los demás métodos, incluidos los tradicionales y la Imputación Hankel. Esto se reflejó en errores notablemente inferiores en todas las métricas y en ambos periodos de estudio. Dentro de este grupo de modelos avanzados, **SAITS** se destacó por ofrecer el rendimiento más preciso de manera generalizada, obteniendo los errores más bajos en la mayoría de las evaluaciones, aunque seguido de cerca por el modelo **Transformer**. Su capacidad para modelar dependencias a largo plazo y aprender representaciones complejas de los datos les otorgó una ventaja decisiva en la reconstrucción de información faltante, estableciendo un claro liderazgo en la jerarquía de precisión.

Análisis Comparativo entre Periodos de Estudio

- **Variación del Error entre Periodos:** Se observó una tendencia general hacia errores de imputación ligeramente mayores durante el Periodo 2 en comparación con el Periodo 1 para la mayoría de los métodos, especialmente los de mejor rendimiento. Esto incluyó a la **Imputación Hankel**, cuyo error, si bien se mantuvo muy bajo, también mostró un ligero incremento. Esta diferencia podría deberse a que el Periodo 2 es mucho más reducido, limitando la cantidad de datos para la inferencia.
- **Estabilidad en la Jerarquía de Métodos:** A pesar de las fluctuaciones en los niveles de error absoluto entre los dos periodos, la jerarquía de rendimiento

relativo entre los modelos se mantuvo en gran medida estable. Los modelos **SAITS** y **Transformer** continuaron siendo los más precisos, seguidos consistentemente por la **Imputación Hankel**, la cual mantuvo su sólida posición como el tercer método más preciso, confirmando su robustez y fiabilidad frente a los métodos más simples.

Observaciones Clave por Tipo de Métrica de Error

- **Impacto de Errores Grandes (RMSE y MSE):** Las métricas que magnifican el impacto de errores grandes, como RMSE y MSE, confirmaron la robustez de **SAITS**, **Transformer** y la **Imputación Hankel**. Sus bajos valores en estas métricas sugieren que estos modelos no solo son precisos en promedio, sino que también son menos propensos a cometer errores de gran magnitud.
- **Error Promedio Absoluto (MAE):** El MAE, que mide la desviación promedio absoluta, también resaltó la superioridad de **SAITS**, **Transformer** y la **Imputación Hankel**, indicando que sus imputaciones se encuentran, en promedio, más próximas a los valores verdaderos.
- **Error Relativo (MRE):** Al considerar el error desde una perspectiva porcentual a través del MRE, se reafirmó la mayor precisión de los modelos **SAITS**, **Transformer** y la **Imputación Hankel** en comparación con los otros métodos evaluados.

Análisis de los Resultados en el Dataset de Poqueira bajo diferentes escenarios

Se evaluaron los métodos de imputación con diferentes porcentajes de ausencia (5 %, 10 %, 20 %, 50 %) y patrones ('point' y 'subseq' con longitud 8-12h).

- **Impacto del Porcentaje de Datos Faltantes (Patrón 'point'):** Al aumentar el porcentaje de datos faltantes (de 5 % a 50 %) con el patrón 'point' (Tablas 4 a 9), se observa una tendencia general al **incremento de los errores** para todos los métodos. Esto es esperado, ya que una mayor cantidad de datos ausentes reduce la información disponible para realizar imputaciones precisas. Sin embargo, los modelos avanzados (Transformer y SAITS) mantienen su superioridad, mostrando un aumento de error más moderado en comparación con los métodos tradicionales, lo que subraya su robustez ante escenarios con mayor escasez de datos.
- **Impacto del Patrón de Ausencia ('point' vs. 'subseq'):** Al comparar los resultados para un mismo porcentaje de ausencia (por ejemplo, 10 %, 20 %, 50 %) pero con diferentes patrones (Tablas 2/3 vs. 10/11), se observa que el patrón 'subseq' generalmente resulta en **errores mayores** para todos los métodos. Las ráfagas de datos faltantes (subsecuencias) representan un desafío mayor que las ausencias puntuales, ya que la información temporal contigua es más limitada.

En este escenario más complejo, la ventaja de los modelos Transformer y SAITS se hace aún más pronunciada, demostrando su capacidad para capturar dependencias a largo plazo y reconstruir secuencias completas de datos faltantes de manera más efectiva que los métodos simples. El método Linear, aunque mejor que Mean/Median/Ffill/Bfill en el patrón 'point', también ve su rendimiento deteriorarse significativamente con 'subseq' en comparación con los modelos avanzados.

- **Rendimiento de los Métodos Tradicionales:** Los métodos como Median, Mean, Ffill y Bfill muestran un rendimiento consistentemente inferior, especialmente a medida que la proporción de datos faltantes o la complejidad del patrón de ausencia aumenta. Su dependencia de información local o agregada simple los hace menos efectivos para inferir valores en series temporales con patrones complejos o extensos periodos de ausencia.

5.9 VALIDACIÓN EN OTRO DATASET DE CONTAMINACIÓN DE LA LITERATURA

Para complementar las pruebas con datos de Poqueira y validar la capacidad de SAITS en un benchmark reconocido, se realizó un experimento adicional con el conjunto de datos de calidad del aire de Beijing (Air-Quality) (cf. [Du et al., 2023b](#)). Este dataset incluye datos horarios de contaminantes atmosféricos de 12 sitios de monitoreo en Beijing, recopilados durante 48 meses (marzo de 2013 a febrero de 2017), resultando en 132 variables de características. Originalmente presenta una tasa de valores faltantes del 1.6%, y para la evaluación controlada del rendimiento de los modelos, se generaron ausencias artificiales **puntuales** en el 10% de los valores observados. Este experimento, que replicó parte de la metodología del estudio de referencia, tuvo como objetivo demostrar la eficacia del modelo en un contexto externo. Los resultados obtenidos para las métricas de error, registrados en la [Tabla 16](#), confirman el desempeño superior de los modelos basados en autoatención. El **Jupyter Notebook**⁴ se encuentra disponible en el repositorio de Github.

Cuadro 16: Resultados de Evaluación - Dataset Air-Quality de Beijing (10 % ausencia, patrón 'point')

Métrica	Median	Mean	Ffill	Transformer	Saits
RMSE	0,6435	0,6170	0,5493	0,5320	0,5156
MSE	0,4141	0,3806	0,3018	0,2831	0,2659
MAE	0,3116	0,3202	0,1896	0,1940	0,1793
MRE	0,4145	0,4259	0,2517	0,2580	0,2385

⁴ https://github.com/eigenric/TFG/blob/main/notebooks/Notebook_Pekin.ipynb

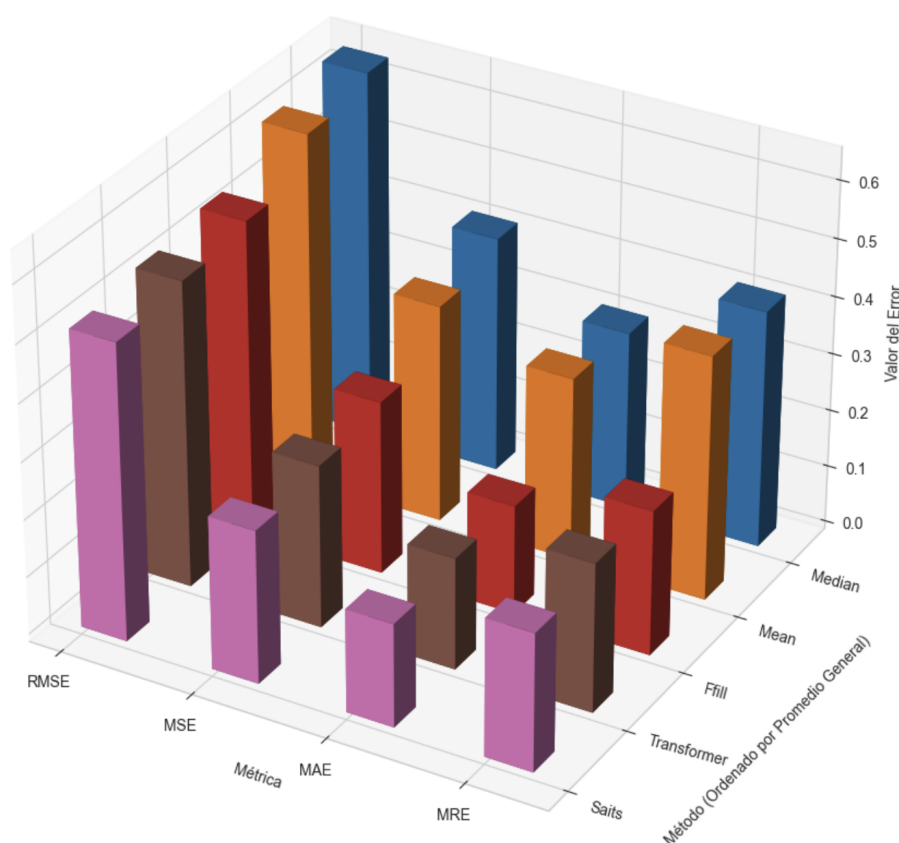


Figura 25: Comparativa tridimensional de las métricas de error para los diferentes métodos de imputación en el Dataset de Air-Quality de Beijing.

Extendiendo el análisis de patrones de ausencia, se evaluaron dos configuraciones adicionales para el dataset de calidad del aire de Beijing, utilizando un patrón de eliminación por **ráfaga** (**'subseq'**) para secuencias de 5 horas, con porcentajes de datos faltantes del 20 % y 50 %. Los resultados se presentan en las [Tabla 17](#) y [Tabla 18](#) respectivamente.

- **Impacto del Porcentaje de Datos Faltantes:** Al comparar los resultados de los patrones de ráfaga con longitud 5 (Tablas 17 y 18), se observa una clara tendencia: a medida que el porcentaje de datos faltantes aumenta del 20 % al 50 %, todas las métricas de error para todos los métodos tienden a **incrementar**. Esto es esperable, ya que una mayor escasez de información reduce la capacidad de inferencia. Sin embargo, los modelos Transformer y SAITS consistentemente mantienen los errores más bajos en comparación con los métodos tradicionales.
- **Rendimiento de los Métodos ante el Patrón 'Subseq':** Bajo el patrón de eliminación 'subseq', los modelos Transformer y SAITS continúan demostrando una superioridad significativa en todas las métricas. La imputación por ffill muestra un rendimiento competitivo frente a la mediana y la media en este tipo de patrón estructurado, y en algunos casos supera incluso a la media. Como es común, los métodos simples como la mediana y la media registran los errores más

elevados, reflejando su limitada capacidad para manejar patrones de ausencia complejos.

Cuadro 17: Resultados de Evaluación - Dataset Air-Quality de Beijing (20 % Faltantes, Patrón Subseq, Longitud 5)

Métrica	Median	Mean	Ffill	Transformer	Saits
RMSE	0,6891	0,6374	0,6971	0,4827	0,4710
MSE	0,4749	0,4063	0,4860	0,2330	0,2219
MAE	0,3643	0,3588	0,3009	0,1961	0,1827
MRE	0,4845	0,4772	0,3991	0,2608	0,2429

Cuadro 18: Resultados de Evaluación - Dataset Air-Quality de Beijing (50 % Faltantes, Patrón Subseq, Longitud 5)

Métrica	Median	Mean	Ffill	Transformer	Saits
RMSE	0,7945	0,7506	0,7778	0,6385	0,6254
MSE	0,6313	0,5635	0,6050	0,4076	0,3911
MAE	0,3770	0,3696	0,3605	0,2167	0,2021
MRE	0,4981	0,4883	0,4752	0,2864	0,2670

CRONOGRAMAS Y PRESUPUESTO

6.1 PLANIFICACIÓN DEL TRABAJO Y ANÁLISIS DEL PRESUPUESTO

En esta sección se detalla tanto la planificación estimada inicial como la seguida durante el desarrollo del Trabajo Fin de Grado (TFG), junto con un análisis del presupuesto basado en el tiempo real invertido. La planificación y el seguimiento se visualizan a través de diagramas de Gantt generados con el software de código abierto **GanttProject**¹. El análisis presupuestario se fundamenta en las **horas reales de trabajo registradas** mediante la herramienta de gestión de tiempo de código abierto **Timewarrior**². El objetivo es ofrecer una valoración transparente y fiel a la dedicación invertida en el proyecto, contrastando la planificación inicial con la ejecución real.

6.1.1 *Planificación del Trabajo: Estimación vs. Realidad*

El diseño inicial y el posterior seguimiento de la planificación del proyecto se han gestionado utilizando GanttProject. En el **Anexo A** se presentan dos diagramas de Gantt para ilustrar la diferencia entre la estimación inicial y el progreso real:

- La **Figura 26** muestra una planificación inicial optimista, elaborada al comienzo del proyecto.
- La **Figura 27** refleja la progresión y duración real de las tareas, tal como se desarrollaron.

¹ <https://www.ganttproject.biz/>

² <https://timewarrior.net/>

6.2 CRONOGRAMAS

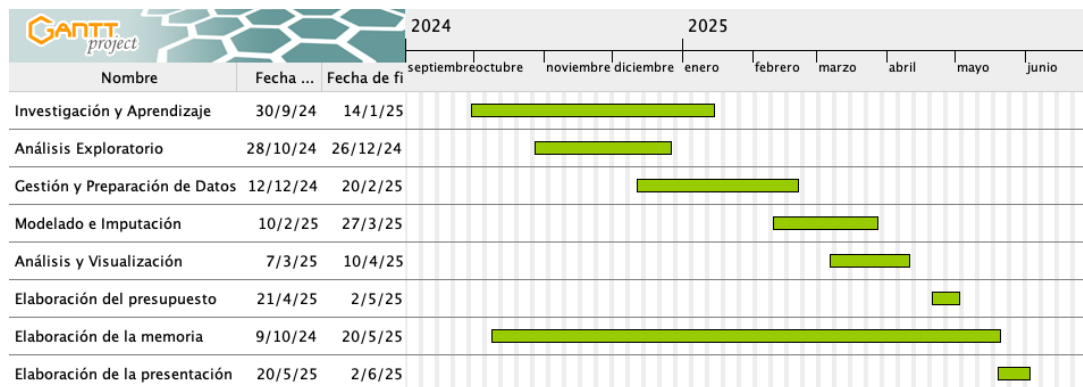


Figura 26: Diagrama de Gantt de la planificación inicial estimada para la realización del TFG.

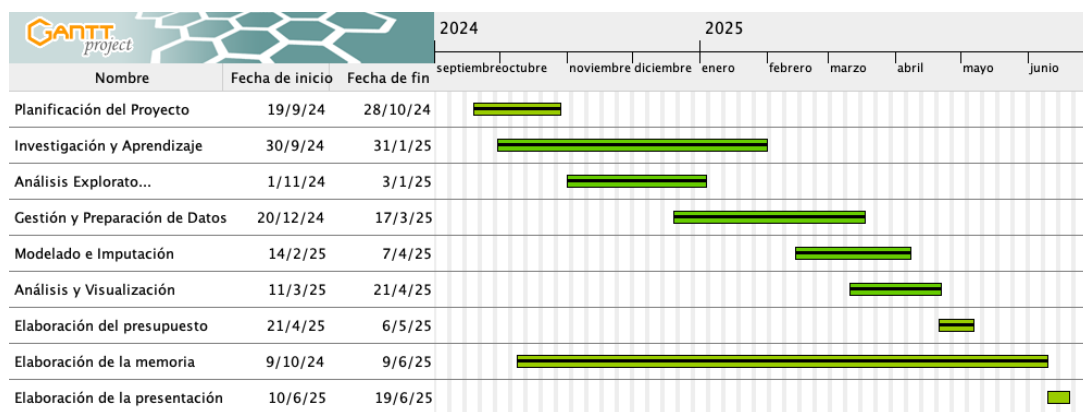


Figura 27: Diagrama de Gantt del progreso real durante la realización del TFG.

La comparación entre ambos cronogramas evidencia que el desarrollo real del proyecto sufrió una desviación con respecto a la planificación inicial. Este retraso se debió principalmente a los siguientes factores:

- **Profundización en el Estado del Arte:** La revisión bibliográfica y el estudio de trabajos previos requirieron más tiempo del previsto para establecer una base teórica robusta y contextualizar adecuadamente el proyecto.
 - **Complejidad de la Arquitectura Transformer:** Comprender en detalle los mecanismos de atención y la implementación de los modelos Transformer supuso una inversión de tiempo considerable, fundamental para su correcta aplicación.
- **Curva de Aprendizaje de Bibliotecas:** La familiarización y el manejo eficiente de las bibliotecas de Python utilizadas (e.g., PyPOTS basada en PyTorch, Pandas, Numpy para manipulación de datos y Seaborn para visualización) implicaron un periodo de adaptación necesario para la implementación práctica.

6.3 METODOLOGÍA UTILIZADA

La metodología adoptada en este Trabajo de Fin de Grado se enmarca en la tipología de **TFG de investigación**, dado que su propósito es generar conocimiento y validar una hipótesis mediante un proceso sistemático. Dicha investigación se sustenta en una **fundamentación matemática**. Este rigor teórico se complementa con una **revisión sistemática** del estado del arte, que garantiza una comprensión de las técnicas de imputación existentes y posiciona las contribuciones del trabajo.

En el aspecto práctico, la construcción del paquete de software se llevó a cabo siguiendo una **metodología ágil**, en particular el marco de trabajo **Scrum**, lo que permitió un desarrollo iterativo y adaptable. El proceso general de investigación siguió las etapas del **método científico**, desde la formulación del problema y la hipótesis, hasta el diseño y ejecución de experimentos empíricos con evaluación de resultados. Este enfoque integrado de investigación fundamental y desarrollo práctico busca ofrecer una solución validada al desafío de la imputación de datos en series temporales multivariantes.

6.4 ANÁLISIS DEL PRESUPUESTO DEL PROYECTO

El presupuesto presentado a continuación se basa en la contabilización precisa de las horas dedicadas a cada tarea del TFG, registradas sistemáticamente con Timewarrior a lo largo de todo el desarrollo. Para la valoración económica, se ha aplicado la tarifa de 20 euros/hora. (Guillén Perales et al., 2025)

Para facilitar la comprensión y el análisis del presupuesto, el trabajo registrado se ha clasificado en tres categorías principales, siguiendo una estructura habitual:

- **Parte teórica:** Recoge las tareas relacionadas con el análisis conceptual, el estudio del estado del arte y la fundamentación teórica del proyecto. Corresponde a la categoría agrupada *Investigación y Aprendizaje*.
- **Parte práctica:** Reúne las actividades directamente vinculadas con la implementación, programación, experimentación, análisis de datos y validación. Engloba las categorías de *Pipeline y Datos*, *Modelado y Entrenamiento*, *Imputación y Nulos*, *Análisis y Visualización*, y *Metodología y Estructura*.
- **Parte general:** Agrupa las labores transversales de gestión del proyecto, comunicación (reuniones con tutores, correos), documentación y elaboración de la memoria. Corresponde a la categoría *Gestión y Comunicaciones*.

Además de los costes derivados de las horas de trabajo, se incluye el coste amortizado del equipamiento principal utilizado (Macbook Air M2), considerando una vida

útil estimada de 7 años ³. El desglose detallado del presupuesto se presenta en el (Cuadro 19).

6.5 PRESUPUESTO ESTIMADO

Cuadro 19: Estimación Detallada del Coste de Desarrollo del Proyecto (TFG) - Duración 9 Meses

Concepto	Horas	Coste Bruto(Sueldo) (euros)	Coste SS(Empresa) (euros)	Coste TotalProyecto (euros)
1. Coste de Personal				
Gestión y Comunicaciones	1,1	22,00	6,58	28,58
Investigación y Aprendizaje	223,9	4478,00	1339,42	5817,42
Pipeline y Datos	35,0	700,00	209,30	909,30
Modelado y Entrenamiento	32,6	652,00	194,95	846,95
Imputación y Nulos	185,3	3706,00	1108,09	4814,09
Análisis y Visualización	5,6	112,00	33,49	145,49
Metodología y Estructura	3,5	70,00	20,93	90,93
Tutores (2 personas)	72,0	2880,00	861,12	3741,12
Subtotal Coste de Personal	559.0	12620.00	3773.88	16393.88
2. Coste de Material (Hardware/Software)				
Amortización Macbook Air M2 (9 meses de uso)				139,29
Subtotal Coste de Material				139.29
3. Costes Indirectos (Estimación 9 meses)				
Suministro Eléctrico	9 meses			293,81
Suministro de Agua	9 meses			221,49
Conexión a Internet	9 meses			180,00
Subtotal Costes Indirectos				695.30
COSTE TOTAL ESTIMADO DEL PROYECTO				17228.47

Nota: El coste de personal se basa en las horas registradas, aplicando una tarifa bruta consolidada de 20 €/hora para todas las tareas de desarrollador. Para el rol de Tutor, la tarifa se ha fijado en 40 €/hora (basado en 2 horas/semana durante 9 meses). Los costes de Seguridad Social a cargo de la empresa se han calculado como un 29.9 % del coste bruto del sueldo. El Impuesto sobre la Renta de las Personas Físicas (IPRF) es una retención aplicada al sueldo bruto del trabajador (no un coste adicional para el proyecto). La amortización del equipo considera un uso de 9 meses para el proyecto sobre una vida útil de 7 años (coste original del equipo 1300€). Los costes indirectos son una estimación mensual prorrateada para la duración supuesta del proyecto (9 meses).

³ <https://www.refurb.me/es/blog/cuanto-dura-un-macbook>

Parte IV

BLOQUE IV: CONCLUSIONES Y TRABAJO FUTURO

CONCLUSIONES Y TRABAJO FUTURO

Este capítulo final presenta las principales conclusiones derivadas del desarrollo de este Trabajo de Fin de Grado, centrado en la imputación de datos faltantes en series temporales de tráfico y contaminación mediante modelos Transformer. Se resumirán los hallazgos más significativos, se evaluará el cumplimiento de los objetivos planteados y se discutirán las limitaciones del estudio. Finalmente, se propondrán diversas líneas de investigación futuras que podrían surgir como continuación o extensión de este trabajo.

7.1 CONCLUSIONES PRINCIPALES

La investigación llevada a cabo y los resultados experimentales obtenidos permiten extraer una serie de conclusiones relevantes sobre la eficacia de los modelos basados en autoatención, y en particular SAITS, para la tarea de imputación de datos en series temporales multivariantes complejas, como las provenientes del sistema IoT del proyecto Smart Poqueira.

1. **Superioridad de los Modelos Basados en Transformer para Imputación:** Los resultados de la evaluación comparativa, presentados en las [Tabla 2](#) del [Capítulo 5](#), demuestran que los modelos basados en arquitecturas Transformer, específicamente el modelo SAITS ([Du et al., 2023b](#)) y un Transformer estándar adaptado para imputación, ofrecen una capacidad de recuperación de valores faltantes sustancialmente mejor que los métodos tradicionales (media, mediana, propagación), la interpolación lineal y el reciente método basado en matrices conocido como Imputación Hankel ([Poudevigne and Jones, 2024](#)). En todas las métricas de error evaluadas (RMSE, MSE, MAE y MRE) y para los dos periodos de estudio analizados, SAITS y Transformer lograron consistentemente los errores más bajos, indicando una mayor precisión en la estimación de los datos ausentes. Particularmente, el modelo SAITS mostró una notable mejora en la métrica MAE (Error Absoluto Medio) en comparación con la interpolación lineal, logrando una reducción del error de aproximadamente un **28,78 %** en el Periodo 1 y un **19,39 %** en el Periodo 2. Esta reducción significativa del error subraya la mayor precisión de SAITS en la estimación de los datos ausentes.

2. **Eficacia de la Interpolación Lineal entre Métodos Simples:** Entre los métodos de imputación más simples y tradicionales evaluados, la interpolación lineal se destacó como la opción preferible, superando consistentemente a la imputación por media, mediana y a las técnicas de propagación (forward fill y backward fill). Esto sugiere que, en ausencia de modelos predictivos complejos, aprovechar la información local de los puntos adyacentes de forma lineal es más beneficioso que utilizar estadísticas globales o la simple persistencia del último valor.
3. **Influencia de las Características del Periodo en la Imputación:** Se observó una ligera degradación en el rendimiento de todos los métodos de imputación durante el Periodo 2 en comparación con el Periodo 1. Aunque la jerarquía de rendimiento entre modelos se mantuvo estable, esta variación sugiere que las características intrínsecas de los datos (posiblemente diferentes patrones de variabilidad, estacionalidad o interacciones en el periodo más corto) pueden influir en la dificultad de la tarea de imputación y en la precisión alcanzable.
4. **Viabilidad de la Imputación en Variables con Alta Incidencia de Faltantes:** A pesar de la considerable proporción de datos faltantes en variables clave como eBC_tot, SO₂ y PM₁₀ (ver [Figura 7](#)), los modelos avanzados como SAITS demostraron capacidad para generar imputaciones coherentes, como se ilustra en las [Figuras 13 a 16](#). Esto es fundamental para poder realizar análisis posteriores más completos sobre la calidad del aire y sus fuentes.
5. **Rendimiento Generalizado para otros Escenarios de Ausencia (Dataset de Beijing):** La evaluación complementaria realizada con el dataset de calidad del aire de Beijing (Air-Quality) ([Tabla 17](#) a [Tabla 18](#)) confirmó la robustez y superioridad de los modelos Transformer y SAITS en una variedad de escenarios de datos faltantes. Se observó que una eliminación artificial de ráfagas de datos incrementa el error para todos los métodos. No obstante, SAITS y Transformer mantuvieron consistentemente el mejor rendimiento, demostrando su resiliencia y mayor capacidad para inferir valores ausentes en contextos más complejos, como las ausencias estructuradas y extensas.

7.2 CUMPLIMIENTO DE OBJETIVOS

A continuación, se evalúa el grado de consecución de los objetivos generales y específicos planteados en la [Sección 1.3](#) de este TFG:

- **Objetivo General:** (Investigar y aplicar modelos avanzados de aprendizaje profundo, basados en Transformers, para la imputación de datos faltantes en series temporales multivariantes del sistema IoT de Poqueira, evaluando su eficacia y desarrollando un prototipo funcional) *Este objetivo se considera cumplido.* Se ha realizado una investigación exhaustiva, se ha implementado y evaluado el modelo SAITS y un Transformer, y se ha desarrollado el paquete `pampaneira_imputation`.

■ **Objetivos Específicos:**

1. *Establecer una Fundamentación Teórica:* Se ha revisado y consolidado la base matemática y conceptual necesaria, como se detalla en el [Capítulo 2](#) y el [Capítulo 3](#).
2. *Analizar el Estado del Arte en Imputación de Series Temporales:* Se ha realizado una revisión de la literatura científica sobre los modelos y técnicas estadísticas de imputación más utilizadas, y se ha aportado una formalización matemática homogénea para el problema de series temporales.
3. *Aplicar un Modelo de Vanguardia a un Problema Real:* El modelo SAITS se aplicó a los datos del IoT de Poqueira, como se describe en el [Capítulo 5](#).
4. *Desarrollar un Prototipo de Software Funcional y Abierto:* Se ha diseñado e implementado un paquete de software en Python que encapsula la lógica necesaria para la implementación, aplicación y evaluación sistemática de modelos de imputación, abarcando desde métodos elementales e intermedios, como Hankel Imputation, hasta arquitecturas avanzadas como Transformer y SAITS. Este paquete, `pampaneira_imputation`, está disponible en GitHub, detallado en la [Sección 5.4](#).
5. *Evaluar el Desempeño del Modelo Implementado:* Se ha realizado una evaluación cuantitativa ([Sección 5.8](#)) utilizando múltiples métricas y periodos.

En general, se considera que los objetivos propuestos para este Trabajo de Fin de Grado han sido alcanzados satisfactoriamente, proporcionando tanto una comprensión teórica como una aplicación práctica y una evaluación de la imputación de datos con modelos Transformer.

7.3 LIMITACIONES DEL ESTUDIO

A pesar de los resultados positivos, es importante reconocer ciertas limitaciones inherentes a este estudio:

- **Alcance de los Datos:** El análisis se centró en dos periodos específicos de operación del camión de medición. Una mayor extensión temporal y variabilidad de los datos podría ofrecer una visión más completa del rendimiento de los modelos.
- **Hiperparámetros de los Modelos:** Aunque se utilizaron configuraciones razonables para los modelos de aprendizaje profundo, una búsqueda de hiperparámetros (hyperparameter tuning) mejorarían aún más su rendimiento.
- **Patrones de Datos Faltantes Artificiales:** Aunque se exploraron algunos patrones de eliminación de datos artificiales ("point", "subseq", "block"), estos son una simplificación que intenta reproducir los complejos mecanismos de ausencia que pueden ocurrir en el mundo real.
- **Evaluación en Tareas Posteriores (Downstream Tasks):** Si bien el objetivo era la imputación precisa, no se evaluó formalmente el impacto de las diferentes

calidades de imputación en tareas posteriores específicas, como el modelado predictivo de la contaminación o el análisis causal.

- **Comparativa con Modelos No Neuronales Avanzados:** La comparativa se centró en métodos simples y modelos de aprendizaje profundo. La inclusión de otros métodos estadísticos avanzados para series temporales (e.g., imputación multivariante con otros modelos de series temporales) podría enriquecer la perspectiva.

7.4 TRABAJO FUTURO

Este TFG abre diversas vías para futuras investigaciones:

1. **Mejora y Adaptación de Modelos Transformer para Imputación:** Incluye explorar arquitecturas más eficientes, mejores encodings posicionales, funciones de pérdida específicas para series temporales.
2. **Integración con Tareas de Análisis Posteriores:** Se propone evaluar el impacto de la imputación en modelos predictivos o explicativos (e.g., pronóstico de contaminación) y desarrollar pipelines integrados de imputación y análisis.
3. **Desarrollo de Herramientas de Software Robustas:** Continuar el desarrollo del paquete `pampaneira_imputation` (con más métodos, visualizaciones, optimización de hiperparámetros) y contribuir a bibliotecas como PyPOTS (Du, 2023).
4. **Aplicación para Objetivos de Desarrollo Sostenible (ODS):** Investigar cómo la imputación de datos de monitorización ambiental y movilidad contribuye a ODS como el 11 (Ciudades Sostenibles), 3 (Salud) y 13 (Acción por el Clima), y desarrollar estudios de caso para indicadores de sostenibilidad.

7.5 CONCLUSIÓN FINAL

Este trabajo ha demostrado la superioridad de los modelos basados en Transformers, particularmente SAITS, para la imputación precisa de datos faltantes en series temporales complejas del sistema IoT de Poqueira, superando a métodos tradicionales. Se constató la eficacia de la interpolación lineal entre los enfoques más simples y se observó la influencia de las características del periodo en la dificultad de la imputación. Se han logrado los objetivos propuestos, sentando bases para futuras investigaciones en la mejora de la calidad de datos orientada a la sostenibilidad.

BIBLIOGRAFÍA

- Abu-Mostafa, Y. S., Magdon-Ismail, M., and Lin, H.-T. (2012a). *Learning from Data*. AMLBook.
- Abu-Mostafa, Y. S., Magdon-Ismail, M., and Lin, H.-T. (2012b). *Learning From Data*. AMLBook.
- Ahmed, E., Yaqoob, I., Gani, A., Imran, M., and Guizani, M. (2017). Big data fusion in IoT for smart cities applications: challenges and opportunities. *IEEE Access*, 5:13848–13864.
- Allison, P. D. (2001). *Missing data*, volume 136. Sage publications.
- Alpaydin, E. (2020). *Introduction to Machine Learning*. MIT Press, 4th edition.
- Apostol, T. M. (1974). *Mathematical Analysis*. Addison-Wesley, 2nd edition.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- Badat, A. (2020). A quick guide to auc-roc in machine learning models.
- Barlow, H. B. (1989). Unsupervised learning. *Neural Computation*, 1(3):295–311.
- Bengio, Y. (2009). Learning deep architectures for ai. *Foundations and Trends® in Machine Learning*, 2(1):1–127.
- Bengio, Y., Simard, P., and Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Bolaños-Martinez, D., Bermudez-Edo, M., and Garrido, J. L. (2024). Clustering pipeline for vehicle behavior in smart villages. *Information Fusion*, 104:102164.
- Boser, B. E., Guyon, I. M., and Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. In *Proceedings of the fifth annual workshop on Computational learning theory (COLT '92)*, pages 144–152.
- Box, G. E. P., Jenkins, G. M., Reinsel, G. C., and Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control*. Wiley, 5th edition.
- Boyd, S. and Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

- Bradley, A. P. (1997). The use of the area under the roc curve in the evaluation of machine learning algorithms. *Pattern Recognition*, 30(7):1145–1159.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.
- Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth & Brooks/Cole Advanced Books & Software.
- Burden, R. L. and Faires, J. D. (2011). *Numerical Analysis*. Brooks/Cole, 9th edition.
- Cao, W., Wang, D., Li, J., Zhou, H., Li, L., and Li, Y. (2018). BRITS: Bidirectional recurrent imputation for time series. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, pages 6775–6785.
- Casella, G. and Berger, R. L. (2002). *Statistical Inference*. Duxbury Press, 2nd edition.
- Chai, T. and Draxler, R. R. (2014a). Root mean square error (rmse) or mean absolute error (mae)? – arguments against avoiding rmse in the literature. *Geoscientific Model Development*, 7(3):1247–1250.
- Chai, T. and Draxler, R. R. (2014b). Root mean square error (RMSE) or mean absolute error (MAE)? – arguments against avoiding RMSE in the literature. *Geoscientific Model Development*, 7(3):1247–1250.
- Che, Z., Purushotham, S., Cho, K., Sontag, D., and Liu, Y. (2018). Recurrent neural networks for multivariate time series with missing values. *Scientific Reports*, 8(1):6085.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.
- Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3):273–297.
- Cover, T. M. and Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, IT-13(1):21–27.
- Cox, D. R. (1958). The regression analysis of binary sequences (with discussion). *Journal of the Royal Statistical Society, Series B (Methodological)*, 20(2):215–242.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314.
- De, S., Bermudez-Edo, M., Xu, H., and Cai, Z. (2022). Deep Generative Models in the Industrial Internet of Things: A Survey. *IEEE Transactions on Industrial Informatics*, 18(9):5728–5737.
- DeGroot, M. H. and Schervish, M. J. (2012). *Probability and Statistics*. Addison-Wesley, 4th edition.

- Dempster, A. P., Laird, N. M., and Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B (Methodological)*, 39(1):1–38.
- Donders, A. R. T., van der Heijden, G. J., Stijnen, T., and Moons, K. G. (2006). Review of methods for estimating missing data in comparative longitudinal research. *BMC medical research methodology*, 6(1):1–13.
- Drucker, H., Burges, C. J. C., Kaufman, L., Smola, A. J., and Vapnik, V. (1997). Support vector regression machines. In *Advances in Neural Information Processing Systems 9 (NIPS 1996)*, pages 155–161.
- Du, W. (2023). PyPOTS: a Python toolbox for data mining on Partially-Observed Time Series. *arXiv preprint arXiv:2305.18811*.
- Du, W., Cote, D., and Liu, Y. (2023a). SAITS: Self-Attention-based Imputation for Time Series. *Expert Systems with Applications*, 219:119619.
- Du, W., Cote, D., Liu, Y., Wang, X., and An, S. (2023b). SAITS: Self-attention based imputation for Time Series. *Expert Systems with Applications*, 219:119119.
- Enders, C. K. (2010). *Applied missing data analysis*. Guilford press.
- Ester, M., Kriegel, H.-P., Sander, J., and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, pages 226–231.
- Fawcett, T. (2006). An introduction to roc analysis. *Pattern Recognition Letters*, 27(8):861–874.
- Galton, F. (1886). Regression towards mediocrity in hereditary stature. *The Journal of the Anthropological Institute of Great Britain and Ireland*, 15:246–263.
- Glorot, X., Bordes, A., and Bengio, Y. (2011). Deep sparse rectifier neural networks. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS-11)*, pages 315–323.
- Golub, G. H. and Van Loan, C. F. (2013). *Matrix Computations*. Johns Hopkins University Press, 4th edition.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016a). *Deep Learning*. MIT Press.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016b). *Deep Learning*. MIT Press.
- Goutte, C. and Gaussier, E. (2005). A probabilistic interpretation of precision, recall and f-score, with implication for evaluation. In *Advances in Information Retrieval, 27th European Conference on IR Research, ECIR 2005*, pages 345–359.

- Gray, R. M. et al. (2006). Toeplitz and circulant matrices: A review. *Foundations and Trends® in Communications and Information Theory*, 2(3):155–239.
- Gubbi, J., Buyya, R., Marusic, S., and Palaniswami, M. (2013). Internet of things (iot): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7):1645–1660.
- Guillén Perales, A., Martínez Cámara, E., Fernández Luna, J. M., García Sánchez, P., Noguera García, M., Rodríguez Fórtiz, M. J., and Romero Zaliz, R. C. (2025). Cómo escribir la memoria de tu tfg del grado en ingeniería informática y presentarlo sin morir en el intento.
- Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2nd edition.
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778.
- Hinton, G. E. and Salakhutdinov, R. (2006). Reducing the dimensionality of data with neural networks. In *Science*, volume 313, pages 504–507.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441.
- Hyndman, R. J. and Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International journal of forecasting*, 22(4):679–688.
- James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013). *An Introduction to Statistical Learning: with Applications in R*. Springer.
- Japkowicz, N. and Shah, M. (2011). *Evaluating Learning Algorithms: A Classification Perspective*. Cambridge University Press.
- Johnson, S. C. (1967). Hierarchical clustering schemes. *Psychometrika*, 32(3):241–254.
- Jr., D. W. H., Lemeshow, S., and Sturdivant, R. X. (2013). *Applied Logistic Regression*. Wiley, 3rd edition.
- Kincaid, D. R. and Cheney, E. W. (2002). *Numerical Analysis: Mathematics of Scientific Computing*. Brooks/Cole, 3rd edition.
- Kolda, T. G. and Bader, B. W. (2009). Tensor decompositions and applications. *SIAM Review*, 51(3):455–500.
- Kreyszig, E. (1989). *Introductory Functional Analysis with Applications*. Wiley.

- LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521(7553):436–444.
- Lehmann, E. L. and Casella, G. (1998). *Theory of Point Estimation*. Springer, 2nd edition.
- Little, R. J. and Rubin, D. B. (2019). *Statistical analysis with missing data*. John Wiley & Sons, 3rd edition.
- Little, R. J. A. and Rubin, D. B. (2002). *Statistical Analysis with Missing Data*. John Wiley & Sons, 2nd edition.
- Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137.
- Luo, Y., Cai, X., Zhang, Y., Xu, J., and Yuan, X. (2018). E²GAN: End-to-end generative adversarial network for multivariate time series imputation. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI-18)*, pages 3094–3100.
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics*, pages 281–297.
- Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). Efficient estimation of word representations in vector space. In *Proceedings of the 1st International Conference on Learning Representations (ICLR 2013), Workshop Track*.
- Molenberghs, G., Fitzmaurice, G., Kenward, M., Tsiatis, A., and Verbeke, G., editors (2015). *Handbook of Missing Data Methodology*. CRC Press.
- Montgomery, D. C., Peck, E. A., and Vining, G. G. (2021). *Introduction to Linear Regression Analysis*. Wiley, 6th edition.
- Nagelkerke, N. J. D. (1991). A note on a general definition of the coefficient of determination. *Biometrika*, 78(3):691–692.
- Nair, V. and Hinton, G. E. (2010). Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, pages 807–814.
- Noble, B. and Daniel, J. W. (1988). *Applied Linear Algebra*. Prentice Hall, 3rd edition.
- Parzen, E. (1962). On estimation of a probability density function and mode. *The Annals of Mathematical Statistics*, 33(3):1065–1076.
- Pascanu, R., Mikolov, T., and Bengio, Y. (2013). On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning (ICML 2013)*, pages 1310–1318.

- Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine Series 6*, 2(11):559–572.
- Pennington, J., Socher, R., and Manning, C. D. (2014). GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543.
- Poudevigne, T. and Jones, O. (2024). Time-series imputation using low-rank matrix completion. Preprint.
- Powers, D. M. W. (2011). Evaluation: From precision, recall and f-measure to roc, informedness, markedness & correlation. *Journal of Machine Learning Technologies*, 2(1):37–63.
- Reyes-Ochoa (2020). Representaciones del aprendizaje reutilizando los gradientes de la retropropagacion.
- Rohatgi, V. K. and Saleh, A. K. M. E. (2015). *An Introduction to Probability and Statistics*. Wiley, 3rd edition.
- Rosenblatt, M. (1956). Remarks on some nonparametric estimates of a density function. *The Annals of Mathematical Statistics*, 27(3):832–837.
- Ross, S. M. (2014). *A First Course in Probability*. Pearson, 9th edition.
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3):581–592.
- Rudin, W. (1976). *Principles of Mathematical Analysis*. McGraw-Hill, 3rd edition.
- Rudin, W. (1987). *Real and Complex Analysis*. McGraw-Hill, 3rd edition.
- Rudin, W. (1991). *Functional Analysis*. McGraw-Hill, 2nd edition.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986a). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986b). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.
- Sandradewi, J., Prévôt, A. S. H., Szidat, S., Perron, N., Alfarra, M. R., Lanz, V. A., Weingartner, E., and Baltensperger, U. (2008). Source apportionment of carbonaceous aerosol using the Aethalometer model. *Atmospheric Chemistry and Physics*, 8(11):2749–2760.
- Schafer, J. L. (1997). *Analysis of incomplete multivariate data*. CRC press.
- Seinfeld, J. H. and Pandis, S. N. (2016). *Atmospheric chemistry and physics: from air pollution to climate change*. John Wiley & Sons, 3rd edition.

- Shao, J. and Zhong, B. (1992). Handling systematically missing data by the new-observation-carried-forward method in longitudinal clinical trials. *Statistics in medicine*, 11(16):2075–2085.
- Strang, G. (2016). *Introduction to Linear Algebra*. Wellesley-Cambridge Press, 5th edition.
- Universidad de Granada (2021). SmartPoqueira: Experimentación de tecnologías inteligentes para la gestión sostenible de las visitas a zonas rurales y parques nacionales. Sitio web oficial del proyecto. <https://wpd.ugr.es/smartpoqueira/>.
- Van Buuren, S. (2018). *Flexible imputation of missing data*. CRC press, 2nd edition.
- van der Maaten, L. and Hinton, G. (2008). Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017a). Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 5998–6008.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017b). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.
- Wallace, J. M. and Hobbs, P. V. (2006). *Atmospheric science: an introductory survey*, volume 92. Elsevier Academic Press, 2nd edition.
- Weisberg, S. (2005). *Applied Linear Regression*. Wiley, 3rd edition.
- Widom, H. (1966). Hankel matrices. *Transactions of the American Mathematical Society*, 121(1):1–35.
- Willmott, C. J. and Matsuura, K. (2005). Advantages of the mean absolute error (mae) over the root mean square error (rmse) in model performance evaluation. *Climate Research*, 30:79–82.